

Componentes/Interface/STICancelItem.PRW

294 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "POSCSS.CH"
4 #INCLUDE "STICANCELITEM.CH"
5 #INCLUDE "STPOS.CH"
6
7 Static oLblProdDados      := Nil                      // Label pro dados
8 Static cGetRsnCanIt       := Space(50)
9 Static oReasons           := Nil                      // ListBox Seleção de Motivo de Cancelamento
10 Static lMotVen            := SuperGetMV( "MV_LJMVPE", Nil, .F. ) // Motivo de venda perdida
11 Static lClientHtml       := GetCliHTML()             //Totvs PDV Web
12
13
14 //-----
15 /*{Protheus.doc} STICancelItem
16 Interface Cancelamento de Item
17
18 @param
19 @return
20 @obs
21 @sample
22 */
23 //-----
24 Function STICancelItem()
25
26 Local oTotal              := STFGetTot()              // Recebe o Objeto totalizador
27 Local nTotalVend          := oTotal:GetValue("L1_VLRTOT") // Valor total da venda
28
29 If nTotalVend > 0
30     /*
31         Chamo workflow que avalia qual a forma de cancelamento,
32         sendo que é possível excluir qualquer item ou apenas o último registrado.
33     */
34     STWChkCancel()
35     STIBtnDeActivate()
36 Else
37     STFMessage(ProcName(),"STOP",STR0008) // "Nenhum item foi registrado!"
38     STFShowMessage(ProcName())
39 EndIf
40
41 Return
42
43 //-----
44 /*{Protheus.doc} STIPanItCancel
45 Contexto Cancelamento de Item
46
47 @param
48 @return oPanConsult          Painel Consulta de Produto
49 @obs
50 @sample
51 */
52 //-----

```

```

53 Function STIPanItCancel()
54
55 Local oPanelMVC      := Nil           // Painel Principal
56 Local oPanGetCancel  := Nil           // Painel de Get de Consulta
57 Local oGetCancel     := Nil           // TGet Cancelamento de Item
58 Local cGetProd       := Space(40)     // Variável do Get
59
60 Local oButton        := Nil           // Botao "Consultar Produto"
61 Local oLblCab        := Nil           // Label Cabecario
62 Local oLblItem       := Nil           // Label Itens
63 Local oLblDados      := Nil           // Label com a primeira coluna de dados
64 Local cDados         := ""            // String da primeira coluna de dados
65 Local lMotVen        := SuperGetMV( "MV_LJMVPE", Nil, .F. ) //Motivo de venda perdida
66 Local nPosVert       := 0             // Posicao vertical
67 Local aReasons       := {}            // Retorna Motivos de Venda Perdida para seleção
68 Local aDataReasons   := STDRFLSGet()  // Armazena Motivos de Venda Perdida Cadastrados
69 Local nI             := 0             // Contador
70 Local nCol           := 0             //Coordenada horizontal
71 Local nLargura       := 0             //Largura
72 Local nTamAltListBox := 0             //Tamanho: Altura do ListBox
73 Local oButCan        := Nil           //Botao cancelar
74 Local cLabel         := ""
75 Local nTpPesquisa    := SuperGetMv("MV_LJTPESQ",, 1) //Define tipo de pesquisa no cancelamento do produto, 0 sera por Numero do Item
76 Local lCancel        := .T.          // indica que eh cancelamento para a pesquisa de produto, assim a pesquisa de produto nao pedira peso para
    produto balanca
77 Local oSayRsnCIt     := Nil
78 Local oGetObsRsnCIt  := Nil
79 Local cSaltoLinhas   := Iif(lMotVen .And. lClientHtml,CRLF,CRLF+CRLF )
80 Local lMataObj       := ExistFunc("StiMataObj")
81
82 oPanelMVC            := STIGetPanel() // Painel Principal
83 oPanGetCancel := TPanel():New(00,00,"",oPanelMVC,,,,,oPanelMVC:nWidth/2,(oPanelMVC:nHeight)/2) // Painel de Get de Consulta
84 nPosVert           := oPanelMVC:nHeight/3.365
85 nCol               := (oPanGetCancel:nWidth / 2) * 0.03 //Coordenada horizontal
86 nLargura           := (oPanGetCancel:nWidth / 2) - (2 * nCol) //Largura
87 nTamAltListBox     := oPanelMVC:nHeight * 0.11 //Tamanho: Altura do ListBox
88
89 cLabel := iif(nTpPesquisa==1,STR0017,STR0010) // STR0017 =="Produto" STR0010 =="Número do Item"
90 /*
91   Objetos
92 */
93 oLblCab := TSay():New(POSHOR_CAB,POSHOR_1,{||STR0009},oPanGetCancel,,,,,.T.,,,) //"Venda > Cancelamento de Item"
94 oLblCab:SetCSS( POSCSS (GetClassName(oLblCab), CSS_BREADCUMB ))
95
96 oLblItem := TSay():New(15,POSHOR_1,{||cLabel},oPanGetCancel,,,,,.T.,,,)
97 oLblItem:SetCSS( POSCSS (GetClassName(oLblItem), CSS_LABEL_FOCAL ))
98
99 oGetCancel := TGet():New( 12,POSHOR_1+130,{|u| If(PCount())>0,cGetProd:=u,cGetProd}}, ;
100     oPanGetCancel,150,ALTURAGET,,,,,,.T.,,,,,,"cGetProd")
101 oGetCancel:SetCSS( POSCSS (GetClassName(oGetCancel), CSS_GET_FOCAL ))
102
103 cDados := STR0012+cSaltoLinhas+STR0013+cSaltoLinhas+STR0014+cSaltoLinhas+STR0015 // "Item no." + "Código" + "Descrição do Produto" + "Qtd x Preço unitário = Total do Item"

```

```

104
105 oLbLDados:= TSay():New(30,POSHOR_1,{||cDados},oPanGetCancel,,,,,T,,,,80)
106 oLbLDados:SetCSS( POSCSS (GetClassName(oLbLDados), CSS_LABEL_FOCAL ))
107
108 /* Motivo de Nao Venda */
109 If lMotVen
110
111     FreeObj(oReasons)
112
113     For nI := 1 To Len(aDataReasons)
114
115         AADD( aReasons , aDataReasons[nI][1] + " - " + UPPER(aDataReasons[nI][2]) )
116
117     Next nI
118
119     If Len(aReasons) > 0 .AND. ValType(oPanGetCancel) <> "U"
120
121         oReasons := TListBox():Create(oPanGetCancel, POSVERT_GET3 * 0.85 ,POSHOR_1, Nil, aReasons, nLargura, nTamAltListBox,,,,,T,,,/*{ || STISetReason(oReasons),
oButton:SetFocus() }*/)
122         oReasons:SetCSS( POSCSS (GetClassName(oReasons), CSS_LISTBOX ))
123
124         oSayRsnCit := TSay():New(POSVERT_GET3 * 1.30 ,POSHOR_1,{||STR0018},oPanGetCancel,,,,,T,,,,) //Justificativa
125         oSayRsnCit:SetCSS( POSCSS (GetClassName(oSayRsnCit), CSS_LABEL_FOCAL ))
126
127         oGetObsRsnCit:= TGet():New(POSVERT_GET3 * 1.39, POSHOR_1,{||u| If(PCount()>0,cGetRsnCanIt:=u,cGetRsnCanIt)},,
oPanGetCancel,nLargura,12,/*bValid*/,,,,,T,,,,,F,,,cGetRsnCanIt",,,,,,T.)
128         oGetObsRsnCit:SetCSS( POSCSS (GetClassName(oGetObsRsnCit), CSS_GET_FOCAL ))
129
130     EndIf
131
132 EndIf
133 EndIf
134
135 oGetCancel:bGotFocus := { || STWReaderFocus(.T.) }
136 oGetCancel:bLostFocus := { || STWReaderFocus(.F.),STIShowProdData(cGetProd,oPanGetCancel,lCancel),oButton:SetFocus()}
137
138 oButton := TButton():New( POSVERT_BTNFOCAL,POSHOR_BTNFOCAL,STR0007,oPanGetCancel,{ || IIf(STWItemCancel(cGetProd , oReasons,lCancel,,cGetRsnCanIt ),(oLbLProdDados :=
Nil, Iif(lMataObj,StiMataObj(oPanGetCancel),Nil), STIRegItemInterface()),Nil) }, ; //"Cancelar Item"
LARGBTN,ALTURABTN,,,,T.)
139
140 oButton:SetCSS( POSCSS (GetClassName(oButton), CSS_BTN_FOCAL ))
141
142 oButCan := TButton():New( POSVERT_BTNFOCAL,POSHOR_1,STR0016,oPanGetCancel,{||Iif(lMataObj, STIMataObj(oPanGetCancel), Nil), STIRegItemInterface() }, ; //"Não Cancelar"
LARGBTN,ALTURABTN,,,,T.,,{||.T.})
143
144 oButCan:SetCSS( POSCSS (GetClassName(oButCan), CSS_BTN_NORMAL ))
145
146
147 oButCan:bLostFocus := {|| Iif(oButton <> Nil, oButton:SetFocus(),Nil)} // Tratamento para não perder o foco do Cancelar Item
148
149
150 oGetCancel:SetFocus()
151
152 Iif (GetFocus() <> oGetCancel::HWND, oGetCancel:SetFocus(),Nil)
153 oLbLProdDados := Nil

```

```

154
155 Return oPanGetCancel
156
157 //-----
158 /*{Protheus.doc} STIShowProdData
159 Mostra na tela as informacoes do item que sera cancelado
160
161 @param
162 @return Nil
163 @obs
164 @sample
165 */
166 //-----
167 Static Function STIShowProdData(cGetProd,oPanGetCancel,lCancel)
168
169 Local oPanelMVC      := STIGetPanel()      // Painei Principal
170 Local aLines         := FwSaveRows()      // Array de linhas
171 Local oModelCesta    := STDGPBModel()     // Model da cesta
172 Local nItem          := 0
173 Local cProdData      := ""
174 Local cMoedaSimb     := SuperGetMV( "MV_SIMB" + Str(STBGetCurrency(),1) ) // Simbolo da moeda corrente
175 Local lContinua      := .F.
176 Local cSaltoLinhas   := Iif(lMotVen .And. lClientHtml,CRLF,CRLF+CRLF)
177
178 DEFAULT cGetProd := ""
179 DEFAULT lCancel   := .F. // indica que eh cancelamento para a pesquisa de produto, assim a pesquisa de produto nao pedira peso para produto balanca
180
181 nItem             := STBCnFindItem( cGetProd ,lCancel) //Procura Cod. produto recebido pois pode ser um codigo de barras , e passa indicador se vem ou nao de um
                    cancelamento
182
183 oModelCesta := oModelCesta:GetModel("SL2DETAIL")
184
185 If !Empty(cGetProd) .AND. nItem > 0
186     lContinua := STBValCnItem( nItem , oModelCesta )
187
188     If lContinua
189
190         oModelCesta:GoLine(nItem)
191         cProdData := AllTrim(oModelCesta:GetValue("L2_ITEM"))+ cSaltoLinhas
192         cProdData += AllTrim(oModelCesta:GetValue("L2_PRODUTO"))+ cSaltoLinhas
193         cProdData += AllTrim(oModelCesta:GetValue("L2_DESCRI"))+ cSaltoLinhas
194         cProdData += AllTrim(Str(oModelCesta:GetValue("L2_QUANT")))+ " x " +
195             cMoedaSimb+AllTrim(Transform(oModelCesta:GetValue("L2_VRUNIT"),PesqPict("SL2","L2_VRUNIT")))+ " = " +
196             cMoedaSimb+AllTrim(Transform(oModelCesta:GetValue("L2_VLRITEM"),PesqPict("SL2","L2_VLRITEM")))
197
198         If ValType(oLblProdDados) <> "U"
199             oLblProdDados:SetText(cProdData)
200         Else
201             oLblProdDados:= TSay():New(30,POSHOR_1+130,{||cProdData},oPanGetCancel,,,,,T,,,,,100)
202             oLblProdDados:SetCSS( POSCSS (GetClassName(oLblProdDados), CSS_LABEL_FOCAL ))
203         EndIf
204     EndIf

```

```

205 EndIf
206
207 FwRestRows(aLines)
208
209 Return
210
211 //-----
212 /*{Protheus.doc} STILastItCancel
213 Interface de Cancelamento do Ultimo Item
214
215 @param
216 @return  oPanGetCancel          Painel Cancelamento de Produto
217 @obs
218 @sample
219 */
220 //-----
221 Function STILastItCancel()
222 Local oPanelMVC      := STIGetPanel()                // Painel Principal
223 Local oPanGetCancel := TPanel():New(00,00,"",oPanelMVC,,,,,oPanelMVC:nWidth/2,(oPanelMVC:nHeight)/2) // Painel do Cancelamento
224 Local oLblCab       := Nil                          // Label do Cabecalho
225 Local oModelCesta   := STDGPBModel()                // Model da cesta
226 Local cMoedaSimb     := SuperGetMV( "MV_SIMB" + Str(STBGetCurrency(),1 ) ) // Simbolo da moeda corrente
227 Local oLblDados1    := Nil                          // Label com a primeira coluna de dados
228 Local oLblDados2    := Nil                          // Label com a segunda coluna de dados
229 Local cDados1        := ""                          // String da primeira coluna de dados
230 Local cDados2        := ""                          // String da segunda coluna de dados
231
232 oLblCab:= TSay():New(POSVERT_CAB,POSHOR_1,{||STR0011},oPanGetCancel,,,,,T,,,,) //"Apenas o último item poderá ser cancelado. Confirma?"
233 oLblCab:SetCSS( POSCSS (GetClassName(oLblCab), CSS_BREADCUMB ))
234
235 oModelCesta := oModelCesta:GetModel("SL2DETAIL")
236 oModelCesta:GoLine(oModelCesta:Length())
237
238 /* Item */
239 cDados1 := STR0012+CRLF+CRLF //"Item nº"
240 cDados2 := AllTrim(oModelCesta:GetValue("L2_ITEM"))+CRLF+CRLF
241
242 /* Código */
243 cDados1 += STR0013+CRLF+CRLF //"Código"
244 cDados2 += AllTrim(oModelCesta:GetValue("L2_PRODUTO"))+CRLF+CRLF
245
246 /* Descrição */
247 cDados1 += STR0014+CRLF+CRLF //"Descrição do Produto"
248 cDados2 += AllTrim(oModelCesta:GetValue("L2_DESCRI"))+CRLF+CRLF
249
250 /* Quantidade x Preço unitário = Total */
251 cDados1 += STR0015 //"Qtd x Preço unitário = Total do Item"
252 cDados2 += AllTrim(Str(oModelCesta:GetValue("L2_QUANT"))+" x "+;
253             cMoedaSimb+AllTrim(Transform(oModelCesta:GetValue("L2_VRUNIT"),PesqPict("SL2","L2_VRUNIT") ))+" = "+;
254             cMoedaSimb+AllTrim(Transform(oModelCesta:GetValue("L2_VLRITEM"),PesqPict("SL2","L2_VLRITEM"))))
255
256 oLblDados1:= TSay():New(POSVERT_LABEL1,POSHOR_1,{||cDados1},oPanGetCancel,,,,,T,,,,100)

```

```

257 oLbldados1:SetCSS( POSCSS (GetClassName(oLbldados1), CSS_LABEL_FOCAL ))
258
259 oLbldados2:= TSay():New(POSVERT_LABEL1,POSHOR_1+130,{||cDados2},oPanGetCancel,,,,,T,,,,100)
260 oLbldados2:SetCSS( POSCSS (GetClassName(oLbldados2), CSS_LABEL_FOCAL ))
261
262 oBtnOk := TButton():New(POSVERT_BTNFOCAL,POSHOR_BTNFOCAL,STR0007,oPanGetCancel,{|| STWLastCancel(),STIRegItemInterface() }, ; //"Cancelar Item"
263           LARGBTN,ALTURABTN,,,,T.)
264
265 oBtnCa := TButton():New(POSVERT_BTNFOCAL,POSHOR_1,STR0016,oPanGetCancel,{ || STIRegItemInterface()} , ; //"Não Cancelar"
266           LARGBTN,ALTURABTN,,,,T.)
267
268 oBtnOk:SetCSS( POSCSS (GetClassName(oBtnOk), CSS_BTN_FOCAL ))
269 oBtnCa:SetCSS( POSCSS (GetClassName(oBtnCa), CSS_BTN_ATIVO ))
270
271 Return oPanGetCancel
272
273 //-----
274 /*/{Protheus.doc} STIGTRsnCanIt
275 Propriedade GET para retornar o oReasons e cGetRsnCanIt
276
277 @return {oReasons, cGetRsnCanIt}, array, array com objeto objeto motivo da perda de venda e justificativa.
278 */
279 //-----
280 Function STIGTRsnlstIt()
281 Return {oReasons, cGetRsnCanIt}
282
283
284 //-----
285 /*/{Protheus.doc} STISTRsnCanIt
286 Propriedade SET para ZERAR o oReasons e cGetRsnCanIt
287
288 @return Nil
289 */
290 //-----
291 Function STISTRsnlstIt()
292 FreeObj(oReasons)
293 cGetRsnCanIt:=Space(50)
294 Return

```

Componentes/Interface/STICancelSale.PRW

497 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "POSCSS.CH"
4 #INCLUDE "STICANCELSALE.CH"
5 #INCLUDE "STPOS.CH"
6
7 Static aCancel          := {}
8 Static lPdvOn           := ExistFunc("STFPdvOn") .AND. STFPdvOn() // Armazena informações para cancelamento
9 Static oRsnCanCup       := Nil // Verifica se é PDV-Online
10 Static cGetRsnCanCup    := Space(50) // ListBox Seleção de Motivo da perda de venda
11 // Observação do motivo da perda de venda
12
13 //-----
14 /*/{Protheus.doc} STIGetCancel
15 Recupera informações cancelamento
16
17 @param
18 @return aCancel          Retorna informações cancelamento
19 @obs
20 @sample
21 */
22 //-----
23 Function STIGetCancel()
24
25 Return aCancel
26
27 //-----
28 /*/{Protheus.doc} STISetCancel
29 Adiciona informações cancelamento
30
31 @param aInfCancel        Array com informações do cancelamento
32 @return aCancel          Retorna informações cancelamento
33 @obs
34 @sample
35 */
36 //-----
37 Function STISetCancel( aInfCancel )
38
39 aCancel := aInfCancel
40
41 Return
42
43 //-----
44 /*/{Protheus.doc} STICancelSale
45 Chamada Interface Cancelamento de Venda
46
47 @param cGetCanc, string,
48 @return nenhum
49
50 */
51 //-----
52 Function STICancelSale(cGetCanc, cGetSerie, cLInum)

```

```

53 Local lRet      := .F.      // Retorna se pode cancelar a venda
54 Local aValidCancel := {}    // Armazena informações para cancelamento da venda
55 Local lEmitNFCe   := STBGetNFCe() // Indica se NFC-e
56 Local nTamDoc     := 0
57 Local cAuxDoc     := ""
58
59 Default cGetCanc  := ""
60 Default cL1Num    := ""
61
62
63 If !lEmitNFCe .AND. !Empty(cGetCanc)
64
65     LjGrvLog( "STICancelSale", "Antes de capturar o DOC para avaliar o tamanho")
66     STWGetReceipt( @cAuxDoc )
67     LjGrvLog( "STICancelSale", "Depois de capturar o DOC para avaliar o tamanho",cAuxDoc)
68
69     nTamDoc := Len(cAuxDoc)
70     LjGrvLog( "STICancelSale", "Tamanho do DOC - provindo do ECF", nTamDoc)
71     cGetCanc := StrZero( Val(cGetCanc), nTamDoc ) //tamanho fixo para C00 do ECF
72 EndIf
73
74 aValidCancel:= STBCScanCancel(cGetCanc,,cGetSerie, cL1Num)
75 lRet      := aValidCancel[1]
76
77 If lRet
78     /*/
79     Seta Informações cancelamento
80     /*/
81     STISetCancel( aValidCancel )
82
83     If !Empty(cGetCanc)
84         STICancel(cGetCanc)
85     Elseif !lEmitNFCe
86         STIExchangePanel( { || STIPanSaleCancel() } )
87     EndIf
88
89 EndIf
90
91 Return
92
93 //-----
94 /*/{Protheus.doc} STIGerCancel
95 Contexto Cancelar de Venda
96
97 @param  nenhum
98 @return nenhum
99 */
100 //-----
101 Function STIGerCancel()
102
103 Local aValidCancel := {}    // Armazena informações para cancelamento da venda
104 Local lRet         := .F.

```

```

105
106 aValidCancel    := STBCSCanCancel(,.F.)
107 lRet             := aValidCancel[1]
108
109 If lRet
110     STISetCancel( aValidCancel )
111     STIExchangePanel( { || STIPanSaleCancel() } )
112 EndIf
113
114 Return
115
116 //-----
117 /*/{Protheus.doc} STIPanSaleCancel
118 Contexto Cancelar de Venda
119
120 @param nenhum
121 @return oPanSaleCancel, objeto
122 @obs
123 @sample
124 */
125 //-----
126 Function STIPanSaleCancel()
127 Local oPanSaleCancel := Nil           // Painel Cancelamento de Venda
128 Local oPanelMVC      := STIGetPanel() // Painel Principal
129 Local oBtnConfirm    := Nil           // Botao de confirmação "Cancelar Venda"
130 Local oBtnCancel     := Nil
131 Local cTotalSale     := ""            // Valor total da venda
132 Local cNumberSale    := ""            // Numeração da venda
133 Local oGetDoc        := Nil           // Get do Doc a ser cancelado
134 Local cMoedaSimb     := SuperGetMV( "MV_SIMB" + Str(STBGetCurrency() ,1 ) ) // Simbolo da moeda corrente
135 Local cPictTotalSale := PesqPict("SL1","L1_VLRTOT")
136 Local lEmitNfce      := LjEmitNfCe() // Sinaliza se utiliza NFC-e
137 Local nGetHeiPerc    := 90            // Percentual da Altura do Get de Busca
138 Local oLblCancel     := Nil           // Objeto Label do cancelamento
139 Local cGetCanc       := Space(TamSX3("L1_DOC")[1]) // Conteudo inserido no Get de Cancelamento.
140 Local lSAT           := STFGetCfg("lUseSAT",.F.)
141 Local cDocAnt        := ""            //Ultimo documento com Total
142 Local oSay1          := NIL //say 1 - Confirma o Cancelamento da Venda?"
143 Local oSay3          := NIL //say 3 - "Num. da Venda
144 Local oSay4          := NIL //say 4 - "Num. da Venda
145 Local oSay6          := NIL //say 6 - "Valor Total da Venda"
146 Local oSay5          := NIL //say 5 - "Valor Total da Venda"
147 Local cGetSerie      := STFGetStation("SERIE")
148 Local oGetSerie      := Nil
149 Local cSerConf       := GetNewPar("MV_LOJANF","UNI")
150 Local lNfFeActive    := SuperGetMV("MV_FISNOTA", .F., .F.) .and. !Empty(cSerConf) .and. cSerConf <> "UNI"
151 Local aReasons       := {}            // Retorna Motivos de Venda Perdida para seleção
152 Local aDataReasons   := {}
153 Local nLargura       := 0            //Largura ListBox
154 Local nTamAltListBox := 0            //Tamanho: Altura do ListBox
155 Local nCol           := 0
156 Local nI             := 0

```

```

157 Local oGetObsRsnCCup      := NIL
158 Local oSayRsnCCup         := NIL
159 Local lSTIMataObj         := ExistFunc("StiMataObj")
160
161 if lNFeActive .OR. lSAT
162     cGetCanc := STDLastSat("L1_DOC", @cGetSerie)
163 else
164     cGetCanc := STDCSLastSale("L1_DOC", @cGetSerie)
165 EndIf
166
167 //Cancelamento RPS
168 If (lEmitNfce .OR. lSAT) .AND. Empty(cGetCanc) .AND. Empty(cGetSerie) .AND. Len(aCancel)>=11 .AND. !Empty(aCancel[10]) .AND. !Empty(aCancel[11])
169     cGetCanc := aCancel[10]
170     cGetSerie := aCancel[11]
171 Endif
172
173 /*/
174 Panel Abertura de Caixa
175 /*/
176 oPanSaleCancel := TPanel():New(00,00,"",oPanelMVC,,,,,oPanelMVC:nWidth/2,(oPanelMVC:nHeight)/2)
177 oPanSaleCancel:bGotFocus := { || STWReaderFocus(.T.) }
178 oPanSaleCancel:bLostFocus := { || STWReaderFocus(.F.),STIShowProdData(cGetCanc,oPanSaleCancel,lCancel),oBtnConfirm:SetFocus() }
179
180 oSay1 := TSay():New(POSVERT_CAB,POSHOR_1,{||STR0002},oPanSaleCancel,,,,,.T.,,,) //"Confirma o Cancelamento da Venda?"
181 oSay1:SetCSS( POSCSS (GetClassName(oSay1), CSS_BREADCUMB ))
182
183 oBtnConfirm := TButton():New( POSVERT_BTNFOCAL,POSHOR_BTNFOCAL,STR0008,oPanSaleCancel,;
184     { || Iif(lSTIMataObj, StiMataObj(oPanSaleCancel),Nil), IIF(ExistFunc("STBActionCancel"),STBActionCancel(cGetCanc,cGetSerie), .F. ) }, ;
185     //"Cancelar Venda"
186     LARGBTN,ALTURABTN,,,,.T.)
187 oBtnConfirm:SetCSS( POSCSS (GetClassName(oBtnConfirm), CSS_BTN_FOCAL ))
188
189 oBtnCancel := TButton():New( POSVERT_BTNFOCAL,POSHOR_1,STR0009,oPanSaleCancel,{ ||Iif(lSTIMataObj, StiMataObj(oPanSaleCancel),Nil), Iif(ExistFunc("STDRestCanL1"),
190     STDRestCanL1(),Nil) , Iif(STIRInPay(oPanSaleCancel),STIRegItemInterface(),Nil) }, ; //"Não Cancelar"
191     LARGBTN,ALTURABTN,,,,.T.)
192 oBtnCancel:SetCSS( POSCSS (GetClassName(oBtnCancel), CSS_BTN_NORMAL ))
193
194 oBtnCancel:bLostFocus := {|| Iif(oBtnConfirm <> Nil ,oBtnConfirm:SetFocus(),Nil) } // Tratamento para não perder o foco do Cancelar VENDA
195
196 oBtnConfirm:SetFocus()
197
198 If lEmitNfce .AND. !STBCSIsProgressSale()
199     //Preenche o DOC da ultima venda
200     If !lSAT
201         cGetCanc := aCancel[4]
202         If Empty(cGetCanc) .AND. Len(aCancel) > 5
203             If Len(aCancel)>=11 .AND. !Empty(aCancel[10]) .AND. !Empty(aCancel[11]) //Cancelamento RPS
204                 cGetCanc := aCancel[10]
205                 cGetSerie := aCancel[11]
206             Else
207                 cGetCanc := aCancel[6]

```

```

207         Endif
208     EndIf
209 EndIf
210
211 If !Empty(cGetCanc)
212     cTotalSale := cMoedaSimb+AllTrim(Transform(STBCSTotalSale(.F.,;
213                                     STDCSNum(cGetCanc, STFGetStation("PDV"),IIF((Len(aCancel) > 5 .AND. !Empty(aCancel[6])));
214                                     .OR. (Len(aCancel) >=11 .AND. !Empty(aCancel[10]) .AND. !Empty(aCancel[11])),.T.,.F.),;
215                                     cGetSerie, Iif(lPdvOn .AND. Len(aCancel) >=11 .AND. !Empty(aCancel[5]) ,aCancel[5], "" ))),;
216                                     cPictTotalSale))
217 EndIf
218
219 oSay6 := TSay():New(POSVERT_GET1,POSHOR_2,{||cTotalSale},oPanSaleCancel,,,,,.T.,,,)
220 oSay6:SetCSS( POSCSS (GetClassName(oSay6), CSS_LABEL_FOCAL ))
221
222 oLblCancel := TSay():New(POSVERT_LABEL1,POSHOR_1,{|| STR0012 },oPanSaleCancel,,,,,.T.,,,) //"Informe a série e o número do Documento Fiscal"
223 oLblCancel:SetCSS( POSCSS (GetClassName(oLblCancel), CSS_LABEL_FOCAL ))
224
225 oGetSerie:= TGet():New(POSVERT_GET1,POSHOR_1,{||u| If(PCount())>0,cGetSerie:=u,cGetSerie}},;
226                     oPanSaleCancel,nGetHeiPerc-70,,,/*bValid*/,,,,,.T.,,,,,,"cGetSerie")
227 oGetSerie:SetCSS( POSCSS (GetClassName(oGetSerie), CSS_GET_FOCAL ))
228
229 oGetDoc := TGet():New(POSVERT_GET1,POSHOR_1 + 30,{||u| If(PCount())>0,cGetCanc:=u,cGetCanc}},;
230                     oPanSaleCancel,nGetHeiPerc,,,/*bValid*/ ,,,,,.T.,,,,,,"cGetCanc")
231 oGetDoc:SetCSS( POSCSS (GetClassName(oGetDoc), CSS_GET_FOCAL ))
232 cDocAnt := cGetCanc
233
234 oSay5 := TSay():New(POSVERT_LABEL1,POSHOR_2,{||STR0007},oPanSaleCancel,,,,,.T.,,,) //"Valor Total da Venda"
235 oSay5:SetCSS( POSCSS (GetClassName(oSay5), CSS_LABEL_FOCAL ))
236
237 oGetDoc:LostFocus := { || IF( cDocAnt <> cGetCanc, ;
238                             ( IIF( !Empty(cGetCanc), ;
239                                 cTotalSale := cMoedaSimb+AllTrim(Transform(STBCSTotalSale(.F.,STDCSNum(cGetCanc,
240
241                                     ""), ;
242                                     oSay6:Refresh(), cDocAnt := cGetCanc;
243
244                                     ),;
245                                 ),;
246                                 oBtnConfirm:SetFocus() }
247
248 oGetSerie:SetFocus() //Tratamento para não perder o foco no campo Série
249 Else
250
251 If !lEmitNfce .Or. lSAT
252     /*/
253     Numeração da Venda
254     /*/
255     cNumberSale := STBCSNumberSale(aCancel[2],aCancel[4])
256
257     oSay3 := TSay():New(POSVERT_LABEL1,POSHOR_1,{||STR0006},oPanSaleCancel,,,,,.T.,,,) //"Num. da Venda"

```

```

258         oSay3:SetCSS( POSCSS (GetClassName(oSay3), CSS_LABEL_FOCAL ))
259
260         oSay4 := TSay():New(POSVERT_GET1,POSHOR_1,{||cNumberSale},oPanSaleCancel,,,,,T,,,,)
261         oSay4:SetCSS( POSCSS (GetClassName(oSay4), CSS_LABEL_FOCAL ))
262
263     EndIf
264     /*/
265     Valor total da venda
266     /*/
267     cTotalSale := cMoedaSimb+AllTrim(Transform(STBCSTotalSale(aCancel[2],aCancel[5]),cPictTotalSale))
268
269     oSay5 := TSay():New(POSVERT_LABEL1,POSHOR_2,{||STR0007},oPanSaleCancel,,,,,T,,,,) //"Valor Total da Venda"
270     oSay5:SetCSS( POSCSS (GetClassName(oSay5), CSS_LABEL_FOCAL ))
271
272     oSay6 := TSay():New(POSVERT_GET1,POSHOR_2,{||cTotalSale},oPanSaleCancel,,,,,T,,,,)
273     oSay6:SetCSS( POSCSS (GetClassName(oSay6), CSS_LABEL_FOCAL ))
274     oBtnConfirm:SetFocus()
275
276 EndIf
277
278 If SuperGetMV( "MV_LJMVPE", Nil, .F. ) //motivo da venda perdida
279
280     nCol           := (oPanSaleCancel:nWidth / 2) * 0.03           //Coordenada horizontal
281     nLargura       := (oPanSaleCancel:nWidth / 2) - (2 * nCol) //Largura
282     nTamAltListBox := oPanelMVC:nHeight * 0.11
283
284     FreeObj(oRsnCanCup)
285     aDataReasons := STDRFLSGet()
286
287     For nI := 1 To Len(aDataReasons)
288         AADD( aReasons , aDataReasons[nI][1] + " - " + UPPER(aDataReasons[nI][2]) )
289     Next nI
290
291     If Len(aReasons) > 0
292         oRsnCanCup:= TListBox():Create(oPanSaleCancel, POSVERT_GET3 * 0.80, POSHOR_1, Nil, aReasons, nLargura, nTamAltListBox,,,,,T,,,/*{ || STISetReason(oReasons),
oButton:SetFocus() }*/)
293         oRsnCanCup:SetCSS( POSCSS (GetClassName(oRsnCanCup), CSS_LISTBOX ))
294
295         oSayRsnCCup := TSay():New(POSVERT_GET3 * 1.28 ,POSHOR_1,{||STR0013},oPanSaleCancel,,,,,T,,,,)//Justificativa
296         oSayRsnCCup:SetCSS( POSCSS (GetClassName(oSayRsnCCup), CSS_LABEL_FOCAL ))
297
298         oGetObsRsnCCup:= TGet():New(POSVERT_GET3 * 1.38, POSHOR_1,{||u| If(PCount()>0,cGetRsnCanCup:=u,cGetRsnCanCup)},;
oPanSaleCancel,nLargura,12,/*bValid*/,,,,,T,,,,,F,,,cGetRsnCanCup")
299         oGetObsRsnCCup:SetCSS( POSCSS (GetClassName(oGetObsRsnCCup), CSS_GET_FOCAL ))
300
301     EndIf
302 EndIf
303
304
305
306 Return oPanSaleCancel
307
308

```

```

309 //-----
310 /*{Protheus.doc} STICanCancel
311 Validação Interface Cancelar Venda
312
313 @param
314 @return lRet          Retorna se pode cancela a venda
315 @obs
316 @sample
317 */
318 //-----
319 Function STICanCancel()
320 Local lRet          := .F.          // Retorna se pode cancela a venda
321 Local cPergunte     := ""          // Armazena mensagem de confirmação
322 Local aValidCancel  := {}          // Armazena informações para cancelamento da venda
323
324 aValidCancel       := STBCSCanCancel()
325 lRet                := aValidCancel[1]
326
327 If lRet
328     /*
329     Seta Informações cancelamento
330     */
331     STISetCancel( aValidCancel )
332
333     /*
334     Chamada Pergunte Wizard
335     */
336     cPergunte := STBCSQuestion( aCancel[2] , Nil , aCancel[4] )
337
338 EndIf
339
340
341 Return lRet
342
343
344 //-----
345 /*{Protheus.doc} STICSExit
346 Aborta Cancelamento de Venda
347
348 @param oCancel        ListBox de Confirmação de Cancelamento
349 @return Nil
350 @obs
351 @sample
352 */
353 //-----
354 Function STICSExit()
355
356 /*
357 Limpa variável
358 */
359 STISetCancel( {} )
360

```

```

361 STIExchangePanel( { || STIPanItemRegister() } )
362
363 Return Nil
364
365
366 //-----
367 /*{Protheus.doc} STICancel
368 Acionado Cancelamento de Venda
369
370 @param
371 @return Nil
372 @obs
373 @sample
374 */
375 //-----
376 Function STICancel(cGetCanc,cNFisCanc)
377 Local aInfCancel      := {}           // Armazena informações cancelamento
378 Local lUseSat         := STFGetCfg("lUseSAT", .F.) //Utiliza SAT
379 Local lAutomato       := If(Type("lAutomatoX")<>"L",.F.,lAutomatoX)
380 Local lNFeAtv         := .F.
381 Local lUsaPD          := .F.
382 Local lDocRPS         := .F. //Cancelamento RPS
383
384 Default cGetCanc := ""
385
386 aInfCancel := STIGetCancel()
387 lNFeAtv    := len(aInfCancel) > 6
388 lUsaPD     := len(aInfCancel) >= 9
389 lDocRPS    := Len(aInfCancel) >= 11 .AND. !Empty(aInfCancel[10]) .AND. !Empty(aInfCancel[11])
390
391 If !lUseSat
392     STWCancelSale( .F.           , ;           // Força cancelamento?
393                   aInfCancel[2] , ;           // Venda em andamento?
394                   aInfCancel[3] , ;           // Superior que autorizou cancelamento
395                   aInfCancel[4] , ;           // Numero Doc a Cancelar (L1_DOC)
396                   aInfCancel[5] , ;           // Numeração venda a cancelar (L1_NUM)
397                   , ;
398                   , ;
399                   , ;
400                   iif( lNFeAtv, aInfCancel[7],"") , ;           //-- TBC-G0 Serie da nota
401                   iif(lUsaPD,aInfCancel[8],.F.) , ;           // -- Usa CC/CD
402                   iif(lUsaPD,aInfCancel[9],.F.) , ;           // -- Usa PD
403                   lDocRPS )
404 ElseIf lDocRPS
405     STWCancelSale( .F.           , ;           // Força cancelamento?
406                   aInfCancel[2] , ;           // Venda em andamento?
407                   aInfCancel[3] , ;           // Superior que autorizou cancelamento
408                   aInfCancel[10] , ;          // Numero Doc a Cancelar (L1_DOCRPS)
409                   aInfCancel[5] , ;           // Numeração venda a cancelar (L1_NUM)
410                   .F.           , ;
411                   cNFisCanc     , ;
412                   , ;

```

```

413         iif( lNFeAtv, aInfCancel[11],"") ,;          //-- L1_SERRPS
414         iif(!UsaPD,aInfCancel[8],.F.) ,;             //-- -- Usa CC/CD
415         iif(!UsaPD,aInfCancel[9],.F.) ,;             //-- -- Usa PD
416         lDocRPS )
417 Else
418     STWCancelSale( .F. , ; // Força cancelamento?
419     aInfCancel[2] , ; // Venda em andamento?
420     aInfCancel[3] , ; // Superior que autorizou cancelamento
421     aInfCancel[4] , ; // Numero Doc a Cancelar (L1_DOC)
422     aInfCancel[5] , ; // Numeração venda a cancelar (L1_NUM)
423     .F. , ;
424     cNFisCanc , ;
425     , ;
426     iif( lNFeAtv, aInfCancel[7],"") ,;          //-- TBC-G0 Serie da nota
427     iif(!UsaPD,aInfCancel[8],.F.) ,;             //-- -- Usa CC/CD
428     iif(!UsaPD,aInfCancel[9],.F.) ,;             //-- -- Usa PD
429     lDocRPS )
430 EndIf
431
432 If !lAutomato // variavel de controle do Robo
433     STIGridCupRefresh() // Sincroniza a Cesta com a interface
434
435     STIRegItemInterface()
436 Endif
437
438 Return Nil
439
440
441 //-----
442 /*/{Protheus.doc} STIRInPay
443 Retorna para a interface de pagamento
444
445 @param oPanSaleCancel , objeto , objeto da tela de cancelamento
446 @return lRet, lógico, se executa a ação do botão ou sai sem executar
447 */
448 //-----
449 Function STIRInPay(oPanSaleCancel)
450 Local lRet := .T.
451
452 If ExistFunc("STIVerCTef") .And. !STIVerCTef(.F.)
453     oPanSaleCancel:Hide()
454     oPanSaleCancel:= Nil
455     lRet := .F.
456     STIPayment(.T.)
457 EndIf
458
459 Return lRet
460
461
462 //-----
463 /*/{Protheus.doc} STIGTRsnCanCup
464 Propriedade GET para retornar o oRsnCanCup e cGetRsnCanCup

```

```

465
466 @return {oRsnCanCup, cGetRsnCanCup}, array, array com objeto objeto motivo da perda de venda e justificativa.
467 */
468 //-----
469 Function STIGTRsnCanCup()
470 Return {oRsnCanCup, cGetRsnCanCup}
471
472 //-----
473 /*{Protheus.doc} STISTRsnCanCup
474 Propriedade SET para ZERAR o oRsnCanCup e cGetRsnCanCup
475
476 @return nil
477 */
478 //-----
479 Function STISTRsnCanCup()
480 FreeObj(oRsnCanCup)
481 cGetRsnCanCup:=Space(50)
482 Return
483
484
485 /*{Protheus.doc} STIVldRsnInf
486     (Verifica se o motivo de perda foi informado)
487     @type Function
488     @return lRet, lógico, .T. se estiver preenchido ou está desativado
489     */
490 Function STIVldRsnInf()
491 Local lRet :=.T.
492
493 If ExistFunc("STWVldRsnInf")
494     lRet:= STWVldRsnInf(oRsnCanCup,cGetRsnCanCup )
495 Endif
496
497 Return lRet

```

Componentes/Interface/STIChangePay.prw

443 linhas após remoção de metadados de autoria/data

```

1 #Include 'Protheus.ch'
2 #Include "POSCSS.CH"
3 #Include 'STIChangePay.ch'
4
5 Static cCodCli      := ""
6 Static oDlGFrmpagto := Nil
7
8 //-----
9 /*{Protheus.doc} STIChangePay
10 Função para montar tela para mudança de pagamento
11 @param      aPagtos
12 @return     Nil
13 */
14 //-----
15 Function STIChangePay(aPagtos)
16 Local oLbL0rc      := Nil
17 Local oLbLNro0rc   := Nil
18 Local oLbLCli      := Nil
19 Local oLbLCodCli   := Nil
20 Local oLbLFrmPagto := Nil
21 Local oBtnOk        := Nil
22 Local oBtnConf      := Nil
23 Local oBtnCanc      := Nil
24 Local oBtnLimpar    := Nil
25 Local aAlter        := {"L4_NVFORM", "L4_QTDCART", "L4_NVPARC"}
26 Local nCont         := 0
27 Local nPosOrc       := 0
28 Local nPosCodCli    := 0
29 Local nPosLojaCli   := 0
30 Local nPosForma     := 0
31 Local nPosValor     := 0
32 Local nPosIDCard    := 0
33 Local cForma        := ""
34 Local cParc         := ""
35 Local cIdCard       := ""
36 Local cFieldOk      := "STIVldFrm()"
37
38 Private aHeadPagto  := {}
39 Private aColsPagto  := {}
40 Private aPagtosMHI  := {}
41 Private aHeadResumo := {}
42 Private aColsResumo := {}
43 Private oGetPagtos  := Nil
44 Private oGetCards   := Nil
45 Private oGetResumo  := Nil
46 Private cNumOrc     := ""
47
48 Default aPagtos     := {}
49
50 If Len(aPagtos) > 0
51     //Desabilita controles da tela principal. Motivo: Se o usuário utilizar uma tecla de atalho
52     STIBtnDeActivate()

```

```

53
54 nPosOrc := aScan(aPagtos[1],{ |x| AllTrim(x[1]) == "L4_NUM" } )
55 nPosCodCli := aScan(aImported[4][1][1],{ |x| AllTrim(x[1]) == "L1_CLIENTE" } )
56 nPosLojaCli := aScan(aImported[4][1][1],{ |x| AllTrim(x[1]) == "L1_LOJA" } )
57 nPosForma := aScan(aPagtos[1],{ |x| AllTrim(x[1]) == "L4_FORMA" } )
58 nPosValor := aScan(aPagtos[1],{ |x| AllTrim(x[1]) == "L4_VALOR" } )
59 nPosIDCard := aScan(aPagtos[1],{ |x| AllTrim(x[1]) == "L4_FORMAID" } )
60
61 //aHeader da grid de pagamentos do orçamento
62 Aadd( aHeadPagto, { STR0001, "L4_FORMA" , "@" , 35, 0, , , "C", , "V", } ) //"Forma de Pagamento"
63 Aadd( aHeadPagto, { STR0002, "L4_VALOR" , "@E 9,999,999,999,999.99", 16, 2, , , "N", , "V", } ) //"Valor"
64 Aadd( aHeadPagto, { STR0003, "L4_PARCELA" , "@" , 2, 0, , , "C", , "V", } ) //"Parcelas"
65 Aadd( aHeadPagto, { STR0004, "L4_FORMAID" , "@" , 2, 0, , , "C", , "V", } ) //"ID Cartão"
66 Aadd( aHeadPagto, { STR0005, "L4_NVFORM" , "@" , 35, 0, , , "C", "MHICG", "A", } ) //"Nova Forma"
67 Aadd( aHeadPagto, { STR0006, "L4_QTDCART" , "@99", 2, 0, , , "N", , "A", } ) //"Qtde. Cartões"
68 Aadd( aHeadPagto, { STR0007, "L4_NVPARC" , "@99", 2, 0, , , "C", , "A", } ) //"Nova Parcela"
69
70 //aHeader da grid de pagamentos depois das alterações - Resumo
71 Aadd( aHeadResumo, { STR0001, "L4_FORMA" , "@" , 35, 0, , , "C", , "V", } ) //"Forma de Pagamento"
72 Aadd( aHeadResumo, { STR0002, "L4_VALOR" , "@E 9,999,999,999,999.99", 16, 2, , , "N", , "V", } ) //"Valor"
73 Aadd( aHeadResumo, { STR0003, "L4_NVPARC" , "@99", 2, 0, , , "C", , "V", } ) //"Parcelas"
74 Aadd( aHeadResumo, { STR0004, "L4_FORMAID" , "@" , 2, 0, , , "C", , "V", } ) //"ID Cartão"
75
76 //aCols do Resumo
77 aAdd(aColsResumo,Array(Len(aHeadResumo)+1))
78
79 For nCont := 1 To Len(aHeadResumo)
80 If aHeadResumo[nCont][2] == "L4_NVPARC" //Tratamento para esse campo que não existe no SX3
81 aColsResumo[1][nCont] := "00"
82 Else
83 aColsResumo[1][nCont] := CriarVar(aHeadResumo[nCont][2],.T.)
84 EndIf
85 Next nCont
86 aColsResumo[1][Len(aHeadResumo)+1] := .F.
87
88 cIdCard := aPagtos[1][nPosIDCard][2]
89 cNumOrc := aPagtos[1][nPosOrc][2]
90 cCodCli := aImported[4][1][1][nPosCodCli][2] + "/" + aImported[4][1][1][nPosLojaCli][2] + " - " +
Posicione("SA1",1,xFilial("SA1")+aImported[4][1][1][nPosCodCli][2]+aImported[4][1][1][nPosLojaCli][2],"A1_NOME")
91
92 //aCols dos Pagamentos do orçamento
93 For nCont := 1 To Len(aPagtos)
94
95 If AllTrim(aPagtos[nCont][nPosForma][2]) <> cForma
96 cParc := "01"
97 Aadd(aPagtosMHI,{AllTrim(aPagtos[nCont][nPosForma][2]),STDChgPay(aPagtos[nCont][nPosForma][2])})
98
99 Aadd(aColsPagto,{AllTrim(aPagtos[nCont][nPosForma][2]) + " - " + POSICIONE( "SX5", 1, xFilial( "SX5" ) + "24" + AllTrim(aPagtos[nCont][nPosForma][2]), "X5_DESCRI"
),;
100 aPagtos[nCont][nPosValor][2],;
101 Iif(AllTrim(aPagtos[nCont][nPosForma][2]) = "R$", "", cParc),;
102 AllTrim(aPagtos[nCont][nPosIDCard][2]),;

```

```

103         Space(35),;
104         Space(0),;
105         Space(0),;
106         .F.})
107     cForma := AllTrim(aPagtos[nCont][nPosForma][2])
108 Else
109     //Quando o pagamento é em cartão e parcelado
110     If AllTrim(aPagtos[nCont][nPosForma][2]) $ "CC|CD" .And. aPagtos[nCont][nPosIDCard][2] == cIDCard
111         nPos := AScan(aColsPagto,{|x| SubStr(x[1],1,2) == SubStr(aPagtos[nCont][nPosForma][2],1,2) .And. AllTrim(x[4]) == AllTrim(aPagtos[nCont][nPosIDCard][2])})
112         aColsPagto[nPos][2] += aPagtos[nCont][nPosValor][2]
113         cParc := Somal(cParc)
114         aColsPagto[nPos][3] := cParc
115     Else
116         //Pagamento em mais de um cartão
117         cParc := "01"
118         Aadd(aColsPagto,{AllTrim(aPagtos[nCont][nPosForma][2]) + " - " + POSICIONE( "SX5", 1, xFilial( "SX5" ) + "24" + AllTrim(aPagtos[nCont][nPosForma][2]),
119 "X5_DESCRI" ),;
120         aPagtos[nCont][nPosValor][2],;
121         Iif(AllTrim(aPagtos[nCont][nPosForma][2]) = "R$", "", cParc),;
122         AllTrim(aPagtos[nCont][nPosIDCard][2]),;
123         Space(35),;
124         Space(0),;
125         Space(0),;
126         .F.})
127     cForma := AllTrim(aPagtos[nCont][nPosForma][2])
128     cIDCard := aPagtos[nCont][nPosIDCard][2]
129 EndIf
130 Next nCont
131
132 DEFINE MSDIALOG oDlgFrmPagto TITLE STR0008 STYLE DS_MODALFRAME FROM 0,0 TO 650,850 PIXEL OF oMainWnd    //"Troca de Pagamento"
133
134     oDlgFrmPagto:lEscClose      := .F. //Nao permite sair ao pressionar a tecla ESC.
135
136     oLblOrc := TSay():New(010,009,{|| STR0009 },oDlgFrmPagto,,,,,.T,,,,) //"Orçamento:"
137     oLblOrc:SetCSS( POSCSS (GetClassName(oLblOrc), CSS_LABEL_FOCAL ))
138
139     oLblNroOrc := TSay():New(010,050,{|| cNumOrc },oDlgFrmPagto,,,,,.T,,,,) //"Nro. Orçamento"
140     oLblNroOrc:SetCSS( POSCSS (GetClassName(oLblNroOrc), CSS_LABEL_NORMAL ))
141
142     oLblCli := TSay():New(025,009,{|| STR0010 },oDlgFrmPagto,,,,,.T,,,,) //"Cliente:"
143     oLblCli:SetCSS( POSCSS (GetClassName(oLblCli), CSS_LABEL_FOCAL ))
144
145     oLblCodCli := TSay():New(025,050,{|| cCodCli },oDlgFrmPagto,,,,,.T,,,,) //"Cód. Cliente"
146     oLblCodCli:SetCSS( POSCSS (GetClassName(oLblCodCli), CSS_LABEL_NORMAL ))
147
148     oLblFrmPagto := TSay():New(045,009,{|| STR0011 },oDlgFrmPagto,,,,,.T,,,,) //"Forma de Pagamento Original:"
149     oLblFrmPagto:SetCSS( POSCSS (GetClassName(oLblFrmPagto), CSS_LABEL_FOCAL ))
150
151     oGetPagtos := MsNewGetDados():New( 060, 009, 140, 420, GD_UPDATE, "AlwaysTrue", "AlwaysTrue",, aAlter,,999, cFieldOk, "", "AlwaysTrue", oDlgFrmPagto, aHeadPagto,
152     aColsPagto)

```

```

153      oBtnOk      := TButton():New( 150,275, STR0012, oDlgFrmPagto,{|| lRet := STBValidPay()},50,15,,,F.,.T.,.F.,.F.,.F. )    //"Ok"
154      oBtnOk:SetCSS(  POSCSS (GetClassName(oBtnOk)      , CSS_BTN_FOCAL ))
155
156      oLblResumo := TSay():New(170,009,{|| STR0013 },oDlgFrmPagto,,,,,T.,,,) //"Forma de Pagamento Atualizada:"
157      oLblResumo:SetCSS(  POSCSS (GetClassName(oLblResumo), CSS_LABEL_FOCAL ))
158
159      oGetResumo := MsNewGetDados():New( 185, 009, 265, 420, , "AllwaysTrue", "AllwaysTrue",,,,999, "AllwaysTrue", "", "AllwaysTrue", oDlgFrmPagto, aHeadResumo,
aColsResumo)
160
161      oBtnConf     := TButton():New( 300,155, STR0014, oDlgFrmPagto,{|| lRet := STBChangePay(@aPagtos), IIF(lRet,oDlgFrmPagto:End(),Nil), lChkAtiv :=
.F.},50,15,,,F.,.T.,.F.,.F.,.F. )    //"Confirmar"
162      oBtnConf:SetCSS(  POSCSS (GetClassName(oBtnConf)      , CSS_BTN_FOCAL ))
163
164      oBtnCanc      := TButton():New( 300,215, STR0015, oDlgFrmPagto,{|| Iif( ApMsgYesNo( STR0016 + Chr(13) + Chr(10)+ STR0017),oDlgFrmPagto:End(),Nil), lChkAtiv := .T.
},50,15,,,F.,.T.,.F.,.F.,.F. ) //"Cancela"/"Deseja CANCELAR a troca de pagamento?"/"(Se confirmado, será considerado as formas de pagamento do Orçamento.)"
165      oBtnCanc:SetCSS(  POSCSS (GetClassName(oBtnCanc)      , CSS_BTN_FOCAL ))
166
167      oBtnLimpar     := TButton():New( 300,275, STR0018, oDlgFrmPagto, {|| Iif( ApMsgYesNo( STR0019 + Chr(13) + Chr(10)+ STR0020),STILimpaPgto(aColsResumo),Nil)
},50,15,,,F.,.T.,.F.,.F.,.F. ) //"Limpar" //"Deseja limpar TODAS as formas de pagamentos informada?" //(Se confirmado, as novas formas de pagamento serão
apagadas.)"
168      oBtnLimpar:SetCSS(  POSCSS (GetClassName(oBtnLimpar)      , CSS_BTN_FOCAL ))
169
170      ACTIVATE MSDIALOG oDlgFrmPagto CENTERED
171 Else
172      STFMessage( STR0021 , STR0022 , STR0023 )          //"STIChangePay"/"STOP"/"Orçamento não contém pagamentos"
173      LjGrvLog(STR0024,STR0025)    //"Importa_Orçamento:STIChangePay"/"Orçamento não contém pagamentos"
174 EndIf
175 Return Nil
176
177 //-----
178 /*{Protheus.doc} STILimpaPgto
179 Função para limpar as formas de pagamentos da grid de Resumo
180 @param      aColsResumo
181 @return      .T.
182 */
183 //-----
184 Function STILimpaPgto(aColsResumo)
185 Local nCont := 0
186 aColsResumo := {}
187 aAdd(aColsResumo,Array(Len(aHeadResumo)+1))
188 For nCont := 1 To Len(aHeadResumo)
189     If aHeadResumo[nCont][2] == "L4_NVPARC" //Tratamento para esse campo que não existe no SX3
190         aColsResumo[1][nCont] := 0
191     Else
192         aColsResumo[1][nCont] := CriaVar(aHeadResumo[nCont][2],.T.)
193     EndIf
194 Next nCont
195 aColsResumo[1][Len(aHeadResumo)+1] := .F.
196 oGetResumo:SetArray(aColsResumo,.T.)
197 oGetResumo:Refresh()
198 Return .T.
199

```

```

200 //-----
201 /*{Protheus.doc} STIVldFrm
202 Função para validar o campo nova forma, parcelas e qtde de cartões
203 @param
204 @return .T. ou .F.
205 */
206 //-----
207 Function STIVldFrm()
208 Local lRet      := .T.
209 Local nPos      := 0
210 Local aFormas   := {}
211 Local nPosForma := aScan( oGetPagtos:aHeader, { |x| Alltrim(x[2])=="L4_FORMA" } )
212 Local nPosNvFrm := aScan( oGetPagtos:aHeader, { |x| Alltrim(x[2])=="L4_NVFORM" } )
213 Local nPosQtdCard := aScan( oGetPagtos:aHeader, { |x| Alltrim(x[2])=="L4_QTDCART" } )
214 Local nPosNvParc := aScan( oGetPagtos:aHeader, { |x| Alltrim(x[2])=="L4_NVPARC" } )
215
216 If StrTran(Alltrim(ReadVar()),'M->','') == "L4_NVFORM" .Or. StrTran(Alltrim(ReadVar()),'M->','') == "L4_QTDCART" .Or. StrTran(Alltrim(ReadVar()),'M->','') == "L4_NVPARC"
217   If StrTran(Alltrim(ReadVar()),'M->','') == "L4_NVFORM"
218     //Valida a forma de pagamento informada
219     nPos := aScan( aPagtos:MHI, { |x| Alltrim(x[1]) == SubStr(oGetPagtos:aCols[oGetPagtos:nAt][nPosForma],1,2) } )
220     If nPos > 0
221       aFormas := StrToKarr(aPagtos:MHI[nPos][2], ";")
222       If aScan( aFormas, { |x| SubStr(x,1,2) == SubStr(&(StrTran(Alltrim(ReadVar()),'M->','')),1,2) } ) > 0
223         M->L4_NVFORM := SubStr(&(StrTran(Alltrim(ReadVar()),'M->','')),1,2) + " - " + POSICIONE( "SX5", 1, xFilial( "SX5" ) + "24" +
SubStr(&(StrTran(Alltrim(ReadVar()),'M->','')),1,2), "X5_DESCRI" )
224         oGetPagtos:aCols[oGetPagtos:nAt][nPosNvFrm] := SubStr(&(StrTran(Alltrim(ReadVar()),'M->','')),1,2) + " - " + POSICIONE( "SX5", 1, xFilial( "SX5" ) + "24" +
SubStr(&(StrTran(Alltrim(ReadVar()),'M->','')),1,2), "X5_DESCRI" )
225         If SubStr(&(StrTran(Alltrim(ReadVar()),'M->','')),1,2) $ "CC|CD"
226           oGetPagtos:aCols[oGetPagtos:nAt][nPosQtdCard] := 0
227         ElseIf !Empty(&(StrTran(Alltrim(ReadVar()),'M->','')))
228           If SubStr(&(StrTran(Alltrim(ReadVar()),'M->','')),1,2) == "R$"
229             oGetPagtos:aCols[oGetPagtos:nAt][nPosQtdCard] := Space(0)
230             oGetPagtos:aCols[oGetPagtos:nAt][nPosNvParc] := Space(0)
231           Else
232             oGetPagtos:aCols[oGetPagtos:nAt][nPosQtdCard] := Space(0)
233             oGetPagtos:aCols[oGetPagtos:nAt][nPosNvParc] := Space(02)
234           EndIf
235         Else
236           oGetPagtos:aCols[oGetPagtos:nAt][nPosQtdCard] := Space(0)
237           oGetPagtos:aCols[oGetPagtos:nAt][nPosNvParc] := Space(0)
238         EndIf
239       ElseIf
240         MsgInfo(STR0026,STR0027) //"Essa forma de pagamento não está cadastrada para a forma de pagamento original, favor verificar!"/"Atenção"
241         lRet := .F.
242       EndIf
243     EndIf
244   Else
245     If StrTran(Alltrim(ReadVar()),'M->','') == "L4_QTDCART"
246       If Empty(oGetPagtos:aCols[oGetPagtos:nAt][nPosNvFrm]) .Or. !SubStr(oGetPagtos:aCols[oGetPagtos:nAt][nPosNvFrm],1,2) $ "CC|CD"
247         M->L4_QTDCART := Space(0)
248         oGetPagtos:aCols[oGetPagtos:nAt][nPosQtdCard] := Space(0)
249       ElseIf SubStr(oGetPagtos:aCols[oGetPagtos:nAt][nPosNvFrm],1,2) $ "CC|CD"

```

```

250         oGetPagtos:aCols[oGetPagtos:nAt][nPosQtdCard] := &(StrTran(Alltrim(ReadVar()),'M->',''))
251         If &(StrTran(Alltrim(ReadVar()),'M->','')) > 1
252             M->L4_NVPARC := Space(0)
253             oGetPagtos:aCols[oGetPagtos:nAt][nPosNvParc] := Space(0)
254         Else
255             M->L4_NVPARC := "00"
256             oGetPagtos:aCols[oGetPagtos:nAt][nPosNvParc] := "00"
257         EndIf
258     EndIf
259 EndIf
260 If StrTran(Alltrim(ReadVar()),'M->','') == "L4_NVPARC"
261     If Empty(oGetPagtos:aCols[oGetPagtos:nAt][nPosNvFrm]) .Or. SubStr(oGetPagtos:aCols[oGetPagtos:nAt][nPosNvFrm],1,2) $ "CC|CD"
262         If oGetPagtos:aCols[oGetPagtos:nAt][nPosQtdCard] > 1
263             M->L4_NVPARC := Space(0)
264             oGetPagtos:aCols[oGetPagtos:nAt][nPosNvParc] := Space(0)
265         Else
266             oGetPagtos:aCols[oGetPagtos:nAt][nPosNvParc] := &(StrTran(Alltrim(ReadVar()),'M->',''))
267         EndIf
268     ElseIf !SubStr(oGetPagtos:aCols[oGetPagtos:nAt][nPosNvFrm],1,2) $ "CC|CD"
269         oGetPagtos:aCols[oGetPagtos:nAt][nPosNvParc] := &(StrTran(Alltrim(ReadVar()),'M->',''))
270     EndIf
271 EndIf
272 EndIf
273 EndIf
274 oGetPagtos:Refresh()
275 Return lRet
276
277 //-----
278 /*{Protheus.doc} STIChgCard
279 Função para montar tela para pagamento com cartão e que foi definido que será pago com mais de 1 cartão
280 @param      aCartoes
281             aCartoes[1] = ID Cartão
282             aCartoes[2] = Nova Forma de Pagamento
283             aCartoes[3] = Valor
284             aCartoes[4] = Parcela
285             aCartoes[5] = Qtde de Cartões
286             aCartoes[6] = Forma Original
287 @return      .T. ou .F.
288 */
289 //-----
290 Function STIChgCard(aCartoes)
291 Local oDlgFrmCard := Nil
292 Local aAlter      := {"L4_VALOR","L4_NVPARC"}
293 Local aHeadCard   := {}
294 Local aColsCard   := {}
295 Local nCont       := 0
296 Local nCont2      := 0
297 Local cIdCardCD   := "00"
298 Local cIdCardCC   := "00"
299 Local cFieldOk    := "STIVldCard()"
300 Local lRet        := .T.
301

```

```

302 Aadd( aHeadCard, { STR0001, "L4_FORMA" , "@!", 35, 0, , , "C",, "V", } ) //"Forma de Pagamento"
303 Aadd( aHeadCard, { STR0002, "L4_VALOR" , "@E 9,999,999,999,999.99", 16, 2, , , "N",, "A", } ) //"Valor"
304 Aadd( aHeadCard, { STR0003, "L4_NVPARC" , "@E 99", 2, 0, , , "N",, "A", } ) //"Parcelas"
305 Aadd( aHeadCard, { STR0028, "L4_OLDID" , "@!", 2, 0, , , "C",, "V", } ) //"ID Original"
306 Aadd( aHeadCard, { STR0029, "L4_NEWID" , "@!", 2, 0, , , "C",, "V", } ) //"ID Atualizado"
307 Aadd( aHeadCard, { STR0030, "L4_OLDFRM" , "@!", 35, 0, , , "C",, "V", } ) //"Forma de Pagto Anterior"
308 Aadd( aHeadCard, { STR0031, "L4_OLDVLR" , "@E 9,999,999,999,999.99", 16, 2, , , "N",, "V", } ) //"Valor Original"
309 Aadd( aHeadCard, { STR0032, "L4_OLDPAR" , "@E 99", 2, 0, , , "N",, "V", } ) //"Parc. Original"
310
311 For nCont := 1 To Len(aCartoes)
312   For nCont2 := 1 To aCartoes[nCont][5]
313     //Aqui verifico tipo de cartão e ID, para carregar o ID certo para cada tipo de cartão
314     If SubStr(aCartoes[nCont][2],1,2) == "CD"
315       cIdCardCD := Somal(cIdCardCD)
316     Else
317       cIdCardCC := Somal(cIdCardCC)
318     EndIf
319
320     aAdd(aColsCard,{AllTrim(aCartoes[nCont][2]),;
321       0,;
322       0,;
323       AllTrim(aCartoes[nCont][1]),;
324       Iif(SubStr(aCartoes[nCont][2],1,2) == "CD", cIdCardCD, cIdCardCC),;
325       aCartoes[nCont][6],;
326       aCartoes[nCont][3],;
327       aCartoes[nCont][4],;
328       .F.})
329   Next nCont2
330 Next nCont
331
332 DEFINE MSDIALOG oDlgFrmCard TITLE STR0033 STYLE DS_MODALFRAME FROM 90,0 TO 550,850 PIXEL OF oDlgFrmPagto //"Pagamentos com mais cartões"
333
334 oDlgFrmCard:lEscClose := .F. //Nao permite sair ao pressionar a tecla ESC.
335
336 oLblOrc := TSay():New(010,009,{|| STR0009 },oDlgFrmCard,,,,,.T.,,,) //"Orçamento:"
337 oLblOrc:SetCSS( POSCSS (GetClassName(oLblOrc), CSS_LABEL_FOCAL ))
338
339 oLblNroOrc := TSay():New(010,050,{|| cNumOrc },oDlgFrmCard,,,,,.T.,,,) //"Nro. Orçamento"
340 oLblNroOrc:SetCSS( POSCSS (GetClassName(oLblNroOrc), CSS_LABEL_NORMAL ))
341
342 oLblCli := TSay():New(025,009,{|| STR0010 },oDlgFrmCard,,,,,.T.,,,) //"Cliente:"
343 oLblCli:SetCSS( POSCSS (GetClassName(oLblCli), CSS_LABEL_FOCAL ))
344
345 oLblCodCli := TSay():New(025,050,{|| cCodCli },oDlgFrmCard,,,,,.T.,,,) //"Cód. Cliente"
346 oLblCodCli:SetCSS( POSCSS (GetClassName(oLblCodCli), CSS_LABEL_NORMAL ))
347
348 oLblFrmPagto := TSay():New(045,009,{|| STR0034 },oDlgFrmCard,,,,,.T.,,,) //"Defina os pagamentos:"
349 oLblFrmPagto:SetCSS( POSCSS (GetClassName(oLblFrmPagto), CSS_LABEL_FOCAL ))
350
351 oGetCards := MsNewGetDados():New( 060, 009, 140, 420, GD_UPDATE, "AllwaysTrue", "AllwaysTrue",, aAlter,,99, cFieldOk, "", "AllwaysTrue", oDlgFrmCard, aHeadCard,
aColsCard)
352

```

```

353 oBtnConf := TButton():New( 200,155, STR0014, oDlgFrmCard,{|| lRet := STBVldCard(oGetCards:aHeader, oGetCards:aCols),
IIF(lRet,oDlgFrmCard:End(),Nil)},50,15,,,,.F.,.T.,.F.,.F.,.F.,.F. ) //"Confirmar"
354 oBtnConf:SetCSS( POSCSS (GetClassName(oBtnConf) , CSS_BTN_FOCAL ))
355
356 oBtnCanc := TButton():New( 200,215, STR0015, oDlgFrmCard,{|| IIf( ApMsgYesNo( STR0035 + Chr(13) + Chr(10)+ STR0036),oDlgFrmCard:End(),Nil), lRet :=
.F.,50,15,,,,.F.,.T.,.F.,.F.,.F.,.F. ) //"Cancelar"/"Deseja CANCELAR o pagamento com mais cartões?"/"(Se confirmado, será necessário redefinir as formas de pagamento
informadas.)"
357 oBtnCanc:SetCSS( POSCSS (GetClassName(oBtnCanc) , CSS_BTN_FOCAL ))
358
359 oBtnLimpar := TButton():New( 200,275, STR0018, oDlgFrmCard, {|| IIf( ApMsgYesNo( STR0019 + Chr(13) + Chr(10)+ STR0020),STIClearCard(),Nil)
},50,15,,,,.F.,.T.,.F.,.F.,.F.,.F. ) //"Limpar" //"Deseja limpar TODAS as formas de pagamentos informada?" //" (Se confirmado, as novas formas de pagamento serão
apagadas.)"
360 oBtnLimpar:SetCSS( POSCSS (GetClassName(oBtnLimpar) , CSS_BTN_FOCAL ))
361
362 ACTIVATE MSDIALOG oDlgFrmCard CENTERED
363
364 Return lRet
365
366 //-------------------------------------
367 /*{Protheus.doc} STIVldCard
368 Função para validar o campo valor e parcelas da tela de cartões
369 @param
370 @return .T. ou .F.
371 */
372 //-------------------------------------
373 Function STIVldCard()
374 Local lRet := .T.
375 Local nPos := 0
376 Local nVlrTot := 0
377 Local nCont := 0
378 Local cFormaOri := ""
379 Local cFormaNew := ""
380 Local cIdCardOld := ""
381 Local aFormas := {}
382 Local nPosForma := aScan(oGetCards:aHeader,{ |x| AllTrim(x[2]) == "L4_FORMA" } )
383 Local nPosValor := aScan(oGetCards:aHeader,{ |x| AllTrim(x[2]) == "L4_VALOR" } )
384 Local nPosNvParc := aScan(oGetCards:aHeader,{ |x| AllTrim(x[2]) == "L4_NVPARC" } )
385 Local nPosOldID := aScan(oGetCards:aHeader,{ |x| AllTrim(x[2]) == "L4_OLDID" } )
386 Local nPosVlrPgt := aScan(oGetCards:aHeader,{ |x| AllTrim(x[2]) == "L4_OLDVLR" } )
387 Local nPosForOld := aScan(oGetCards:aHeader,{ |x| AllTrim(x[2]) == "L4_OLDFRM" } )
388 Local nPosParcPgt := aScan(oGetCards:aHeader,{ |x| AllTrim(x[2]) == "L4_OLDPAR" } )
389
390 If StrTran(Alltrim(ReadVar()),'M->','') == "L4_VALOR" .Or. StrTran(Alltrim(ReadVar()),'M->','') == "L4_NVPARC"
391 If StrTran(Alltrim(ReadVar()),'M->','') == "L4_VALOR"
392 //Valida se o valor informado é maior que o original
393 If &(StrTran(Alltrim(ReadVar()),'M->','') >= oGetCards:aCols[oGetCards:nAt][nPosVlrPgt]
394 MsgInfo(STR0037,STR0027) //"Valor informado maior ou igual que o valor original do pagamento, favor verificar!"/"Atenção"
395 lRet := .F.
396 Else
397 cFormaOri := oGetCards:aCols[oGetCards:nAt][nPosForOld]
398 cFormaNew := oGetCards:aCols[oGetCards:nAt][nPosForma]
399 cIdCardOld := oGetCards:aCols[oGetCards:nAt][nPosOldID]

```

```

400         For nCont := 1 To Len(oGetCards:aCols)
401             If oGetCards:aCols[nCont][nPosForOld] == cFormaOld .And. oGetCards:aCols[nCont][nPosForma] == cFormaNew .And. oGetCards:aCols[nCont][nPosOldID] == cIdCardOld
402                 If nCont == oGetCards:nAt
403                     nVlrTot+= &(StrTran(Alltrim(ReadVar()),'M->',''))
404                 Else
405                     nVlrTot+= oGetCards:aCols[nCont][nPosValor]
406                 EndIf
407             EndIf
408         Next nCont
409
410         If nVlrTot > oGetCards:aCols[oGetCards:nAt][nPosVlrPgt]
411             MsgInfo(STR0038,STR0027) //"Valor informado maior que o valor original do pagamento, favor verificar!"/"Atenção"
412             lRet := .F.
413         EndIf
414     EndIf
415     ElseIf StrTran(Alltrim(ReadVar()),'M->','') == "L4_NVPARC"
416         //Valida se a parcela informada é maior que o original
417         If &(StrTran(Alltrim(ReadVar()),'M->','')) > Iif(SubStr(oGetCards:aCols[oGetCards:nAt][nPosForOld],1,2) == "R$", 1, Val(oGetCards:aCols[oGetCards:nAt][nPosParcPgt]))
418             MsgInfo(STR0039,STR0027) //"Atenção"
419             lRet := .F.
420         EndIf
421     EndIf
422 EndIf
423 oGetPagtos:Refresh()
424 Return lRet
425
426 //-----
427 /*{Protheus.doc} STIClearCard
428 Função para limpar as formas de pagamentos definidos na tela de Cartões
429 @param
430 @return      Nil
431 */
432 //-----
433 Function STIClearCard()
434 Local nCont      := 0
435 Local nPosValor := aScan(oGetCards:aHeader,{ |x| AllTrim(x[2]) == "L4_VALOR" } )
436 Local nPosNvParc:= aScan(oGetCards:aHeader,{ |x| AllTrim(x[2]) == "L4_NVPARC" } )
437
438 For nCont := 1 To Len(oGetCards:aCols)
439     oGetCards:aCols[nCont][nPosValor] := 0
440     oGetCards:aCols[nCont][nPosNvParc] := 0
441 Next nCont
442 oGetCards:Refresh()
443 Return Nil

```

Componentes/Interface/STIConfCash.prw

690 linhas após remoção de metadados de autoria/data

```

1 #Include 'Protheus.ch'
2 #include "fwbrowse.ch"
3 #INCLUDE "POSCSS.CH"
4 #INCLUDE "STPOS.CH"
5 #INCLUDE "STICONFCASH.CH"
6
7 Static aBrwContent := {}          // Conteudo do Browse
8 Static oBrwContent := Nil
9 Static aPaym       := {}
10 Static nListPos    := 0
11 Static lListWhen   := .T.
12 Static oViewBkp    := Nil
13 Static cPicTotal   := PesqPict("SL1","L1_VLRTOT")
14 Static oSayTotal   := Nil
15 Static oPayList    := Nil
16 Static aNumerario := {}          //Variavel responsavel por armazenar as formas de pagamento e seu respectivo valor
17
18 Static oValCaixa    := Nil
19 Static oValEstac    := Nil
20 Static oValPDV      := Nil
21 Static oValDtAbert  := Nil
22 Static oValAbHora   := Nil
23 Static oValDtFech   := Nil
24 Static oValFcHora   := Nil
25
26 #DEFINE POS_DESCFP      2
27 #DEFINE POS_QTDE        5
28 #DEFINE POS_VALDIG      7
29 #DEFINE POS_VALAPU      8
30
31 //-----
32 /*{Protheus.doc} STIConfCash
33 Relaiza a conferencia de caixa
34 @param cTpOpCl - 1 para abertura de caixa, 2 para fechamento de caixa
35 @return lRet - Retorna se realizou a conferencia
36 @obs
37 @sample
38 */
39 //-----
40 Function STIConfCash()
41
42 STIExchangePanel( { || STIPanConfCash() } )
43
44 Return
45
46 //-----
47 /*{Protheus.doc} STWConfCash
48 Relaiza a conferencia de caixa
49 @param cTpOpCl - 1 para abertura de caixa, 2 para fechamento de caixa
50 @return lRet - Retorna se realizou a conferencia
51 @obs
52 @sample

```

```

53 */
54 //-----
55 Static Function STIPanConfCash()
56
57 Local oPanelMVC      := STIGetDlg()           // Objeto do Panel onde a interface de selecao de clientes sera criada
58 Local oMainPanel     := TPanel():New(00,00,"",oPanelMVC,,,,,oPanelMVC:nWidth/2,(oPanelMVC:nHeight)/2) // Painel de Get de Consulta
59 Local oLblCab        := Nil
60 Local oLblPayList    := Nil
61 Local oLblValue      := Nil
62 Local oGetValue      := Nil
63 Local nGetValue      := 0
64 Local oGroupInf      := Nil
65 Local oLblInf        := Nil
66 Local oBtnAddPay     := Nil
67 Local oBtnConfirm    := Nil
68 Local oBtnCa         := Nil
69 Local oBtnCancela    := Nil
70 Local aPaymOpt       := {}
71 Local nX             := 0
72 Local lReduZ         := .F.
73 Local oReduZ         := Nil
74 Local lLeitX         := .F.
75 Local oLeitX         := Nil
76
77 oPayList := Nil
78 aPaym := STDCreatGd()
79 aNumerario := aClone(aPaym)
80
81 STIBtnDeActivate()
82
83 oMainPanel:SetCSS( POSCSS (GetClassName(oMainPanel), CSS_PANEL_CONTEXT ))
84
85 For nX := 1 To Len(aPaym)
86     Add(aPaymOpt,AllTrim(Str(nX))+ " - "+aPaym[nX,POS_DESCFP])
87 Next nX
88
89 oLblCab:= TSay():New(POSVERT_CAB,POSHOR_1,{||STR0001},oMainPanel,,,,,.T.,,,) //"Conferência de Caixa"
90 oLblCab:SetCSS( POSCSS (GetClassName(oLblCab), CSS_BREADCUMB ))
91
92 oLblPayList := TSay():New(POSVERT_LABEL1,POSHOR_1, {||STR0002}, oMainPanel,,,,,.T.) //'Selecione a forma de pagamento'
93 oLblPayList:SetCSS( POSCSS (GetClassName(oLblPayList), CSS_LABEL_FOCAL ))
94
95 oPayList := TListBox():Create(oMainPanel, POSVERT_GET1, POSHOR_1, Nil,aPaymOpt , LARG_LIST_CONSULT , oMainPanel:nHeight/10 ,,,,,.T.,,{|| STIListAction(oGetValue)},,,{||
    lListWhen})
96 oPayList:SetCSS( POSCSS (GetClassName(oPayList), CSS_LISTBOX ))
97
98 oLblValue := TSay():New(oPanelMVC:nHeight/4.78235,POSHOR_1, {||STR0003}, oMainPanel,,,,,.T.) //'Valor'
99 oLblValue:SetCSS( POSCSS (GetClassName(oLblValue), CSS_LABEL_FOCAL ))
100
101 oGetValue:= TGet():New( oPanelMVC:nHeight/4.278947,POSHOR_1,{|u| If(PCount())>0,nGetValue:=u,nGetValue)}, ;
102     oMainPanel,oPanelMVC:nWidth/6 ,18,cPicTotal,,,,,.T.,,{|| !lListWhen},,,;
103     ,,,,"nGetValue")

```

```

104 oGetValue:SetCSS( POSCSS (GetClassName(oGetValue), CSS_GET_NORMAL ))
105
106 // Botao Adicionar Pagamento
107 oBtnAddPay := TButton():New( oPanelMVC:nHeight/4.278947,oPanelMVC:nWidth/5,STR0004,oMainPanel,{ || STIAddListPay(@nGetValue)}, ;
108                               LARGBTN,20,,,,.T.,,,{|| !lListWhen}) //"Adicionar Pagamento"
109 oBtnAddPay:SetCSS( POSCSS (GetClassName(oBtnAddPay), CSS_BTN_NORMAL ))
110
111 oBtnAddPay:bLostFocus := { || oPayList:GoTop(), oPayList:SetFocus()}
112
113 // Botao Cancelar
114 oBtnCa := TButton():New( (oPanelMVC:nHeight/4.278947),oPanelMVC:nWidth/3,STR0023,oMainPanel,{ || STICancAction(@nGetValue) }, ;
115                          LARGBTN,20,,,,.T.,,,{|| !lListWhen}) //"Cancelar"
116 oBtnCa:SetCSS(POSCSS(GetClassName(oBtnCa), CSS_BTN_NORMAL))
117
118 oBtnCa:bLostFocus := { || oPayList:GoTop(), oPayList:SetFocus()}
119
120 oGroupInf := TGroup():New(oPanelMVC:nHeight/3.12692,POSHOR_1,oPanelMVC:nHeight/2.71,oPanelMVC:nWidth/2.0645,',',oMainPanel,,5,.T.)
121
122 oLblInf := TSay():New(oPanelMVC:nHeight/3.011111,POSHOR_1+50, {||STR0005+CRLF+; //'Para alterar um valor já incluído na conferência, selecione a'
123                               '          '+STR0006}, oGroupInf,,,,.T.,,,80) //'mesma forma de pagamento e altere o valor'
124 oLblInf:SetCSS( POSCSS (GetClassName(oLblInf), CSS_LABEL_FOCAL ))
125
126 //Valido se é NFCE ou SAT e não apresento as opções
127 If !(STFGetCfg("lUseSAT",.F.) .Or. STFGetCfg("lUseNFCE",.F.))
128     //Leitura X
129     oLeitX := TCheckBox():New(POSVERT_BTNFOCAL+5,POSHOR_1,STR0026,{|| lLeitX },oGroupInf,80,,,{|| (lLeitX := VldLeitX(lLeitX),oLeitX:Refresh())}) //Leitura X
130
131     //Redução Z
132     oReduZ := TCheckBox():New(POSVERT_BTNFOCAL+5,POSHOR_1+60,STR0027,{|| lReduZ },oGroupInf,80,,,{|| (lReduZ := STVldRedZ(lReduZ),oReduZ:Refresh())}) //Redução Z
133
134 EndIf
135
136 //Botao Cancelar Conferencia
137 oBtnCancela := TButton():New( POSVERT_BTNFOCAL,POSHOR_BTNFOCAL-90,STR0023,oMainPanel,{||STICancelConference(oMainPanel)}, ; //"Cancelar"
138                               LARGBTN,ALTURABTN,,,,.T.)
139 oBtnCancela:SetCSS(POSCSS(GetClassName(oBtnCancela), CSS_BTN_FOCAL))
140
141 //Botao Confirmar Conferencia
142 oBtnConfirm := TButton():New( POSVERT_BTNFOCAL,POSHOR_BTNFOCAL,STR0007,oMainPanel,{||STIConfirmConference(lLeitX,lReduZ,oMainPanel)}, ; //"Confirmar"
143                               LARGBTN,ALTURABTN,,,,.T.)
144 oBtnConfirm:SetCSS( POSCSS (GetClassName(oBtnConfirm), CSS_BTN_FOCAL ))
145
146 STIRightPanel()
147
148 oPayList:GoTop()
149 oPayList:SetFocus()
150
151
152 Return oMainPanel
153
154 //-----
155 /*{Protheus.doc} STIRightPanel

```

```

156 Relaza a conferencia de caixa
157 @param cTpOpCl - 1 para abertura de caixa, 2 para fechamento de caixa
158 @return lRet - Retorna se realizou a conferencia
159 @obs
160 @sample
161 */
162 //-----
163 Static Function STIRightPanel()
164
165 Local oViewRight:= STIGetRDlg()
166 Local oTop      := Nil //Objeto para cabecalho
167 Local oContent  := Nil //Objeto para itens
168 Local oBottom   := Nil //Objeto para rodape
169
170 oViewBkp := oViewRight
171
172 oViewRight:FreeChildren()
173
174 @ 000,000 BITMAP oTop RESOURCE "x.png" NOBORDER SIZE 000,130 OF oViewRight ADJUST PIXEL
175 oTop:Align := CONTROL_ALIGN_TOP
176 oTop:SetCSS( POSCSS (GetClassName(oTop), CSS_PANEL_HEADER ))
177 oTop:ReadClientCoors(.T.,.T.)
178
179 STICab( oTop )
180
181 @ 000,000 BITMAP oBottom RESOURCE "x.png" NOBORDER SIZE 000,055 OF oViewRight ADJUST PIXEL
182 oBottom:Align := CONTROL_ALIGN_BOTTOM
183 oBottom:SetCSS( POSCSS (GetClassName(oBottom), CSS_PANEL_FOOTER ))
184 oBottom:ReadClientCoors(.T.,.T.)
185
186 STIRodape( oBottom )
187
188 @ 000,000 BITMAP oContent RESOURCE "x.png" NOBORDER SIZE 000,000 OF oViewRight ADJUST PIXEL
189 oContent:Align := CONTROL_ALIGN_ALLCLIENT
190 oContent:ReadClientCoors(.T.,.T.)
191
192 STIIitems( oContent )
193
194 Return
195
196 //-----
197 /*{Protheus.doc} STICab
198 Cabeçalho da tela
199
200 @param
201 @return
202 @obs
203 @sample
204 */
205 //-----
206 Static Function STICab( o )
207

```

```

208 Local oLblCab      := Nil          //Objeto label
209 Local oLblCaixa    := Nil
210 Local oLblEstac    := Nil
211 Local oLblPDV      := Nil
212 Local oLblCxGeral  := Nil
213 Local oValCxGeral  := Nil
214 Local oLblEmp      := Nil
215 Local oValEmp      := Nil
216 Local oLblCab2     := Nil
217 Local oLblCab3     := Nil
218 Local oLblDtAbert   := Nil
219 Local oLblAbHora    := Nil
220 Local oLblDtFech    := Nil
221 Local oLblFcHora    := Nil
222 Local aDtConf       := STDDtAbCx() //Informacoes de abertura e fechamento do caixa
223 Local oFontTxt      := TFont():New('Arial',,-20,.T.)
224 Local aStation      := STBInfoEst( 1, .T. )
225
226 oLblCab := TSay():New(10, 10, {||STR0008}, o,,oFontTxt,,,,.T.,,100,) //"Dados da Estação"
227 oLblCab:SetCSS( POSCSS (GetClassName(oLblCab), CSS_BREADCUMB ))
228
229 /* Caixa */
230
231 oLblCaixa := TSay():New(30, 10, {||STR0009}, o,,oFontTxt,,,,.T.,,100,) //"Caixa"
232 oLblCaixa:SetCSS( POSCSS (GetClassName(oLblCaixa), CSS_LABEL_NORMAL ))
233
234 oValCaixa := TSay():New(40, 10, {|| aStation[1]}, o,,oFontTxt,,,,.T.,,100,)
235 oValCaixa:SetCSS( POSCSS (GetClassName(oValCaixa), CSS_LABEL_FOCAL ))
236
237 /* Estação */
238
239 oLblEstac := TSay():New(30, 50, {||STR0010}, o,,oFontTxt,,,,.T.,,100,) //"Estação"
240 oLblEstac:SetCSS( POSCSS (GetClassName(oLblEstac), CSS_LABEL_NORMAL ))
241
242 oValEstac := TSay():New(40, 50, {|| aStation[2]}, o,,oFontTxt,,,,.T.,,100,)
243 oValEstac:SetCSS( POSCSS (GetClassName(oValEstac), CSS_LABEL_FOCAL ))
244
245 /* PDV */
246
247 oLblPDV := TSay():New(30, 90, {||STR0011}, o,,oFontTxt,,,,.T.,,100,) //"PDV"
248 oLblPDV:SetCSS( POSCSS (GetClassName(oLblPDV), CSS_LABEL_NORMAL ))
249
250 oValPDV := TSay():New(40, 90, {|| aStation[4]}, o,,oFontTxt,,,,.T.,,100,)
251 oValPDV:SetCSS( POSCSS (GetClassName(oValPDV), CSS_LABEL_FOCAL ))
252
253 /* Caixa Geral */
254
255 oLblCxGeral := TSay():New(30, 120, {||STR0012}, o,,oFontTxt,,,,.T.,,100,) //"Caixa Geral"
256 oLblCxGeral:SetCSS( POSCSS (GetClassName(oLblCxGeral), CSS_LABEL_NORMAL ))
257
258 oValCxGeral := TSay():New(40, 120, {|| aStation[1]}, o,,oFontTxt,,,,.T.,,100,)
259 oValCxGeral:SetCSS( POSCSS (GetClassName(oValCxGeral), CSS_LABEL_FOCAL ))

```

```

260
261 /* Emp Filial */
262
263 oLb1Emp := TSay():New(30, 170, {||STR0013}, o,,oFontTxt,,,,.T.,,100,) //"Empresa / Filial"
264 oLb1Emp:SetCSS( POSCSS (GetClassName(oLb1Emp), CSS_LABEL_NORMAL ))
265
266 oValEmp := TSay():New(40, 170, {|| cEmpAnt+" / "+cFilAnt}, o,,oFontTxt,,,,.T.,,100,)
267 oValEmp:SetCSS( POSCSS (GetClassName(oValEmp), CSS_LABEL_FOCAL ))
268
269 /*Abertura */
270
271 oLb1Cab2:= TSay():New(70, 10, {||STR0014}, o,,oFontTxt,,,,.T.,,100,) //"Abertura"
272 oLb1Cab2:SetCSS( POSCSS (GetClassName(oLb1Cab2), CSS_BREADCUMB ))
273
274 oLb1DtAbert := TSay():New(90, 10, {||STR0015}, o,,oFontTxt,,,,.T.,,100,) //"Data"
275 oLb1DtAbert:SetCSS( POSCSS (GetClassName(oLb1DtAbert), CSS_LABEL_NORMAL ))
276
277 oValDtAbert := TSay():New(100, 10, {||aDtConf[1]}, o,,oFontTxt,,,,.T.,,100,)
278 oValDtAbert:SetCSS( POSCSS (GetClassName(oValDtAbert), CSS_LABEL_FOCAL ))
279
280 oLb1AbHora := TSay():New(90, 60, {||STR0016}, o,,oFontTxt,,,,.T.,,100,) //"Hora"
281 oLb1AbHora:SetCSS( POSCSS (GetClassName(oLb1AbHora), CSS_LABEL_NORMAL ))
282
283 oValAbHora := TSay():New(100, 60, {||aDtConf[2]}, o,,oFontTxt,,,,.T.,,100,)
284 oValAbHora:SetCSS( POSCSS (GetClassName(oValAbHora), CSS_LABEL_FOCAL ))
285
286 /* Fechamento */
287
288 oLb1Cab3:= TSay():New(70, 170, {||STR0017}, o,,oFontTxt,,,,.T.,,100,) //"Fechamento"
289 oLb1Cab3:SetCSS( POSCSS (GetClassName(oLb1Cab3), CSS_BREADCUMB ))
290
291 oLb1DtFech := TSay():New(90, 170, {||STR0015}, o,,oFontTxt,,,,.T.,,100,) //"Data"
292 oLb1DtFech:SetCSS( POSCSS (GetClassName(oLb1DtFech), CSS_LABEL_NORMAL ))
293
294 oValDtFech := TSay():New(100, 170, {|| dDataBase }, o,,oFontTxt,,,,.T.,,100,)
295 oValDtFech:SetCSS( POSCSS (GetClassName(oValDtFech), CSS_LABEL_FOCAL ))
296
297 oLb1FcHora := TSay():New(90, 220, {||STR0016}, o,,oFontTxt,,,,.T.,,100,) //"Hora"
298 oLb1FcHora:SetCSS( POSCSS (GetClassName(oLb1FcHora), CSS_LABEL_NORMAL ))
299
300 oValFcHora := TSay():New(100, 220, {||AllTrim(Left(Time(),5))}, o,,oFontTxt,,,,.T.,,100,)
301 oValFcHora:SetCSS( POSCSS (GetClassName(oValFcHora), CSS_LABEL_FOCAL ))
302
303 Return .T.
304
305 //-----
306 /*{Protheus.doc} STIIItens
307 Criação do grid dos itens
308
309 @param
310 @return
311 @obs

```

```

312 @sample
313 */
314 //-----
315 Static Function STIIItens( o )
316
317 Local oContent //Conteudo do css
318 Local oView      := FwLoadView("STIPosMain")
319 Local oColumn     := Nil
320 Local nX          := 0
321 Local lConfCega    := SuperGetMV("MV_LJEXAPU",.T.,.F.) == .F.
322
323 aBrwContent := {} // zera a matriz para evitar duplicidade de informações
324 For nX := 1 To Len(aPaym)
325     aAdd(aBrwContent,{aPaym[nX,POS_DESCFP],aPaym[nX,POS_QTDE],AllTrim(Transform(aPaym[nX,POS_VALAPU],cPicTotal)),AllTrim(Transform(0,cPicTotal))})
326 Next nX
327
328 oContent := POSBrwContainer(o)
329
330 oView:oModel:SetOperation(4)
331 oView:oModel:Activate()
332
333 DEFINE FWBROWSE oBrwContent DATA ARRAY ARRAY aBrwContent NO LOCATE NO CONFIG NO REPORT OF oContent
334
335     oBrwContent:nRowHeight := 30
336     oBrwContent:SetVScroll(.F.)
337
338     ADD COLUMN oColumn DATA {|| aBrwContent[oBrwContent:nAt][1] }    TITLE STR0018    SIZE 015 OF oBrwContent  //"Forma de Pagamento"
339
340     If !lConfCega
341         ADD COLUMN oColumn DATA {|| aBrwContent[oBrwContent:nAt][2] }    TITLE STR0019    SIZE 005 OF oBrwContent  //"Qtd."
342         ADD COLUMN oColumn DATA {|| aBrwContent[oBrwContent:nAt,3] } TITLE STR0020    SIZE 005 OF oBrwContent  //"Valor Apurado"
343     EndIf
344     ADD COLUMN oColumn DATA {|| aBrwContent[oBrwContent:nAt,4] }    TITLE STR0021    SIZE 005 OF oBrwContent  //"Valor Informado"
345
346 ACTIVATE FWBROWSE oBrwContent
347
348 Return .T.
349
350
351 //-----
352 /*{Protheus.doc} STIRodape
353 Criação das barra dos totalizadores
354
355 @param
356 @return
357 @obs
358 @sample
359 */
360 //-----
361 Static Function STIRodape( o )
362
363 Local nTop := 005

```

```

364 Local oSay
365
366 nTop := 006
367
368 @ nTop,o:nWidth/2-75 SAY oSay PROMPT STR0022 SIZE o:nWidth/2-35,40 OF o PIXEL HTML //"Total"
369 oSay:SetCSS( POSCSS (GetClassName(oSay), CSS_LABEL_NORMAL ))
370
371 nTop += 10
372
373 @ nTop,o:nWidth/2-120 SAY oSayTotal PROMPT AllTrim(Str(0,12,2)) SIZE 100,40 RIGHT OF o PIXEL HTML
374 oSayTotal:SetCSS( POSCSS (GetClassName(oSayTotal), CSS_LABEL_TOTAL ))
375
376 Return .T.
377
378
379 //-----
380 /*{Protheus.doc} STIListAction
381 Indica a opcao selecionada no listbox e o desativa e habilita o get e o botao para adicionar pagamentos.
382
383 @param oGetValue - Get do valor de pagamento.
384 @return lRet - Retorna se realizou a conferencia
385 @obs
386 @sample
387 */
388 //-----
389 Static Function STIListAction(oGetValue)
390
391 lListWhen := .F.
392 oGetValue:SetFocus()
393 nListPos := oPayList:nAt
394
395 Return
396
397
398 //-----
399 /*{Protheus.doc} STICancAction
400 Cancela a insercao do pagamento e foca no list
401
402 @param oGetValue - Get do valor de pagamento.
403 @return lRet - Retorna se realizou a conferencia
404 @obs
405 @sample
406 */
407 //-----
408 Static Function STICancAction(oGetValue)
409
410 Default oGetValue := Nil
411
412 lListWhen := .T.
413 oPayList:GoTop()
414 oPayList:SetFocus()
415

```

```

416 Return
417
418
419 //-----
420 /*{Protheus.doc} STIListAction
421 Indica a opcao selecionada no listbox e o desativa e habilita o get e o botao para adicionar pagamentos.
422
423 @param oGetValue - Get do valor de pagamento.
424 @return lRet - Retorna se realizou a conferencia
425 @obs
426 @sample
427 */
428 //-----
429 Static Function STIAddListPay(nGetValue)
430 Local nX      := 0
431 Local nTotal  := 0
432
433 If !(AllTrim(aBrwContent[nListPos,1]) == AllTrim(aPaym[nListPos,2]))
434     nListPos:= aScan(aBrwContent, {|x| AllTrim(x[1])== AllTrim(aPaym[nListPos,2]) })
435 Endif
436
437 aBrwContent[nListPos,4] := AllTrim(Transform(nGetValue,cPicTotal))
438
439
440 oBrwContent:SetArray(aBrwContent)
441 oBrwContent:Refresh(.T.)
442
443 nGetValue := 0
444 nListPos  := 0
445 lListWhen := .T.
446
447 oPayList:SetFocus()
448
449 For nX := 1 To Len(aBrwContent)
450     nTotal += Val(StrTran(SubStr(aBrwContent[nX,4], 1, Len(aBrwContent[nX,4]) - 3),'.','') + '.' + SubStr(aBrwContent[nX,4],Len(aBrwContent[nX,4]) - 1, 2))
451 Next nX
452
453 oSayTotal:SetText(AllTrim(Transform(nTotal,cPicTotal)))
454
455 Return .T.
456
457 //-----
458 /*{Protheus.doc} STIConfirmConference
459 Indica a opcao selecionada no listbox e o desativa e habilita o get e o botao para adicionar pagamentos.
460
461 @param oGetValue - Get do valor de pagamento.
462 @return lRet - Retorna se realizou a conferencia
463 @obs
464 @sample
465 */
466 //-----
467 Static Function STIConfirmConference(lLeitX, lReduZ,oMainPanel)

```

```

468 Local nX          := 0
469 Local cTexto       := ''
470 Local lStiRelApu    := ExistBlock("STRECFXCX")      //PE para impressão de relatorio
471 Local lStFiCfCx     := ExistBlock("STFICFCX")       //PE para execução após finalização do processo de conferencia
472 Local nPosaPaym     := 0
473
474 Default lLeitX      := .F.
475 Default lReduZ      := .F.
476 Default oMainPanel := Nil
477
478 If ApMsgYesNo(STR0031) //#"Deseja realmente processar a conferência de caixa?"
479     If STWPOSCloseCash()
480         /* Passo os valores digitados que estao contidos no Browse para o Array de pagamentos para posteriormente gravar as informacoes */
481         For nX := 1 To Len(aPaym)
482             nPosaPaym:= aScan(aBrwContent, {|x| AllTrim(x[1]) == AllTrim(aPaym[nX][2]) })
483             aPaym[nX,POS_VALDIG] := StrTran(SubStr(aBrwContent[nPosaPaym,4], 1, Len(aBrwContent[nPosaPaym,4]) - 3),'.','') + ;
484                 '.' + SubStr(aBrwContent[nPosaPaym,4],Len(aBrwContent[nPosaPaym,4]) - 1, 2)
485         Next
486
487         STDGrvSLT(aPaym,dDataBase,.F.)
488
489         If lStiRelApu
490             cTexto := ExecBlock("STRECFXCX", .F., .F., {aPaym,;
491                 oValCaixa:cCaption,;
492                 oValEstac:cCaption,;
493                 oValPDV:cCaption,;
494                 oValDtAbert:cCaption,;
495                 oValAbHora:cCaption,;
496                 oValDtFech:cCaption,;
497                 oValFcHora:cCaption,;
498                 STDGLstNumMov()})
499         Else
500             cTexto := STIRelConf(aPaym)
501         EndIf
502
503         aBrwContent := {}
504         aPaym := {}
505
506         //Relatorio da conferencia de caixa
507         STWManagReportPrint( cTexto ,1 )
508
509         //Emite a Leitura X
510         If lLeitX
511             STBReadingX()
512         EndIf
513
514         //Emite a Redução Z
515         If lReduZ
516             STWZReduction(.F.)
517         EndIf
518
519         STIExchangePanel( { || Iif(ExistFunc("STIMataObj"),STIMataObj(oMainPanel),Nil) ,STIPanOpenCash() } )

```

```

520     STISetRDlg()
521
522     //Seta variavel statica para permitir sair do sistema apos conferencia
523     If ExistFunc("STFSetPerExit")
524         STFSetPerExit(.T.)
525     EndIf
526
527     If lStFicfCx
528         ExecBlock("STFICFCX", .F., .F.)
529     Endif
530
531     Else
532         STFMessage(ProcName(),"STOP",STR0034) //"0 correu um erro ao realizar o fechamento do Caixa!"
533         STFShowMessage(ProcName())
534     EndIf
535 EndIf
536
537 Return
538
539
540 //-----
541 /*{Protheus.doc} STIRelConf
542 Imprime relatorio gerencial da conferencia de caixa
543
544 @param aPaym - Parametro das formas de pagto e seus respectivos valores
545 @return      .T.
546 @obs
547 @sample
548 */
549 //-----
550 Static Function STIRelConf(aPaym)
551
552 Local nX           := 0                // Variavel de loop
553 Local cTexto       := ''               // Conteudo do relatorio
554 Local cTotal       := ''               // Total da Conferência de Caixa
555 Local lConfApu     := SuperGetMV("MV_LJEXAPU",.T.,.F.) //exibe a coluna do valor apurado
556 Local nTotApu      := 0                // total apurado
557 Local nTotDig      := 0                // total digitado
558
559 cTexto := CRLF + CRLF
560 cTexto += STR0009 + '.....: ' + oValCaixa:cCaption + CRLF //Caixa
561 cTexto += STR0010 + '...: ' + oValEstac:cCaption + CRLF //Estação
562 cTexto += STR0011 + '.....: ' + oValPDV:cCaption + CRLF //PDV
563 cTexto += STR0014 + '...: ' + oValDtAbert:cCaption + ' - ' + STR0016 + ': ' + AllTrim(oValAbHora:cCaption) + CRLF //Abertura //Hora
564 cTexto += STR0017 + ': ' + oValDtFech:cCaption + ' - ' + STR0016 + ': ' + AllTrim(oValFcHora:cCaption) + CRLF //Fechamento //Hora
565 cTexto += STR0029 + '.: ' + STDGLstNumMov() + CRLF //Movimento
566 cTexto += CRLF + CRLF
567 if lConfApu
568     cTexto += STR0024 + '!' + STR0025 + ' ' + STR0036 + '!' + STR0035 + CRLF //Forma //Desc. Forma //Vl. Digitado/Vl. Apurado
569 Else
570     cTexto += STR0024 + '!' + STR0025 + ' ' + STR0003 + CRLF //Forma //Desc. Forma //Valor
571 EndIf

```

```

572 cTexto += CRLF
573
574 For nX := 1 To Len(aPaym)
575   if lConfApu
576     cTexto += SubStr(aPaym[nX][1],1,5) + '!' + SubStr(aPaym[nX][2],1,17) + '!' + Str(Val(aPaym[nX][7]),9,2) + '!' + Str(aPaym[nX][8],9,2) + CRLF
577     nTotApu := nTotApu + aPaym[nX][8]
578     nTotDig := nTotDig + Val(aPaym[nX][7])
579   Else
580     cTexto += aPaym[nX][1] + '!' + SubStr(aPaym[nX][2],1,22) + Space(3) + '!' + Str(Val(aPaym[nX][7]),10,2) + CRLF
581   Endif
582 Next nX
583
584 cTotal := AllTrim(oSayTotal:cCaption)
585
586 cTexto += CRLF + CRLF
587
588 If lConfApu
589   cTexto += STR0037 + ' : ' + Str(nTotApu ,9,2) + CRLF          //Tot. Apurado
590   cTexto += STR0038 + ' : ' + Str(nTotDig ,9,2) + CRLF + CRLF  //Tot. Digitado
591 Else
592   cTexto += STR0022 + '.....: ' + cTotal + Replic("-",Int((32-Len(cTotal))/2))+Replic(CRLF,2) //Total + Tracejado
593 Endif
594
595 cTexto += STR0032 + " " + Replic("_",28) + Replic(CRLF,2) //"Ass. Caixa  :"
596 cTexto += STR0033 + " " + Replic("_",28) + Replic(CRLF,6) //"Ass. Superior:"
597
598 Return cTexto
599
600 //-----
601 /*{Protheus.doc} STVldRedZ
602 Validador da opcao de processamento da reducao Z
603
604 @param      lReduZ - Variavel que indica se será processada a redução Z no final
605              do fechamento de caixa
606 @return     lReduZ
607 @obs
608 @sample
609 */
610 //-----
611 Static Function STVldRedZ(lReduZ)
612
613 // A solicitação da senha do superior ja esta contida na função que realiza a redução Z
614 lReduz := !lReduz
615 If lReduZ
616   If !ApMsgYesNo(STR0028) //"Deseja realmente processar a Redução Z após a conferência de caixa?"
617     lReduz := .F.
618   Endif
619 Endif
620
621 Return lReduz
622
623 //-----

```

```

624 /*{Protheus.doc} VldLeitX
625 Validador da opcao de processamento da leitura X
626
627 @param      lLeitX - Validador que indica se será processada a leitura X no final
628               do fechamento de caixa
629 @return      lLeitX
630 @obs
631 @sample
632 */
633 //-----
634 Static Function VldLeitX(lLeitX)
635
636 lLeitX := !lLeitX
637 If lLeitX
638     If !STFProFile(21)[1] //Leitura X?
639         lLeitX := .F.
640     Endif
641 Endif
642
643 Return lLeitX
644
645 //-----
646 /*{Protheus.doc} STICancelConference
647 Tela para cancelamento de conferencia de caixa.
648
649 @param      oMainPanel, Objeto, contem a tela de conferencia de caixa.
650 @return      Nil
651 @obs
652 @sample
653 */
654 //-----
655 Static Function STICancelConference(oMainPanel)
656
657 Local lConfirm      := .F.
658 Local lObgFecha     := SuperGetMv("MV_LJOBGCF",,.T.)    // Obrigatorio o fechamento de caixa.
659
660 Default oMainPanel := Nil // objeto da tela quando for WebAgent precisamos matar ele ao cancelar a operação
661
662 If ApMsgYesNo(STR0030) //"Deseja realmente cancelar a conferência de caixa?"
663     lConfirm := .T.
664 EndIf
665
666 If lConfirm
667     Iif(ExistFunc("STIMataObj"),STIMataObj(oMainPanel),Nil)
668     STIRegItemInterface()
669     STISetRDlg()
670
671     //Seta variavel statica para permitir sair do sistema apos cancelamento da conferencia, caso o parametro MV_LJOBGCF esteja .T. não permite sair do sistema sem realizar a
    conferencia
672     If FindFunction("STFSetPerExit") .AND. !lObgFecha
673         STFSetPerExit(.T.)
674     EndIf

```

```
675 EndIf
676
677 Return
678
679 //-----
680 /*{Protheus.doc} STICancelConference
681 Retorna numerario atual armazena na variavel aNumerario
682
683
684 @return      aNumerario
685 @obs
686 @sample
687 */
688 //-----
689 Function STIGetNume()
690 Return aNumerario
```

Componentes/Interface/STIConsentTerm.PRW

89 linhas após remoção de metadados de autoria/data

```

1 #Include 'TOTVS.ch'
2 #INCLUDE "STPOS.CH"
3 #INCLUDE "POSCSS.CH"
4 #INCLUDE "STIConsentTerm.CH"
5
6 /*/
7 {Protheus.doc} STIConsentTerm
8
9 Função responsavel por criar o texto que será exibido no termo de consentimento
10
11 @type function
12 @return Nil, Nulo
13
14 /*/
15
16 Function STIConsentTerm()
17
18 Local llJTermCo      := ExistBlock("LjTermCo")    // Indica se ponto de entrada LjTermCo esta compilado no RP0
19 Local cTextHtml      := ""                        // Texto contendo informações presentes no Termo.
20
21 /* Template Padrão do termo de consentimento */
22 cTextHtml += '<br>'
23 cTextHtml += '<font size="10" color="black" >' + STR0001 + '</font><br>' //"Termo de consentimento"
24 cTextHtml += '<br>'
25 cTextHtml += '<br>'
26 cTextHtml += '<font size="5" color="#424242">' + STR0002 + '</font><br>' //"Informar ao cliente que os dados pessoais cadastrados serão utilizados com as finalidades de sua
    identificação na realização de: "
27 cTextHtml += '<br>'
28 cTextHtml += '<font size="5" color="#424242">' + STR0003 + '</font><br>' //" - Vendas"
29 cTextHtml += '<br>'
30 cTextHtml += '<font size="5" color="#424242">' + STR0004 + '</font><br>' //" - Devoluções"
31 cTextHtml += '<br>'
32 cTextHtml += '<font size="5" color="#424242">' + STR0005 + '</font><br>' //" - Entrega a domicílio"
33 cTextHtml += '<br>'
34 cTextHtml += '<font size="5" color="#424242">' + STR0006 + '</font><br>' //" - Análise de crédito"
35 cTextHtml += '<br>'
36 cTextHtml += '<font size="5" color="#424242">' + STR0007 + '</font><br>' //" - Títulos financeiro"
37 cTextHtml += '<br>'
38 cTextHtml += '<font size="5" color="#424242">' + STR0008 + '</font><br>' //" - Campanhas de propaganda"
39
40 If llJTermCo
41     cTextHtml := ExecBlock("LjTermCo",.F.,.F.,{cTextHtml})
42 EndIf
43
44 If Empty(cTextHtml)
45     STIIncCustomer()
46     LjGrvLog("STIConsentTerm","Ponto de entrada LjTermCo Compilado")
47     LjGrvLog("STIConsentTerm","A Tela com Termo de consentimento não será exibida pois o retorno do ponto de entrada LjTermCo esta vazio.")
48 Else
49     LjGrvLog("STIConsentTerm","Mensagem que será exibida para o termo de consentimento: ",cTextHtml)
50     STFCleanInterfaceMessage()
51     STIExchangePanel({|| STITerm(cTextHtml) })

```

```

52 Endif
53
54 return
55
56 /*/
57 {Protheus.doc} STITerm
58
59 Função responsável por Printar o parametro cTextHtml contendo as informações do termo de consentimento
60
61 @type function
62 @param cTextHtml, caracter, Texto que será exibido como termo de consentimento
63 @return oMainPanel, Objeto com informações da PaineL que será aberto.
64
65 /*/
66
67 Static Function STITerm(cTextHtml)
68
69 Local oPanelMVC      := STIGetDlg() //PaineL principal do dialog
70 Local oMainPanel     := TPanel():New(00,00,"",oPanelMVC,,,,,oPanelMVC:nWidth/2,(oPanelMVC:nHeight)/2) //PaineL do Termo de consentimento
71 Local lHtml          := .T. //Indica que o Say sera em HTML
72 Local oFont          := TFont():New('Courier new',,-18,.T.) //Fonte padrão do Say
73 Local oTexto         := TSay() //Objeto TSAY
74 Local oBtnAceito     := TButton() //Botão Aceito
75 Local oBtnDeclino    := TButton() //Botão Declino
76 Local lStiMataObj    := Iif(ExistFunc("StiMataObj"),.T.,.F.) //Verifica se a função StiMataObj() existe
77
78 oMainPanel:SetCSS(POSCSS(GetClassName(oMainPanel),CSS_PANEL_CONTEXT))
79
80 oTexto := TSay():New(020,008, {||cTextHtml},oMainPanel,,oFont,,,,.T.,,,oMainPanel:nWidth/2.10,oPanelMVC:nHeight/2,,,,,lHtml)
81 oTexto:SetCSS( POSCSS(GetClassName(oTexto),CSS_LABEL_FOCAL))
82
83 oBtnAceito:= TButton():New(POSVERT_BTNFOCAL,POSHOR_BTNFOCAL,STR0009,oMainPanel,{ || Iif(lStiMataObj,StiMataObj(oMainPanel),Nil),
    STIIncCustomer()},LARGBTN,ALTURABTN,,,,.T.)// "Aceitar"
84 oBtnAceito:SetCSS(POSCSS(GetClassName(oBtnAceito),CSS_BTN_FOCAL))
85
86 oBtnDeclino:= TButton():New(POSVERT_BTNFOCAL,POSHOR_1,STR0010,oMainPanel,{ || Iif(lStiMataObj,StiMataObj(oMainPanel),Nil), STIRegItemInterface()
    },LARGBTN,ALTURABTN,,,,.T.)// "Declinar"
87 oBtnDeclino:SetCSS(POSCSS(GetClassName(oBtnDeclino),CSS_BTN_NORMAL))
88
89 Return oMainPanel

```

Componentes/Interface/STIConsult.PRW

176 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STICONCONSULT.CH"
4
5 Static oSayName      := Nil
6 Static oSayValue     := Nil
7
8 //-----
9 /*{Protheus.doc} STIConsult
10 Interface Consulta de Produto
11
12 @param
13 @return
14 @obs
15 @sample
16 */
17 //-----
18 Function STIConsult()
19
20 STIExchangePanel( { || STIPanConsult() } )
21
22 Return
23
24
25 //-----
26 /*{Protheus.doc} STIPanConsult
27 Contexto Consulta de Produto
28
29 @param
30 @return oPanConsult      Painel Consulta de Produto
31 @obs
32 @sample
33 */
34 //-----
35 Function STIPanConsult()
36
37 Local oPanConsult      := Nil           // Painel Cancelamento de Venda
38 Local oPanBtnSale      := Nil           // Painel de Botões da Venda
39 Local oPanGetConsult   := Nil           // Painel de Get de Consulta
40 Local oPanelMVC        := STIGetPanel() // Painel Principal
41
42 /*
43     Título - "Consulta de Produto"
44 */
45 Local nSay1HorPerc      := 0.015       // Percentual de Posição Horizontal do Say "Consulta de Produto"
46 /*
47     Say - "Código do Produto"
48 */
49 Local nSay2HorPerc      := 0.03         // Percentual de Posição Horizontal do Say "Código do Produto"
50 Local nSay2VerPerc      := 0.08         // Percentual de Posição Vertical do Say "Código do Produto"
51 /*
52     Get Codigo do Produto

```

```

53 /*/
54 Local oCodProd      := Nil           // TGet Codigo do Produto
55 Local cGetProd      := Space(40)    // Variável do Get
56 Local nGetVerPerc   := 0.1          // Percentual da Posição Vertical do Get
57 Local nGetHorPerc   := 0.03         // Percentual da Posição Horizontal do Get
58 Local nGetHeiPerc   := 0.05         // Percentual de Altura do Get
59 /*/
60 Say - "Nome do Produto"
61 /*/
62 Local nSay3VerPerc   := 0.2          // Percentual de Posição Vertical do Say "Nome do Produto"
63 Local nSay3HorPerc   := 0.03         // Percentual de Posição Horizontal do Say "Nome do Produto"
64 /*/
65 Say - "Valor"
66 /*/
67 Local nSay4VerPerc   := 0.2          // Percentual de Posição Vertical do Say "Valor"
68 Local nSay4HorPerc   := 0.7          // Percentual de Posição Horizontal do Say "Valor"
69 /*/
70 Say - Set do Nome do Produto
71 /*/
72 Local oSay5          := Nil           // Say de Set do Nome do Produto
73 Local nSay5VerPerc   := 0.22         // Percentual de Posição Vertical do Say Set do Nome do Produto
74 Local nSay5HorPerc   := 0.03         // Percentual de Posição Horizontal do Say Set do Nome do Produto
75 /*/
76 Say - Set do Valor
77 /*/
78 Local oSay6          := Nil           // Say de Set do Valor
79 Local nSay6VerPerc   := 0.22         // Percentual de Posição Vertical do Say Set do Valor
80 Local nSay6HorPerc   := 0.7          // Percentual de Posição Horizontal do Say Set do Valor
81 /*/
82 Botão "Consultar Produto"
83 /*/
84 Local oButton        := Nil           // Botao "Consultar Produto"
85 Local nBtnWidPerc    := 0.25         // Percentual de Largura do Botão "Consultar Produto"
86 Local nBtnHeiPerc    := 0.08         // Percentual de Altura do Botão "Consultar Produto"
87 Local nBtnVerPerc    := 0.3          // Percentual da Posição Vertical do Botão "Consultar Produto"
88 Local nBtnHorPerc    := 0.65         // Percentual da Posição Horizontal do Botão "Consultar Produto"
89
90 /*/
91 Panel Consulta de Produto
92 /*/
93 oPanConsult := TPanel():New(00,00,"",oPanelMVC,,,,,oPanelMVC:nWidth/2,oPanelMVC:nHeight/2)
94 /*/
95 Panel Botoes da Venda
96 /*/
97 oPanBtnSale := STIPanBtnSale( oPanConsult )
98 /*/
99 Panel Get do Produto
100 /*/
101 oPanGetConsult := TPanel():New(oPanBtnSale:nHeight/2,00,"",oPanConsult,,,,,oPanConsult:nWidth/2,(oPanConsult:nHeight - oPanBtnSale:nHeight/2)/2)
102
103 /*/
104 Objetos

```

```

105 /*/
106 oSay1      := TSay():New(000,oPanGetConsult:nWidth/2 * nSay1HorPerc,{||STR0001},oPanGetConsult,,,,,.T,,,,) //"Venda > Consulta de Produto"
107
108 oSay2      := TSay():New(oPanGetConsult:nHeight/2 * nSay2VerPerc,oPanGetConsult:nWidth/2 * nSay2HorPerc,{||STR0002},oPanGetConsult,,,,,.T,,,,) //"Código do Produto"
109
110 oCodProd   := TGet():New( oPanGetConsult:nHeight/2 * nGetVerPerc,oPanGetConsult:nWidth/2 * nGetHorPerc,{|u| If(PCount())>0,cGetProd:=u,cGetProd)}, ;
111             oPanGetConsult,oPanGetConsult:nWidth/2 - (2*nGetHorPerc*oPanGetConsult:nWidth/2) ,oPanGetConsultrod:nHeight/2 *
             nGetHeiPerc,,,,,.T,,,,,"cGetProd")
112
113 oSay3      := TSay():New(oPanGetConsult:nHeight/2 * nSay3VerPerc,oPanGetConsult:nWidth/2 * nSay3HorPerc,{||STR0003},oPanGetConsult,,,,,.T,,,,)
             //"Nome do Produto"
114
115 oSay4      := TSay():New(oPanGetConsult:nHeight/2 * nSay4VerPerc,oPanGetConsult:nWidth/2 * nSay4HorPerc,{||STR0004},oPanGetConsult,,,,,.T,,,,) //"Valor"
116
117 oSay5      := TSay():New(oPanGetConsult:nHeight/2 * nSay5VerPerc,oPanGetConsult:nWidth/2 * nSay5HorPerc,{||Space(40)},oPanGetConsult,,,,,.T,,,,)
118 oSayName   := oSay5
119
120 oSay6      := TSay():New(oPanGetConsult:nHeight/2 * nSay6VerPerc,oPanGetConsult:nWidth/2 * nSay6HorPerc,{||Space(20)},oPanGetConsult,,,,,.T,,,,)
121 oSayValue  := oSay6
122
123 oButton    := TButton():New( oPanGetConsult:nHeight/2 * nBtnVerPerc,oPanGetConsult:nWidth/2 * nBtnHorPerc,STR0005,oPanGetConsult,{ || STIProdConsult(cGetProd,oSay5,oSay6)
             }, ; //"Consultar Produto"
             oPanGetConsult:nWidth/2 * nBtnWidPerc,oPanGetConsult:nHeight/2 * nBtnHeiPerc,,,,.T.)
124
125
126 oCodProd:SetFocus()
127
128 Return oPanConsult
129
130
131 //-----
132 /*/{Protheus.doc} STIProdConsult
133 Realiza a Consulta pelo código informado
134
135 @param
136 @return
137 @obs
138 @sample
139 */
140 //-----
141 Function STIProdConsult(cGetProd)
142
143 Local nValue      := 0           // Valor do Produto
144 Local aInfoItem   := {}         // Armazena informações do produto
145
146 Default cGetProd  := ""
147 Default oSayName  := Nil
148 Default oSayValue := Nil
149
150 If !(Empty(cGetProd)) .AND. (oSayName <> Nil) .AND. (oSayValue <> Nil)
151
152     aInfoItem := STWFindItem( AllTrim(cGetProd) )
153

```

```
154     If Len(aInfoItem) > 0
155
156         /*/
157         Descrição
158         /*/
159         oSayName:SetText(aInfoItem[8])
160
161         /*/
162         Valor
163         /*/
164         nValue := STBSearchPrice(aInfoItem[2])
165         oSayValue:SetText( "R$" + " " + cValToChar(nValue) )
166
167     Else
168
169         STFMessage("STIConsult","STOP",STR0006) //"Item nao Encontrado"
170         STFSHOWMessage("STIConsult")
171
172     EndIf
173
174 EndIf
175
176 Return
```

Componentes/Interface/STIContTit.prw

140 linhas após remoção de metadados de autoria/data

[illegible]

[illegible]

[illegible]

Componentes/Interface/STICustomerSelection.prw

514 linhas após remoção de metadados de autoria/data

```

1 #Include 'Protheus.ch'
2 #INCLUDE 'FWBROWSE.CH'
3 #INCLUDE "PARMTYPE.CH"
4 #INCLUDE "FWEVENTVIEWCONSTS.CH"
5 #INCLUDE "FWADAPTEREAI.CH"
6 #INCLUDE "FWMVCDEF.CH"
7 #INCLUDE "POSCSS.CH"
8 #INCLUDE "STPOS.CH"
9 #INCLUDE "STICUSTOMERSELECTION.CH"
10
11 Static aRecno           := {}
12 Static oGetSearch       := Nil //TGet Cancelamento de Item
13 Static aCartaoMA6       := {} //Para tabela de Cartões - MA6 (Integração TOTVSPDV x SIGACRD)
14 Static oGetCrdCart      := Nil //STIFilCustomerData
15 Static lPosCrd          := Nil //Ativa Integração com SIGACRD
16 Static aDataCustomers  := {} //Dados solicitados dos clientes utilizados na gravação.
17
18 //-----
19 /*{Protheus.doc} STICustomerSelection
20 Funcao responsavel por chamar a troca do painel atual pelo painel de selecao de clientes.
21
22
23 */
24 //-----
25
26 Function STICustomerSelection()
27 Local oTotal           := STFGetTot() // Recebe o Objeto totalizador
28 Local nTotalVend       := oTotal:GetValue("L1_VLRTOT") // Valor total da venda
29 Local lContinua       := .T.
30
31 If nTotalVend > 0
32     lContinua := .F.
33     STFMessage(ProcName(),"STOP",STR0005) // "Não será possível alterar o cliente pois a venda já foi iniciada."
34     STFShowMessage(ProcName())
35 EndIf
36
37 //Tratativa para LGPD
38 If lContinua .And. ExistFunc("LjPDAcesso")
39     lContinua := LjPDAcesso({"L1_NOMCLI"}) //Verifica se o usuário pode acessar uma determinada rotina devido a regra de proteção de dados.
40 EndIf
41
42 If lContinua
43     STIChangeCssBtn('oBtnClient')
44     STIExchangePanel( { || STIPanCustomerSelection() } )
45 EndIf
46
47 Return
48
49 //-----
50 /*{Protheus.doc} STIPanCustomerSelection
51 Funcao responsavel pela criacao do Painel de selecao de clientes.
52

```

```

53 @return oMainPanel - Objeto contendo o painel principal de selecao de clientes.
54 */
55 //-----
56
57 Function STIPanCustomerSelection()
58 Local oPanelMVC      := STIGetPanel()           // Objeto do Panel onde a interface de selecao de clientes sera criada
59 Local oMainPanel     := TPanel():New(00,00,"",oPanelMVC,,,,,oPanelMVC:nWidth/2,(oPanelMVC:nHeight)/2) // Painel de Get de Consulta
60 Local oLblCab        := Nil                    // Objeto do label "Seleção de Cliente"
61 Local oLblHelpGet    := Nil                    // Objeto do label Help
62
63 /*/
64 Get Combo de Busca
65 /*/
66 Local oLblList       := Nil
67 Local oGetList       := Nil                    // TComboBox Escolha do campo a ser utilizado na busca
68 Local cGetList       := ""                    // Variável do Get
69 /*/
70 Get de Busca
71 /*/
72 Local oLblSearch     := Nil                    // Label do ComboBox
73 Local cGetCustomer   := Space(40)             // Variável do Get
74 Local cGetCrdCart    := Space(16)             // Variável do Get para Cartão para Sigacrd
75 Local oLblCrdCart    := Nil                    // Label do Cartão para Sigacrd
76 Local oGetCrdCart    := Nil                    // Objeto do Cartão para Sigacrd
77 Local noLblListVert  := 0                     // Posição Vertical em oLblList
78 Local noGetListVert  := 0                     // Posição Vertical em oGetList
79 Local noGetListAlt   := 0                     // Altura em oGetList
80
81 /*/
82 Botão "Selecionar Cliente"
83 /*/
84 Local oButton        := Nil                    // Botao "Selecionar Cliente"
85 Local lBusca         := .F.                    // Variavel de Habilidade de Busca. - .F. não efetuou pesquisa de busca. - .T. efetuou pesquisa de busca
86 Local oListFont      := TFont():New("Courier New") // Fonte utilizada no listbox
87 Local bConfirm       := { || IIF(lBusca, STIFilCustomerData(oGetList),Nil)}
88 Local oButPesq       := Nil                    //Botao pesquisar
89 Local oButCan        := Nil                    //Botao cancelar
90 Local lMobile        := STFGetCfg("lMobile", .F.) //Smart Client Mobile
91 Local bFind          := { || Iif(!Empty(AllTrim(cGetCustomer))),(STIAddFilter(cGetCustomer, oGetList, @lBusca, "", 1),oGetList:SetFocus()),Nil)}
92
93 /*/
94 Variaveis do PE
95 /*/
96 Local aFields        := STDWhatFields()         //Campos que serão importados
97 Local lSTIALTC       := ExistBlock("STIALTC")   //Verifica se existe o ponto de entrada STIALTC na seleção do cliente
98 Local aCustomers     := {}                     //Dados dos clientes buscado no PE
99
100 If lSTIALTC
101     oGetList := TListBox():Create(oMainPanel, noGetListVert, POSHOR_1, {|| If(PCount())>0,cGetList:=u,cGetList}}, , LARG_LIST_CONSULT ,
        noGetListAlt,,,,.T.,bConfirm,oListFont)
102     oGetList:SetCSS( POSCSS (GetClassName(oGetList), CSS_LISTBOX ))
103

```

```

104 LjGrvLog( NIL, "Antes da execução do PE STIALTC")
105 aRet := ExecBlock("STIALTC",.F.,.F.,{aFields})
106
107 aCustomers := aRet[1]
108 aDataCustomers := aRet[2]
109 lBusca := .T.
110
111 oGetList:SetArray(aCustomers)
112 oGetList:Select(1)
113 STIFilCustomerData(oGetList)
114
115 LjGrvLog( NIL, "Depois da execução do PE STIALTC")
116 Else
117     /*
118     Objetos
119     */
120     oLblCab:= TSay():New(POSVERT_CAB,POSHOR_1,{||STR0001},oMainPanel,,,,,.T.,,,) // Seleção de Cliente
121     oLblCab:SetCSS( POSCSS (GetClassName(oLblCab), CSS_BREADCUMB ))
122
123     oLblSearch := TSay():New(POSVERT_LABEL1,POSHOR_1, {|| STR0006 }, oMainPanel,,,,,.T.) // "Pesquisar Cliente: Código / Nome / Loja / CPF/CNPJ"
124
125     oGetSearch:= TGet():New(POSVERT_GET1,POSHOR_1,{|u| If(PCount(>0,(lBusca := .F., cGetCustomer:=u),cGetCustomer)}),;
126         oMainPanel,120 ,ALTURAGET,"@!",,,,,,.T.,,,,,,"cGetCustomer")
127
128     oLblHelpGet:= TSay():New(POSVERT_GET1 + 2.5 ,POSHOR_1 + 123 ,{||STR0025},oMainPanel,,,,,.T.,,,) // "Para busca de palavras contidas utilize *"
129     oLblHelpGet:SetCSS( POSCSS (GetClassName(oLblCab), CSS_LABEL_NORMAL ))
130
131     //Botao pesquisar para versao mobile
132     If ValType(lMobile) == "L" .AND. lMobile
133         oButPesq := TButton():New( POSVERT_GET1,POSHOR_1+130,"",oMainPanel,, ;
134             20,20,,,,.T.,,,,{||.T.})
135         oButPesq:SetCSS( POSCSS (GetClassName(oButPesq), CSS_BTN_LUPA ))
136     Else
137
138         oGetSearch:bLostFocus := bFind
139
140     EndIf
141
142     oGetSearch:SetCSS( POSCSS (GetClassName(oGetSearch), CSS_GET_NORMAL ))
143     oGetSearch:bLostFocus := bFind
144
145     oLblSearch:SetCSS( POSCSS (GetClassName(oLblSearch), CSS_LABEL_FOCAL ))
146
147     If lPosCrd == nil .AND. ExistFunc("STBSetCrdIdent") .AND. ExistFunc("STBGetCrd")
148         lPosCrd := CrdxInt(.F.,.F.) // Integração com SIGACRD
149     EndIf
150
151     If lPosCrd //Se integração com CRD habilitado
152
153         //Inicialização de variáveis CRD
154         aCartaoMA6 := {}
155         STBSetCrdIdent("", "", "", "")

```

```

156
157 //Informe o Número do Cartão / Informe a Validade
158 oLblCrdCart := TSay():New(POSVERT_LABEL2,POSHOR_1, {|STR0024}, oMainPanel,,,,,.T.) // "Ou Informe o Número do Cartão"
159 oGetCrdCart := TGet():New(POSVERT_GET2,POSHOR_1,{|u| If(PCount())>0,(lBusca := .F., cGetCrdCart:=u),cGetCrdCart)},,
160 oMainPanel,120 ,ALTURAGET,"@!",,,,,,.T.,,,,,,"cGetCrdCart")
161 oGetCrdCart:SetCSS( POSCSS (GetClassName(oGetCrdCart), CSS_GET_NORMAL ))
162 oGetCrdCart:bLostFocus := { || IIF( !lBusca .AND. !Empty(cGetCrdCart) .AND. STIAddFilter("",oGetList, @lBusca, cGetCrdCart,2),(cGetCustomer :=
Space(Len(cGetCustomer))),Nil)}
163
164 oLblCrdCart:SetCSS( POSCSS (GetClassName(oLblCrdCart), CSS_LABEL_FOCAL ))
165
166 noLbllistVert := POSVERT_LABEL3
167 noGetListVert := POSVERT_GET3
168 noGetListAlt := oPanelMVC:nHeight/(4.4865+2)
169
170 Else
171
172 noLbllistVert := POSVERT_LABEL2
173 noGetListVert := POSVERT_GET2
174 noGetListAlt := ALT_LIST_CONSULT
175
176 EndIf
177
178 oLbllist := TSay():New(noLbllistVert,POSHOR_1, {|IIF(lPosCrd,STR0029,STR0007)}, oMainPanel,,,,,.T.) // Nome / Código / Loja / CPF/CNPJ / Cartão
179 oLbllist:SetCSS( POSCSS (GetClassName(oLbllist), CSS_LABEL_FOCAL ))
180
181 oGetList := TListBox():Create(oMainPanel, noGetListVert, POSHOR_1, {|u| If(PCount())>0,cGetList:=u,cGetList)}, , LARG_LIST_CONSULT ,
noGetListAlt,,,,.T.,,bConfirm,oListFont)
182 oGetList:SetCSS( POSCSS (GetClassName(oGetList), CSS_LISTBOX ))
183
184 oButton := TButton():New( POSVERT_BTNFOCAL,POSHOR_BTNFOCAL,STR0004,oMainPanel,bConfirm, ; // Selecionar Cliente
185 LARGBTN,ALTURABTN,,,,.T.)
186
187 oButton:SetCSS( POSCSS (GetClassName(oButton), CSS_BTN_FOCAL ))
188
189 oButCan := TButton():New( POSVERT_BTNFOCAL,POSHOR_1,STR0022,oMainPanel,{|| Iif(ExistFunc("STIMataObj"), STIMataObj(oMainPanel), Nil), STIRegItemInterface() }, ;
//"Cancelar"
190 LARGBTN,ALTURABTN,,,,.T.,,{|.T.})
191 oButCan:SetCSS( POSCSS (GetClassName(oButCan), CSS_BTN_NORMAL ))
192 EndIf
193
194 Return oMainPanel
195
196 //-----
197 /*{Protheus.doc} STIAddFilter
198 Responsavel por capturar os dados dos clienes conforme enviado pelo usuario.
199
200 @param cGetCustomer, Caracter, String digitada pelo usuario utilizada para pesquisar o cliente.
201 @param oGetList, objeto, Campo que será utilizado para pesquisa.
202 @param lBusca, Logico,Ativar busca
203 @param cGetCartao, Caracter,Digitar cartão para pesquisa integração SIGACRD
204 @param nTipo, Numerico,1-Só tela CNPJ, 2-Tela integração SIGACRD

```

```

205 @return Logico, indica se o processo foi concluido com sucesso.
206 */
207 //-----
208
209 Static Function STIAddFilter(cGetCustomer, oGetList, lBusca, cGetCartao, nTipo)
210 Local lBuscaLike      := .F.           // -- Indica se devera realizar a busca like
211 Local lRet            := .F.           // -- Retorno da função
212 Local aCustomers      := {}           // -- Dados dos clientes buscado
213 Local nLimitRegs      := SuperGetMV("MV_LJQTDPL",,20) // -- Limite de clientes na busca
214 Local aFields         := STDWhatFields() // -- Campos que serão importados
215 Local lIntCRD         := .F.           // -- Indica se é integração CRD
216 Local nMV_LJPEATU     := 0           // -- Define aonde será buscado o cliente (0=Pesquisa Local, 1=Pesquisa preferencialmente Local se falar pesquisa na
    retaguarda, 2=Pesquisa somente na retaguarda)
217 Local nSec1, nSec2    := 0           // -- Utilizado para
218 Local aParam          := {}           // -- Parametros utilizados para busca na retaguarda
219 Local uResult         := Nil          // -- Resultado da busca na retaguarda
220 Local aRet            := {}           // -- Retorno da função
221
222 Default cGetCustomer  := ""           // -- Busca do cliente
223 Default cGetCartao    := ""           // -- Cartão private label CRM
224 Default nTipo         := 1           // -- 1-Só tela CNPJ, 2-Tela integração SIGACRD
225
226 nSec1 := Seconds()
227 aRecno := {}
228
229 cGetCustomer := AllTrim(cGetCustomer)
230 cGetCartao   := AllTrim(cGetCartao)
231
232 // -- Caso esteja com a integração CRD ativa nTipo == 2 a consulta deverá ser sempre na retaguarda. (nMV_LJPEATU = 2)
233 If nTipo == 2
234     lIntCRD := .T.
235     nMV_LJPEATU := 2
236 Else
237     nMV_LJPEATU := SuperGetMV("MV_LJPEATU",,0)
238 EndIf
239
240 // -- É necessario que seja digitado pelo menos 3 letras para a busca, ou caso inicie com numero ignora o minimo para casos aonde é realizado a busca por codigo do cliente
    ex.: "1" ou "12" Etc..
241 If (Len(cGetCustomer) >= 3 .Or. Len(cGetCartao) > 3 ) .Or. ( Len(cGetCustomer) < 3 .AND. SubStr(cGetCustomer,1,1) $ "0123456789" )
242
243     lRet := .T.
244     // -- Identifica se é busca Like ou convencional.
245     If "*" $ cGetCustomer
246         lBuscaLike := .T.
247         cGetCustomer := Replace(cGetCustomer,"*","")
248     EndIf
249
250     If nMV_LJPEATU == 0 // -- Pesquisa Somente local
251         LjGrvLog( ProcName(),"Parametro MV_LJPEATU = 0, as busca serão realizadas apenas no PDV")
252
253         STFMessage(ProcName(), "RUN", STR0020 ,{ || aRet := STDSearchCostumer(cGetCustomer,nLimitRegs,lBuscaLike,Nil,aFields) })
254         STFShowMessage(ProcName())

```

```

255
256     aCustomers      := aRet[1]
257     aDataCustomers  := aRet[2]
258
259     If Len(aCustomers) == 0
260         LjGrvLog( ProcName(), "Nenhum cliente encontrado.")
261         STFMessage(ProcName(), "STOP", STR0009) //Nenhum cliente encontrado.
262         STFShowMessage(ProcName())
263         oGetList:SetFocus()
264     EndIf
265
266 ElseIf nMV_LJPEATU == 2 // -- Pesquisa Somente na retaguarda
267     LjGrvLog( ProcName(), "Parametro MV_LJPEATU = 2, as busca serão realizadas apenas na Retaguarda")
268
269     aParam := {cGetCustomer, nLimitRegs, .T., Nil, aFields, lIntCRD, cGetCartao}
270
271     STFMessage(ProcName(), "RUN", STR0020 ,{ || lRet := STBRemoteExecute("STDSearchCostumer", aParam,,, @uResult) })
272     STFShowMessage(ProcName())
273
274     If lRet
275         aCustomers      := uResult[1]
276         aDataCustomers  := uResult[2]
277         aCartaoMA6      := uResult[3]
278     Else
279         LjGrvLog( ProcName(), "Sem comunicação com o servidor.")
280         STFMessage(ProcName(), "STOP", STR0028) //"Sem comunicação com o servidor."
281         STFShowMessage(ProcName())
282     EndIf
283
284     If Len(aCustomers) == 0
285         LjGrvLog( ProcName(), "Nenhum cliente encontrado.")
286         STFMessage(ProcName(), "STOP", STR0009) //Nenhum cliente encontrado.
287         STFShowMessage(ProcName())
288         oGetList:SetFocus()
289     EndIf
290 Else
291
292     LjGrvLog( ProcName(), "Parametro MV_LJPEATU = 1, as busca serão realizadas no PDV e caso nenhum cliente seja encontrado será realizada uma nova busca na retaguarda")
293
294     STFMessage(ProcName(), "RUN", STR0020 ,{ || aRet := STDSearchCostumer(cGetCustomer, nLimitRegs, lBuscaLike, Nil, aFields) })
295     STFShowMessage(ProcName())
296
297     aCustomers      := aRet[1]
298     aDataCustomers  := aRet[2]
299
300     If Len(aCustomers) == 0
301
302         // -- Se não encontrou nenhum cliente realiza a busca na retaguarda.
303         STFMessage(ProcName(), "YESN0", STR0018) //Cliente não encontrado na base local. Deseja realizar a busca no servidor?
304
305         LjGrvLog( ProcName(), "Nenhum cliente encontrado nas busca no PDV.")
306

```

```

307     If STFSShowMessage(ProcName())
308         LjGrvLog( ProcName(),"Usuario optou por realizar nova busca na retaguarda")
309
310         aParam := {cGetCustomer,nLimitRegs,.T.,Nil,aFields,lIntCRD,cGetCartao}
311
312         STFMessage(ProcName(), "RUN", STR0020 ,{ || lRet := STBRemoteExecute("STDSearchCostumer", aParam,,, @uResult) })
313         STFSShowMessage(ProcName())
314
315         If lRet
316             aCustomers      := uResult[1]
317             aDataCustomers  := uResult[2]
318             aCartaoMA6      := uResult[3]
319         Else
320             LjGrvLog( ProcName(),"Sem comunicação com o servidor.")
321             STFMessage(ProcName(),"STOP",STR0028) //"Sem comunicação com o servidor."
322             STFSShowMessage(ProcName())
323         EndIf
324
325         If Len(aCustomers) == 0
326             LjGrvLog( ProcName(),"Nenhum cliente encontrado.")
327             STFMessage(ProcName(),"STOP",STR0009) //Nenhum cliente encontrado.
328             STFSShowMessage(ProcName())
329             oGetList:SetFocus()
330         EndIf
331     Else
332         oGetList:SetFocus()
333         LjGrvLog( ProcName(),"Nenhum cliente encontrado nas busca no PDV e o usuario optou por não realizar buscas na retaguarda.")
334     EndIf
335 EndIf
336
337 EndIf
338
339 If Len(aCustomers) = 0
340     lRet := .F.
341     oGetList:Reset()
342 Else
343     lBusca := .T.
344     oGetList:SetArray(aCustomers)
345     oGetList:SetFocus()
346     oGetList:GoTop()
347 EndIf
348
349 Else
350     LjGrvLog( ProcName(),"É necessário digitar pelo menos 3 caracteres.")
351     STFMessage(ProcName(),"STOP",STR0008) //É necessário digitar pelo menos 3 caracteres.
352     STFSShowMessage(ProcName())
353     oGetList:SetFocus()
354 EndIf
355
356 nSec2 := Seconds()-nSec1
357
358 Conout("-----")

```

```

359 ConOut(Str(nSec2))
360 Conout("Tempo de espera na pesquisa de clientes: " + AllTrim(Str(nSec2)))
361 LjGrvLog( "Cliente", "Tempo de espera na pesquisa de clientes: " + AllTrim(Str(nSec2)) )
362 Conout("-----")
363
364 Return lRet
365
366 //-----
367 /* {Protheus.doc} STIFilCustomerData
368 Funcao responsavel por setar os dados do cliente selecionado na tabela SL1, além de carregar um model com as informações do cliente
369
370 */
371 //-----
372 Function STIFilCustomerData(oGetList)
373
374 Local oModelCli      := Nil      // Model do cliente selecionado
375 Local nPos           := oGetList:GetPos()
376 Local nRecno         := IIF(Len(aRecno) > 0, aRecno[nPos], 0)
377 Local cCNPJ          := ""       //Cnpj do cliente
378 Local cNome          := ""       //Nome do cliente
379 Local cEnd           := ""       //Endereco do cliente
380 Local nCallCPF       := SuperGetMV("MV_LJDCCLI",,0) //0 momento onde será mostrado o CPF na tela
381 Local lUtilizaCPF    := .F.      //Faz a pergunta se utiliza CPF caso não for NFC-e
382 Local cCartaoMA6     := ""       //Cartão MA6
383 Local nCustomer      := 0        //Variavel utilizada para For
384 Local aCampos        := {}       //Campos que serão gravados
385 Local aDados         := {}       //Dados dos campos que serão gravados.
386 Local lRet           := .T.      //Retorno da função do PE STValidCli
387 Local lEmiteNFe      := .F.      //Se emite NF pelo PDV envia o parametro para MafisIni()
388
389 //Cadastra o cliete selecionado caso não exista localmente
390 If nPos > 0 .AND. nPos <= Len(aDataCustomers)
391     nRecno := STDVerfCadCli(aDataCustomers[nPos][2][2] + aDataCustomers[nPos][3][2])[2]
392     If nRecno == 0
393         aADD(aDados,{})
394         For nCustomer := 1 To len(aDataCustomers[nPos])
395             aADD(aCampos,aDataCustomers[nPos][nCustomer][1])
396             aADD(aDados[Len(aDados)] ,aDataCustomers[nPos][nCustomer][2])
397         Next
398
399         STIConfCus( aDataCustomers[nPos][2][2] ,aDataCustomers[nPos][3][2] ,aDataCustomers[nPos][1][2] ,aDataCustomers[nPos][5][2] ,;
400                   aDataCustomers[nPos][6][2] ,aDataCustomers[nPos][4][2] ,aDataCustomers[nPos][7][2] ,aDataCustomers[nPos][8][2] ,;
401                   aDataCustomers[nPos][9][2] ,aDataCustomers[nPos][10][2],.T. ,.F. ,;
402                   aDataCustomers[nPos][11][2] ,aDataCustomers[nPos][12][2],aDataCustomers[nPos][13][2],aCampos ,;
403                   aDados ,.F. , "TX" ) //Cadastra Cliente
404
405         //Apos o cadastro Pega o numero do recno.
406         nRecno := STDVerfCadCli(aDataCustomers[nPos][2][2] + aDataCustomers[nPos][3][2])[2]
407     EndIf
408
409 EndIF
410

```

```

411 If nPos > 0 .AND. !Empty(nRecno)
412
413   SA1->(DbGoTo(nRecno))
414
415   If ExistBlock("STValidCli")
416     lRet := ExecBlock("STValidCli",.F.,.F.,{SA1->A1_COD,SA1->A1_LOJA})
417   EndIf
418
419
420   If lRet
421
422     oModelCli := STWCustomerSelection(SA1->A1_COD+SA1->A1_LOJA)
423
424     If ExistFunc("STBEmitenF") //Define se vai emitir NF-e
425       lEmitenFe:= STBEmitenF()
426     EndIf
427
428     STDSPBasket("SL1","L1_CLIENTE" ,oModelCli:GetValue("SA1MASTER","A1_COD"))
429     STDSPBasket("SL1","L1_LOJA" ,oModelCli:GetValue("SA1MASTER","A1_LOJA"))
430     STDSPBasket("SL1","L1_TIPOCLI" ,oModelCli:GetValue("SA1MASTER","A1_TIPO"))
431
432     If lEmitenFe
433       STDSPBasket("SL1","L1_TPFRET","S")
434     Endif
435
436     If !STBTaxFoun("IT", 1)
437       STBTaxEnd()
438       STBTaxIni( oModelCli:GetValue("SA1MASTER","A1_COD"), ;
439                 oModelCli:GetValue("SA1MASTER","A1_LOJA"), ;
440                 "C", ;
441                 "S", ;
442                 .F., ;
443                 "SB1", ;
444                 "LOJA701", ;
445                 .T., ;
446                 oModelCli:GetValue("SA1MASTER","A1_TIPO"), ;
447                 iif(lEmitenFe,"S",""), ;
448                 lEmitenFe
449             Endif
450
451     cCNPJ := oModelCli:GetValue("SA1MASTER","A1_CGC")
452     cNome := oModelCli:GetValue("SA1MASTER","A1_NOME")
453     cEnd := oModelCli:GetValue("SA1MASTER","A1_END")
454
455     If (Empty(cCNPJ) .And. !STFGetCfg("lUseSat")) .And. !Empty(oModelCli:GetValue("SA1MASTER","A1_PFISICA"))//Para cliente estrangeiro na NFC-e
456       cCNPJ := AllTrim(oModelCli:GetValue("SA1MASTER","A1_PFISICA"))
457     EndIf
458
459     STISCnpjRec( cCNPJ ) //Seta o Cpf/CNPJ do cliente para ser utilizado no Panel de Recebimento de Titulo
460
461     //Responsável por setar codigo do cliente e codigo da loja para o recebimento de titulo
462     STWSCliLoj(oModelCli:GetValue("SA1MASTER","A1_COD"), oModelCli:GetValue("SA1MASTER","A1_LOJA"))

```

```

463
464 // Integração SIGACRD x TOTVSPDV
465 If lPosCrd
466     cCartaoMA6 := ""
467     If Len(aCartaoMA6) >= nPos
468         cCartaoMA6 := aCartaoMA6[nPos]
469     EndIf
470     STBSetCrdIdent(cCartaoMA6, AllTrim(SA1->A1_CGC), AllTrim(SA1->A1_COD), AllTrim(SA1->A1_LOJA))
471 EndIf
472
473 STFMessage(ProcName(), "STOP", STR0015) // "Cliente Selecionado"
474 STFShowMessage(ProcName())
475 aRecno := {}
476
477 If !( STIPosG0pc() $ "10|11" )
478     If (nCallCPF = 0 .OR. nCallCPF = 1) //0 CPF mostrará antes do lancto. dos itens
479         If !Empty(cCNPJ)
480             STFMessage("STICLICNPJ", "YESNO", STR0017) // "Deseja utilizar o CPF do cliente selecionado para impressão do Comprovante de Venda?"
481             lUtilizaCPF := STFShowMessage("STICLICNPJ")
482         EndIf
483         If !lUtilizaCPF //Se não utilizar o CPF, limpo também o nome e o endereço
484             cCNPJ := ""
485             cNome := ""
486             cEnd := ""
487         EndIf
488         STD7CPF0verReceipt(cCNPJ, cNome, cEnd) //Armazeno nas variáveis de impressão do Cupom Fiscal
489     EndIf
490     If Len(cCNPJ) > 9
491         STDSPBasket("SL1", "L1_CGCCLI", cCNPJ) //Guarda CPF do cliente, permite alterar/limpa na tela padrão de CPF
492     Else
493         STDSPBasket("SL1", "L1_PFIISICA", cCNPJ)
494     EndIf
495 EndIf
496
497 If STIPosG0pc() == "10" .Or. STIPosG0pc() == "11"
498     If STIPosG0pc() == "10"
499         STIPanelReceb('R') //"Recebimento de Titulo"
500     ElseIf STIPosG0pc() == "11"
501         STIPanelReceb('E') //"Estorno de Titulo"
502     EndIf
503 Else
504     STIRegItemInterface()
505 EndIf
506
507 Endif
508 Else
509     STFMessage(ProcName(), "STOP", STR0016) // "Nenhum cliente foi selecionado."
510     STFShowMessage(ProcName())
511     oGetSearch:SetFocus()
512 EndIf
513
514 Return

```

Componentes/Interface/STIDataCheck.prw

556 linhas após remoção de metadados de autoria/data

[illegible]

```

53
54 Local oLblCheq      := Nil //Objeto label cheque
55 Local oGetCheq      := Nil //Objeto get cheque
56 Local cGetChk       := "" //Cheques
57
58 Local oLblBan       := Nil //Objeto label banco
59
60 Local oLblAge        := Nil //Objeto label agencia
61 Local oGetAge        := Nil //Objeto get agencia
62
63 Local oLblConta      := Nil //Objeto label conta
64 Local oGetConta      := Nil //Objeto get conta
65
66 Local oLblNuChk     := Nil //Objeto label numero do cheque
67
68 Local oLblComp       := Nil //Objeto label compensacao
69 Local oGetComp       := Nil //Objeto get compensacao
70
71 Local oLblTel        := Nil //Objeto label telefone
72 Local oGetTel        := Nil //Objeto get telefone
73
74 Local oLblRg         := Nil //Objeto label rg
75 Local oGetRg         := Nil //Objeto get rg
76
77 Local oCheckBox      := Nil //Objeto checkbox
78 Local oGrpChk        := Nil //Objeto groupbox
79 Local oCheckNro      := Nil //Objeto checkbox
80
81 Local nLineCab       := oPnlAdconal:nHeight/26 // Posicao horizontal da linha de gets do cabecalho
82 Local nColCab2       := oPnlAdconal:nWidth/6.1 // Posicao vertical da segunda coluna de objetos do cabecalho
83 Local nColCab3       := oPnlAdconal:nWidth/3.05 // Posicao vertical da terceira coluna de objetos do cabecalho
84
85 Local nLine1         := oPnlAdconal:nHeight/7.8 // Posicao horizontal da primeira linha de objetos (labels)
86 Local nLine2         := oPnlAdconal:nHeight/6 // Posicao horizontal da segunda linha de objetos (gets)
87 Local nLine3         := oPnlAdconal:nHeight/4.333333333 // Posicao horizontal da terceira linha de objetos (labels)
88 Local nLine4         := oPnlAdconal:nHeight/3.7142857 // Posicao horizontal da quarta linha de objetos (gets)
89 Local nLine5         := oPnlAdconal:nHeight/3.1142857 // Posicao horizontal da quinta linha de objetos (labels)
90 Local nLine6         := oPnlAdconal:nHeight/2.8142857 // Posicao horizontal da sexta linha de objetos (gets)
91 Local nCol2          := oPnlAdconal:nWidth/8.133333333 // Posicao vertical da segunda coluna de objetos
92 Local nCol3          := oPnlAdconal:nWidth/4.2068965517 // Posicao vertical da terceira coluna de objetos
93 Local nCol4          := oPnlAdconal:nWidth/2.8372093 // Posicao vertical da quarta coluna de objetos
94 Local nDiferenca     := 0
95 Local oPanelMVC      := STIGetPanel()
96 Local oLblEmi        := Nil //Objeto label emitente
97 Local oGetEmi        := Nil //Objeto get emitente
98 Local bEmiTrue       := {|oLblEmi:lVisibleControl:=.T., oGetEmi:lVisibleControl:=.T., oGetEmi:SetFocus()}
99 Local bEmiFalse      := {|oLblEmi:lVisibleControl:=.F., oGetEmi:lVisibleControl:=.F., oGetBan:SetFocus()}
100 Local lReadOnly      := ExistFunc("STIGetPayR0") .AND. STIGetPayR0() //indica se os campos de pagamento estão como Somente Leitura (permissão Alterar Parcelas do caixa)
101 Local nI             := 0
102 Local nMvLJINTER     := SuperGetMV("MV_LJINTER",, 30)
103 Local lFormaImp       := ExistFunc("STBGFormImp") .And. STBIsImpOrc() .And. !Empty( AllTrim(STDGPBasket("SL1","L1_CONDPG")) )
104 Local aCheqImp       := {}

```

```

105 Local nPosImp      := 0
106 Local aDiasPgChck  := IIF(ExistFunc("STBGetDPgChck"), STBGetDPgChck(), {} )
107
108 Default bCreatePan  := Nil
109 Default aRet        := {}
110 Default nContBtn    := 1
111
112 If !IsInCallStack("STICANCDTCHK")
113   If Len(aRet) > 0
114     For nI := 1 To Len(aRet)
115       cGetVal := cGetVal + aRet[nI][1][11]
116     Next nI
117
118     cGetValTot := cGetValTot - cGetVal
119     cGetParc   := cGetParc - (nContBtn - 1)
120
121     If aRet[nContBtn - 1][1][11] == nValBkp
122       cGetVal := STBRound(oMdlMst:GetValue("L4_VALOR") / oMdlMst:GetValue("L4_PARCELAS"))
123     Else
124       cGetVal := STBRound(cGetValTot / cGetParc, 2)
125     EndIf
126   Else
127     cGetVal := STBRound(cGetValTot / cGetParc, 2)
128     nValBkp := cGetVal
129   EndIf
130
131   nDiferenca := oMdlMst:GetValue("L4_VALOR") - (STBRound( oMdlMst:GetValue("L4_VALOR") / oMdlMst:GetValue("L4_PARCELAS"),2) * oMdlMst:GetValue("L4_PARCELAS"))
132
133   If nContBtn == oMdlMst:GetValue("L4_PARCELAS") .AND. nDiferenca <> 0 .AND. STBRound(cGetVal, 2) == (STBRound(oMdlMst:GetValue("L4_VALOR") /
oMdlMst:GetValue("L4_PARCELAS"),2))
134     cGetVal := cGetVal + nDiferenca
135   EndIf
136 Else
137   cGetVal := aRet[nContBtn][1][11]
138 EndIf
139
140 /* Essas variaveis sao utilizadas qdo a
141 parcela for maiores do que um */
142 bPan := bCreatePan
143
144 cGetChk := AllTrim(Str(nContBtn)) + ' '+STR0013+ ' ' + AllTrim(Str(oMdlMst:GetValue("L4_PARCELAS"))) //Cheques           // de
145
146 If Len(aDiasPgChck) > 0
147   cGetDt := cGetDt + Val( aDiasPgChck[nContBtn] )
148 Else
149   cGetDt := cGetDt + ((nContBtn - 1) * nMvLJINTER)
150 EndIf
151
152 //Tratamento para guardar as informações dos pagamentos em CH do orçamento importado, para que seus valores sejam mantidos no PDV.
153 If lFormaImp
154   aCheqImp := STBGFormImp()
155   If (nPosImp := aScan(aCheqImp, {|x| AllTrim(x[1])=="CH" .And. x[4]==nContBtn }) > 0

```

```

156         cGetVal := aCheqImp[nPosImp][2]          //Valor do cheque
157         cGetDt  := SToD(aCheqImp[nPosImp][3])     //Vencimento do cheque
158     EndIf
159 EndIf
160
161 /* Label e Get: Valor */
162 oLblValor := TSay():New(000, POSHOR_1, {|STR0001}, oPnlAdconal,,,,,.T.,,,8) //'Valor'
163
164 oGetValor := TGet():New(nLineCab,POSHOR_1,{|u| If(PCount()>0,cGetVal:=u,cGetVal)},oPnlAdconal,,ALTURAGET,"@E
    99,999,999.99",/*bValid*/,/*nClrFore*/,/*nClrBack*/,/*oFont*/,/*uParam12*/,/*uParam13*/,.T./*lPixel*/,/*uParam15*/,/*uParam16*/,/*bWhen*/,/*lCenter*/,/*lRight*/,/*bChange
    */,lReadOnly /*readonly*/)
165
166 oLblValor:SetCSS( POSCSS (GetClassName(oLblValor), CSS_LABEL_FOCAL ))
167 oGetValor:SetCSS( POSCSS (GetClassName(oGetValor), CSS_GET_NORMAL ))
168
169 /* Label e Get: Dt Vencimento */
170 oLbldtVen := TSay():New(000, nColCab2, {|STR0002}, oPnlAdconal,,,,,.T.,,,8) //'Dt. Vencimento'
171 oGetDtVen := TGet():New(nLineCab,nColCab2,{|u| If(PCount()>0,cGetDt:=u,cGetDt)},oPnlAdconal,60,ALTURAGET,/*cPicture*/,/*bValid*/,/*nClrFore*/,/*nClrBack*/,/*oFont*/,/*uParam1
    2*/,/*uParam13*/,.T./*lPixel*/,/*uParam15*/,/*uParam16*/,/*bWhen*/,/*lCenter*/,/*lRight*/,/*bChange*/,lReadOnly /*readonly*/)
172
173
174 oLbldtVen:SetCSS( POSCSS (GetClassName(oLbldtVen), CSS_LABEL_FOCAL ))
175 oGetDtVen:SetCSS( POSCSS (GetClassName(oGetDtVen), CSS_GET_NORMAL ))
176
177 /* Label e Get: Cheque */
178 oLblCheq := TSay():New(000, nColCab3, {|STR0003}, oPnlAdconal,,,,,.T.,,,8) //'Cheque'
179 oGetCheq := TGet():New(nLineCab,nColCab3,{|u| If(PCount()>0,cGetChk:=u,cGetChk)},oPnlAdconal,50,ALTURAGET,/*cPicture*/,/*bValid*/,/*nClrFore*/,/*nClrBack*/,/*oFont*/,/*uParam
    12*/,/*uParam13*/,.T./*lPixel*/,/*uParam15*/,/*uParam16*/,/*bWhen*/,/*lCenter*/,/*lRight*/,/*bChange*/,.T. /*readonly*/)
180
181 oLblCheq:SetCSS( POSCSS (GetClassName(oLblCheq), CSS_LABEL_FOCAL ))
182 oGetCheq:SetCSS( POSCSS (GetClassName(oGetCheq), CSS_GET_NORMAL ))
183
184 /* Objeto TGroup */
185 oGrpChk := TGroup():Create(oPnlAdconal, oPnlAdconal:nHeight/9.75, oPanelMVC:nWidth/65.5, oPnlAdconal:nHeight/2.55, oPnlAdconal:nWidth/2.02995, ' ',,,.T.)
186
187 /* Label e Get: Banco */
188 oLblBan := TSay():New(nLine1,POSHOR_1, {|STR0004}, oGrpChk,,,,,.T.,,,8) //'Banco'
189 oGetBan := TGet():New(nLine2,POSHOR_1,{|u| If(PCount()>0,cGetBan:=u,cGetBan)},oGrpChk,,ALTURAGET,"@!",,,,,,.T.)
190 If lLGPD .And. LjPDCmpPrt("EF_BANCO")
191     LjPDOfuscar(oGetBan, "EF_BANCO")
192 EndIf
193
194 oLblBan:SetCSS( POSCSS (GetClassName(oLblBan), CSS_LABEL_FOCAL ))
195
196 /* Label e Get: Agencia */
197 oLblAge := TSay():New(nLine1, nCol2, {|STR0005}, oGrpChk,,,,,.T.,,,8) //'Agencia'
198 oGetAge := TGet():New(nLine2,nCol2,{|u| If(PCount()>0,cGetAge:=u,cGetAge)},oGrpChk,,ALTURAGET,"@!",,,,,,.T.)
199 If lLGPD .And. LjPDCmpPrt("EF_AGENCIA")
200     LjPDOfuscar(oGetAge, "EF_AGENCIA")
201 EndIf
202
203 oLblAge:SetCSS( POSCSS (GetClassName(oLblAge), CSS_LABEL_FOCAL ))

```

```

204
205 /* Label e Get: Conta */
206 oLblConta := TSay():New(nLine1, nCol3, {||STR0006}, oGrpChk,,,,,T,,,,8) //'Conta'
207 oGetConta := TGet():New(nLine2,nCol3,{|u| If(PCount()>0,cGetCon:=u,cGetCon)},oGrpChk,50,ALTURAGET,"@!",,,,,,T.)
208 If lLGPD .And. LjPDCmpPrt("EF_CONTA")
209     LjPDOfuscar(oGetConta, "EF_CONTA")
210 EndIf
211
212 oLblConta:SetCSS( POSCSS (GetClassName(oLblConta), CSS_LABEL_FOCAL ))
213
214 /* Label e Get: Nro Cheque */
215 oLblNuChk := TSay():New(nLine1, nCol4, {||STR0007}, oGrpChk,,,,,T,,,,8) //'Nro Cheque'
216 oGetNuChk := TGet():New(nLine2,nCol4,{|u| If(PCount()>0,cGetNum:=u,cGetNum)},oGrpChk,50,ALTURAGET,"@!",,,,,,T.)
217 If lLGPD .And. LjPDCmpPrt("EF_NUM")
218     LjPDOfuscar(oGetNuChk, "EF_NUM")
219 EndIf
220
221 oLblNuChk:SetCSS( POSCSS (GetClassName(oLblNuChk), CSS_LABEL_FOCAL ))
222
223 /* Label e Get: Compensacao */
224 oLblComp := TSay():New(nLine3, POSHOR_1, {||STR0008}, oGrpChk,,,,,T,,,,8) //'Compensação'
225 oGetComp := TGet():New(nLine4,POSHOR_1,{|u| If(PCount()>0,cGetCom:=u,cGetCom)},oGrpChk,,ALTURAGET,,,,,,T.)
226
227 oLblComp:SetCSS( POSCSS (GetClassName(oLblComp), CSS_LABEL_FOCAL ))
228
229 /* Label e Get: Telefone */
230 oLblTel := TSay():New(nLine3, nCol2, {||STR0009}, oGrpChk,,,,,T,,,,8) //'Telefone'
231 oGetTel := TGet():New(nLine4,nCol2,{|u| If(PCount()>0,cGetTel:=u,cGetTel)},oGrpChk,50,ALTURAGET,"@!",,,,,,T.)
232 If lLGPD .And. LjPDCmpPrt("EF_TEL")
233     LjPDOfuscar(oGetTel, "EF_TEL")
234 EndIf
235
236 oLblTel:SetCSS( POSCSS (GetClassName(oLblTel), CSS_LABEL_FOCAL ))
237
238 /* Label e Get: Rg */
239 oLblRg := TSay():New(nLine3, nCol3, {||STR0018}, oGrpChk,,,,,T,,,,8) //'Rg'
240 oGetRg := TGet():New(nLine4,nCol3,{|u| If(PCount()>0,cGetRg:=u,cGetRg)},oGrpChk,50,ALTURAGET,"@!",,,,,,T.)
241 If lLGPD .And. LjPDCmpPrt("EF_RG")
242     LjPDOfuscar(oGetRg, "EF_RG")
243 EndIf
244
245 oLblRg:SetCSS( POSCSS (GetClassName(oLblRg), CSS_LABEL_FOCAL ))
246
247 /* CheckBox */
248 oCheckBox := TCheckBox():New(nLine6,POSHOR_1,STR0010,{| lEmi},oGrpChk,80,/*nHeight*/,/*uParam8*/,{| IIF(lEmi,lEmi=.F.,lEmi=.T.),IIF(lEmi,Eval(bEmiTrue),Eval(bEmiFalse))},,
/*bValid*/,/*nClrText*/,/*nClrPane*/,/*uParam14*/) //Cheque de terceiro
249 oCheckNro := TCheckBox():New(nLine4,nCol4,STR0014,{| lNro},oGrpChk,80,/*nHeight*/,/*uParam8*/,{| IIF(lNro,lNro=.F.,lNro=.T.),
STIValButton()},,/*bValid*/,/*nClrText*/,/*nClrPane*/,/*uParam14*/, /*lPixel*/ , /*cMsg*/ , /*uParam17*/ , { || IIf( nContBtn == oMdIst:GetValue("L4_PARCELAS") .Or.
lFormaImp, .F., .T.) }/*bWhen*/ ) //Nro cheque sequencial
250
251 /* Label e Get: Emitente */
252 oLblEmi := TSay():New(nLine5, oPnlAdconal:nWidth/6.133333333, {||STR0012}, oGrpChk,,,,,T,,,,8) //'Emitente"

```

[illegible]

```

304 Local nContBtnBkp      := 1           // Variavel para guardar a tela que foi clicado em Nro Sequencial
305 Local nDiferenca      := 0           // Verifica se há diferenca nas parcelas quando a divisao for dizima
306 Local nParcelas       := 0           // Armazena quantidade de parcelas restantes
307 Local nIntervalo      := SuperGetMV("MV_LJINTER") //Intervalo das parcelas
308 Local nDataNSeq       := 0           // Calculo data para CH com numero sequencial
309
310 Default cGetDt := ""
311
312 nParcelas := oMdlMst:GetValue("L4_PARCELAS") - ( nContBtn - 1 )
313
314 //ÚAAAAAAAAAAAAAAAAAAAAAAAAA
315 //³Estrutura do array aRet ³
316 //³1 - Banco do Cheque    ³
317 //³2 - Numero do Cheque   ³
318 //³3 - Agencia do Cheque  ³
319 //³4 - Conta do Cheque    ³
320 //³5 - Compensacao        ³
321 //³6 - Emitente           ³
322 //³7 - Telefone           ³
323 //³8 - RG                 ³
324 //³9 - Checkbox Emitente  ³
325 //³10- Data               ³
326 //³11- Valor              ³
327 //AAAAAAAAAAAAAAAAAAAAAAAAA
328
329 If !lNro
330     nContBtn += 1
331     If Len(aRet) == 0 .OR. Len(aRet) < oMdlMst:GetValue("L4_PARCELAS")
332         If Len(aRet) < (nContBtn - 1)
333             Aadd(aRet, {{cGetBan, cGetNum, cGetAge, cGetCon, cGetCom, cGetEmi, cGetTel, cGetRg, lEmi, cGetDt, cGetVal}})
334         EndIf
335     EndIf
336 Else
337     For nI := 1 To nParcelas
338         If nI == 1
339             cNroCheq := AllTrim(cGetNum)
340         Else
341             cNroCheq := Soma1(AllTrim(cNroCheq), Len(AllTrim(cNroCheq)))
342         EndIf
343
344         If nContBtn == nParcelas .AND. (cGetVal * nParcelas) <> cGetValTot
345             nDiferenca := STBRound( oMdlMst:GetValue("L4_VALOR") - (( STBRound( oMdlMst:GetValue("L4_VALOR") / oMdlMst:GetValue("L4_PARCELAS"),2) ) *
oMdlMst:GetValue("L4_PARCELAS")), 2)
346             cGetVal := cGetVal + nDiferenca
347         EndIf
348
349         nDataNSeq := cGetDt + If(nI = 1, 0, nIntervalo * (nI - 1))
350
351         Aadd(aRet, {{cGetBan, cNroCheq, cGetAge, cGetCon, cGetCom, cGetEmi, cGetTel, cGetRg, lEmi, nDataNSeq, cGetVal}})
352
353         nContBtn += 1
354     Next nI

```

```

355
356     nContBtnBkp := nParcelas
357
358     nContBtn := oMdlMst:GetValue("L4_PARCELAS") + 1
359 EndIf
360
361 For nI := 1 To Len(aRet)
362     If Len(aRet[nI][1]) >= 11
363         nValParc := nValParc + aRet[nI][1][11]
364     Else
365         nValParc := nValParc + aRet[nI][1][10]
366     EndIf
367 Next nI
368
369 If (nValParc <= oMdlMst:GetValue("L4_VALOR"))
370     If !lValField .OR. (STWValField(STR0004, cGetBan) .AND. STWValField(STR0005, cGetAge) .AND. STWValField(STR0006, cGetCon) .AND. STWValField(STR0007, cGetNum)) //Valida o
        preechimento dos dados do cheque
371
372     If (nContBtn - 1) == oMdlMst:GetValue("L4_PARCELAS") .And. nValParc == oMdlMst:GetValue("L4_VALOR")
373         oPnlAdconal:Hide()
374
375         STWSetCkRet(aRet)
376         STIClearVar()
377         STWConChk()
378
379         STIEnblPaymentOptions()
380
381         If ! (ExistFunc("STIBlqMnTef") .And. !STIBlqMnTef())
382             STIBtnActivate()
383         EndIf
384     ElseIf (nContBtn - 1) < oMdlMst:GetValue("L4_PARCELAS")
385         oPnlAdconal:Hide()
386
387         STIDataCheck(bPan, nContBtn,aRet)
388         oGetBan:SetFocus()
389
390         STIEnblPaymentOptions()
391     Else
392         lReajusta := .T.
393
394         STFMessage( ProcName(),"ALERT", STR0021) // Valor total da forma é inferior ao valor da venda.
395         STFShowMessage( ProcName() )
396     EndIf
397 Else
398     lReajusta := .T.
399 EndIf
400 Else
401     lReajusta := .T.
402
403     STFMessage( ProcName(),"ALERT", STR0020) // Valor total da forma ultrapassa valor da venda.
404     STFShowMessage( ProcName() )
405 EndIf

```

[illegible]

[illegible]

[illegible]

Componentes/Interface/STIDiscountBonif.prw

168 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "Protheus.ch"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "POSCSS.CH"
4 #INCLUDE "STPOS.CH"
5 #INCLUDE "StiDiscountBonif.CH"
6
7 //-----
8 /*{Protheus.doc} STIDscBonf()
9 Interface para decisão: a)Bonificação, b)Desconto.
10 Isto ocorrerá quando escolheu bonificação e descst total ao mesmo tempo.
11 OBS: Entrará somente se Regra de Desconto validada anteriormente.
12 @type function
13
14
15 @param nil
16
17 @return oPanBonif {Objeto para retorno em STIExchangePanel}
18 */
19 //-----
20 Function STIDscBonf()
21
22 Local oPanelMVC      := STIGetPanel()           // Objeto do Panel onde a interface de selecao de clientes sera criada
23
24 Local oLblAviso      := Nil                    //Objeto de Label Aviso
25 Local oLblDescri     := Nil                    //Objeto de Label Descrição
26 Local oLblBonif      := Nil                    //Objeto de Label Bonificação
27 Local oGetBonif      := Nil                    //Objeto de Get Bonificação
28 Local oLblDesc       := Nil                    //Objeto de Label Desconto
29 Local oGetDesc       := Nil                    //Objeto de Get Desconto
30
31 Local oBtnBonif      := Nil                    //Objeto do botao Ok
32 Local oBtnDesc       := Nil                    //Objeto do botao Cancelar
33
34 Local oTotal         := STFGetTot()            // Recebe o Objeto totalizador
35 Local nBonif         := 0                      //Valor Bonificação
36 Local nDesconto      := 0                      //Valor Desconto
37
38 oPanBonif            := TPanel():New(00,00,"",oPanelMVC,,,,,oPanelMVC:nWidth/2,oPanelMVC:nHeight/2) //Painel do pagamento
39 nAltura              := (oPanBonif:nHeight / 2) * 0.20 //Altura
40 nCol                 := (oPanBonif:nWidth / 2) * 0.03 //Coordenada horizontal
41 nLargura             := (oPanBonif:nWidth / 2) - (2 * nCol) //Largura
42
43 nPosHozPan           := oPanelMVC:nWidth/76.2 //Posicao horizontal do painel
44 nPosAltPan           := oPanBonif:nHeight/4.807 //Posicao: Altura do Painel
45 nTamLagPan           := oPanBonif:nWidth/2.09836 //Tamanho: Largura do Painel
46 nTamAltPan           := oPanBonif:nHeight/3.4512 //Tamanho: Altura do Painel
47 bCreatePan           := {|TPanel():New(nPosAltPan,nPosHozPan,"",oPanBonif;
48                      ,,,,,nTamLagPan,nTamAltPan)} // Bloco de codigo para criacao do Painel adicional
49
50 If GetAPOInfo("STFTOTALUPDATE.PRW")[4] >= Ctod("19/12/2019")
51     nBonif           := oTotal:GetValue("L1_BONIF") //Valor Bonificação
52 EndIf

```

```

53 nDesconto      := oTotal:GetValue("L1_DESCONT")    //Valor Desconto
54
55 /* Label: Aviso */
56 oLblAviso := TSay():New(POSVERT_CAB, POSHOR_1, {|| STR0001}, oPanBonif,,,,,T,,,nLargura,11.5) //"Atenção!"
57 oLblAviso:SetCSS( POSCSS (GetClassName(oLblAviso), CSS_BREADCUMB ))
58
59 /* Label: Descrição do aviso */
60 oLblDescr := TSay():New(POSVERT_LABEL1 , POSHOR_1, {|| STR0002 + " " + STR0003}, oPanBonif,,,,,T,,,nLargura,40) //"Foi identificado que existe desconto e bonificação
    nesta venda.###"Escolha apenas uma das opções a ser aplicada:"
61 oLblDescr:SetCSS( POSCSS (GetClassName(oLblDescr), CSS_LABEL_FOCAL ))
62
63 /* Label e Get: Bonificação/Desconto */
64 oLblBonif := TSay():New(POSVERT_LABEL3-20, POSHOR_1, {|| STR0004 + "R$ " + Alltrim(Transform( nBonif, "@E 999999.99"))}, oPanBonif,, ,,,,T,,,nLargura,25) //"Bonificação: "
65 oLblBonif:SetCSS( POSCSS (GetClassName(oLblBonif), CSS_LABEL_ALERT ))
66
67 oLblDesc := TSay():New(POSVERT_LABEL3, POSHOR_1, {|| STR0005 + "R$ " + Alltrim(Transform( nDesconto, "@E 999999.99"))}, oPanBonif,, ,,,,T,,,nLargura,25) //"Desconto:"
68 oLblDesc:SetCSS( POSCSS (GetClassName(oLblDesc), CSS_LABEL_ALERT ))
69
70 /* Button: Bonificação */
71 oBtnBonif := TButton():New(POSVERT_BTNFOCAL,POSHOR_BTNFOCAL,STR0006+CRLF+"(Ctrl+B)",oPanBonif,{|| STIOkBonif()},LARGBTN,ALTURABTN,,,,T.) //"Bonificação"
72
73 /* Button: Desconto */
74 oBtnDesc := TButton():New(POSVERT_BTNFOCAL,POSHOR_BTNFOCAL-90,STR0007+CRLF+"(Ctrl+D)",oPanBonif,{|| STIOkDisc() },LARGBTN,ALTURABTN,,,,T.) //"Desconto"
75
76 oBtnBonif:SetCSS( POSCSS (GetClassName(oBtnBonif), CSS_BTN_FOCAL ))
77 oBtnDesc:SetCSS( POSCSS (GetClassName(oBtnDesc), CSS_BTN_FOCAL ))
78
79 SetKey(2, {|| STIOkBonif("Atalho")}) //Ativo CTRL+B
80 SetKey(4, {|| STIOkDisc("Atalho")}) //Ativo CTRL+D
81 LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), STR0002 )
82 LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), STR0003 )
83 LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), STR0004 + " Valor: " + Str(nBonif) )
84 LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), STR0005 + " Valor: " + Str(nDesconto) )
85
86 oBtnBonif:SetFocus()
87
88 Return oPanBonif
89
90
91 //-----
92 /*{Protheus.doc} STIOkBonif()
93 Escolhido Bonificação, deixando de escolher o Desconto Total.
94 Ocorre após Interface para decisão: a)Bonificação, b)Desconto em STIDscBonif()
95 e escolheu a opção A.
96 @type function
97
98
99 @param nil
100
101 @return nil
102 */
103 //-----

```

```

104 Function STIOkBonif()
105
106 LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), "Opção escolhida: Bonificação" )
107
108 SetKey(2, Nil)           //Desativo CTRL+B
109 SetKey(4, Nil)           //Desativo CTRL+D
110
111 STIClearDisc(.T.)         //Anulo desconto
112 STIExchPay()
113
114 Return nil
115
116
117 //-----
118 /*{Protheus.doc} STIOkDisc()
119 Escolhido Desconto, deixando de escolher a Bonificação.
120 Ocorre após Interface para decisão: a)Bonificação, b)Desconto em STIDscBonif()
121 e escolheu a opção B.
122 @type function
123
124
125 @param nil
126
127 @return nil
128 */
129 //-----
130 Function STIOkDisc()
131
132 LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), "Opção escolhida: Desconto" )
133
134 SetKey(2, Nil)           //Desativo CTRL+B
135 SetKey(4, Nil)           //Desativo CTRL+D
136
137 If GetAPOInfo("STFTOTALUPDATE.PRW")[4] >= Ctod("19/12/2019")
138     STFSetTot( "L1_BONIF", 0 ) //Anulo bonificação
139 EndIf
140 STIExchPay()
141
142 Return nil
143
144
145 //-----
146 /*{Protheus.doc} STIExchPay()
147 Chamo direto o STIPayment()
148 Caso ocorra bonificação e desconto ao mesmo tempo, eu chamo a Tela de NCC - o STINCCSelection() antes.
149 @type function
150
151
152 @param nil
153
154 @return nil
155 */

```

```
156 //-----
157 Function STIExchPay()
158 Local aNCCs      := STDGetNCCs("1")
159
160 If Len(aNCCs) > 0
161     //Carrego a tela de NCC
162     STIExchangePanel( { || STINCCSelection() } )
163 Else
164     //Carrego os paineis para os demais pagamentos
165     STIExchangePanel( { || STIPayment() } )
166 EndIf
167
168 Return nil
```

Componentes/Interface/STIDiscountTotal.PRW

429 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STIDISCOUNTTOTAL.CH"
4 #INCLUDE "POSCSS.CH"
5 #INCLUDE "STPOS.CH"
6
7 static lAtivMotor := ExistFunc("PshAtivMTP") .AND. PshAtivMTP()
8
9 //-----
10 /*{Protheus.doc} STIDiscountTotal
11 Abertura da Interface de Desconto na Venda
12
13 @param  nenhum
14 @return NIL
15 @obs
16 @sample
17 */
18 //-----
19 Function STIDiscountTotal()
20 Local lCRdesItTt := SuperGetMv("MV_LJRGDES",,.F.) .AND. SuperGetMv("MV_LJCRDPT",,"0") == "1" .AND.
    FindFunction("totvs.protheus.retail.desconto.RegraDescProdutoTotal.LjCallCalcRegDescProdTotal",.T.) // Verifica se o calculo do desconto por item esta sendo feito no
    final da venda
21 Local lDesligaRD := .F. // Desabilita a Regra de Desconto do Varejo
22
23 If FindFunction("totvs.protheus.retail.desconto.RegraDescProdutoTotal.HasDescManu",.T.)
24     lDesligaRD := totvs.protheus.retail.desconto.RegraDescProdutoTotal.HasDescManu("TOTVSPDV", STDGPBasket('SL1','L1_NUM'), STDGPBasket('SL1','L1_CLIENTE'),
        STDGPBasket('SL1','L1_LOJA'))
25     lCRdesItTt := !lDesligaRD
26 EndIf
27
28 If !lAtivMotor
29     /* Abre o Panel */
30     If !lCRdesItTt .AND. STBValScreenDisc()
31         If STBTaxRet( Nil ,"NF_DESCTOT") == 0
32             STIExchangePanel( { || STIPanDiscTotal() } )
33             STIChangeCssBtn('oBtnDesc')
34         Else
35             //STR0011#"0 desconto aplicado anteriormente sera cancelado, deseja continuar?"
36             //STR0012#"OK"
37             //STR0013#"Cancelar"
38             STIExchangePanel( { || STFYesNo(STR0011,STR0012,STR0013,{||STIClearDisc(.T.),STIChangeCssBtn('oBtnDesc'),STIExchangePanel( { || STIPanDiscTotal() }
                )},{||STIRegItemInterface()},.T.) } )
39         EndIf
40     ElseIf lCRdesItTt
41         STFMessage("STFMTPR0","POPUP", STR0017) // "Não é possível aplicar desconto manual, pois a funcionalidade de aplicação de Regra de Descontos na finalização de venda
        está ativa. Parâmetro MV_LJCRDPT."
42         STFShowMessage("STFMTPR0")
43     EndIf
44 Else
45     STFMessage("STFMTPR0","STOP", STR0016 )// "Não é Possível Aplicar Desconto Manual, o Motor de Promoções está ativado"
46     STFShowMessage("STFMTPR0")
47 EndIf

```

```

48
49 Return
50
51 //-----
52 /*{Protheus.doc} STIPanDiscTotal
53 Contexto desconto no Total da Venda
54
55 @param  nenhum
56 @return NIL
57 @obs
58 @sample
59 */
60 //-----
61 Function STIPanDiscTotal()
62 Local oPanGetDiscount := Nil
63 Local oGetValue       := Nil
64 Local nGetValue       := STBTaxRet( Nil , "NF_DESCTOT" )
65 Local oGetPerc        := Nil
66 Local oTotal          := STFGetTot()           // Totalizador
67 Local nGetPerc        := STBDiscConvert( STBTaxRet( Nil , "NF_DESCTOT" ) , 'V' , oTotal:GetValue( "L1_VALBRUT" ) ) [2]
68 Local oReasons        := Nil                  // ListBox de Motivo de Desconto
69 Local oButton         := Nil                  // Botao Confirmar Desconto na Venda
70 Local oPanelMVC        := STIGetPanel()        // Painei padrao para abertura de PDV
71 Local aReasons        := {}                   // Armazena Motivos de Desconto cadastrados
72 Local nTab1           := 0                    // Largura de TabulaÃ§Ã£o
73 Local nTabPerc        := 0.015               // Percentual de tabulaÃ§Ã£o
74 Local nGetnWidth      := 0                    // Largura dos Gets
75 Local nGetnHeight     := 0                    // Altura dos Gets
76 Local nOuWidth        := 0                    // Largura do Say "OU"
77 Local nOuPerc         := 0.08                 // Percentual da largura do Say "OU"
78 Local aOuPosition     := Array(2)             // PosiÃ§Ã£o exata largura e altura do Say "OU" em relaÃ§Ã£o ao espaÃ§o disponÃ-vel para ele
79 Local nOuPercTop      := 0.3                  // Percentual do comeÃ§o Top da palavra do Say "OU" em relaÃ§Ã£o Ã altura disponÃ-vel do espaÃ§o dele
80 Local nOuPercBotton   := 0.3                  // Percentual do comeÃ§o da palavra do Say "OU" em relaÃ§Ã£o Ã Largura disponÃ-vel do espaÃ§o dele
81 Local nRsnPercHeigth  := 0.15                 // Percentual da Altura do ListBox Motivo de Desconto
82 Local nRsnHeigth      := 0                    // Altura do ListBox Motivo de Desconto
83 Local nPerc2          := 0.15
84 Local nDescImp        := 0
85 Local oModelMaster    := STFGetTot()          // Model da SL1
86
87
88 /* oSay2 */
89 Local nPosLb2Hoz      := oPanelMVC:nWidth/2 * 0.039 //Posicao: Horizontal
90
91 /* oSay3 */
92 Local nPosLb3Hoz      := oPanelMVC:nWidth/2 * 0.406 //Posicao: Horizontal
93
94 /* oSay4 */
95 Local nPosLb4Alt      := (oPanelMVC:nHeight/2 * 0.02) * 8.85 //Posicao: Altura
96 Local nPosLb4Hoz      := oPanelMVC:nWidth/2 * 0.320 //Posicao: Horizontal
97
98 /* oSay6 */
99 Local nPosLb6Alt      := (oPanelMVC:nHeight/2 * 0.02) * 12 //Posicao: Altura

```

```

100
101 /* oGetValue */
102 Local nPosGetHoz      := oPanelMVC:nWidth/2 * 0.039
103 Local nTamGetHoz      := (oPanelMVC:nWidth/2)/2 * 0.49
104
105 /* oGetPerc */
106 Local nPosPerHoz      := oPanelMVC:nWidth/2 * 0.40
107
108 /* ListBox */
109 Local nLisHorPerc      := 0.039    // Percentual de Posição Horizontal do ListBox
110 Local nLisHeiPerc     := 0.19 // Percentual de Altura do ListBox
111
112 Local oSay5           := Nil
113 Local oSay6           := Nil
114
115 /* Panel Desconto na Venda */
116 oPanGetDiscount := TPanel():New(00,00,"",oPanelMVC,,,,,(oPanelMVC:nWidth/2),(oPanelMVC:nHeight/2))
117 oPanGetDiscount:SetCSS( POSCSS (GetClassName(oPanGetDiscount), CSS_PANEL_CONTEXT ))
118
119 nTab1      := oPanGetDiscount:nWidth/2 * nTabPerc
120 nOuWidth   := oPanGetDiscount:nWidth/2 * nOuPerc
121 nGetnWidth := ( ( oPanGetDiscount:nWidth/2 - (2*( nTab1 * 2)) - nOuWidth ) / 2)
122 nGetnHeight := 10
123
124 /* Calcula Posição exata do Say "OU" em relação ao espaço disponível para ele */
125 // Altura
126 aOuPosition[1] := ( (oPanGetDiscount:nHeight/2 * nPerc2) + ( nGetnHeight * nOuPercTop ) )
127 // Largura
128 aOuPosition[2] := ( (nTab1 * 2 + nGetnWidth) + nOuWidth * nOuPercBotton )
129
130 /* Objetos */
131 oSay1 := TSay():New(POSVERT_CAB,POSHOR_1,{||STR0001},oPanGetDiscount,,,,.T.,,) // "Vendas > Desconto na Venda"
132 oSay1:SetCSS( POSCSS (GetClassName(oSay1), CSS_BREADCUMB ))
133
134 oSay2 := TSay():New(POSVERT_LABEL1,nPosLb2Hoz,{||STR0002},oPanGetDiscount,,,,.T.,,) // "Valor(R$)"
135 oSay2:SetCSS( POSCSS (GetClassName(oSay2), CSS_LABEL_FOCAL ))
136
137 oSay3 := TSay():New(POSVERT_LABEL1,nPosLb3Hoz,{||STR0003},oPanGetDiscount,,,,.T.,,) // "Porcentagem(%)"
138 oSay3:SetCSS( POSCSS (GetClassName(oSay3), CSS_LABEL_FOCAL ))
139
140 oSay4 := TSay():New(nPosLb4Alt,nPosLb4Hoz,{||STR0004},oPanGetDiscount,,,,.T.,,) // "ou"
141 oSay4:SetCSS( POSCSS (GetClassName(oSay4), CSS_LABEL_FOCAL ))
142
143 oGetValue := TGet():New(POSVERT_GET1,nPosGetHoz,{|u| If(PCount()>0,nGetValue:=u,nGetValue)}, oPanGetDiscount,nTamGetHoz,, "@E
    999,999.99",,,,,.T.,,,,"nGetValue",,,,,.T.)
144 oGetValue:SetCSS( POSCSS (GetClassName(oGetValue), CSS_GET_FOCAL ))
145
146 oGetPerc := TGet():New(POSVERT_GET1,nPosPerHoz,{|u| If(PCount()>0,nGetPerc:=u,nGetPerc)}, oPanGetDiscount,nTamGetHoz,, "@E 999,999.99",,,,,.T.,,,,"nGetPerc",,,,,.T.)
147 oGetPerc:SetCSS( POSCSS (GetClassName(oGetPerc), CSS_GET_FOCAL ))
148
149 oGetValue:bLostFocus := {|| Iif(GetFocus() <> oGetPerc:HWND,oGetPerc:SetFocus(),Nil) }
150

```

```

151 /* Se Utiliza Motivo de desconto */
152 If STDRsnDesc()
153
154     aReasons      := STIReasons()
155
156     oSay5 := TSay():New(POSVERT_GET2, POSHOR_1,{||STR0005},oPanGetDiscount,,,,,.T.,,,) //"Motivo de Desconto"
157     oSay5:SetCSS( POSCSS (GetClassName(oSay5), CSS_LABEL_FOCAL ))
158
159     nRsnHeigth := oPanGetDiscount:nHeight/2 * nRsnPercHeigth
160
161     oReasons := TListBox():Create(oPanGetDiscount, oPanelMVC:nHeight/4.673, oPanelMVC:nWidth/2 * nLisHorPerc,;
162     Nil, aReasons, ( oPanGetDiscount:nWidth/2 - (2*nLisHorPerc*oPanGetDiscount:nWidth/2) ),oPanGetDiscount:nHeight/2 * nLisHeiPerc,,,,.T.)
163     oReasons:SetCSS( POSCSS (GetClassName(oReasons), CSS_LISTBOX ))
164
165     oSay5:lvisiblecontrol:=.F.
166     oReasons:lvisiblecontrol:=.F.
167
168 EndIf
169
170 If FindFunction("STBGetDscImp") .And. STBIsImpOrc()
171     STIDescImp( @nDescImp)
172     oSay6 := TSay():New(nPosLb6Alt,nPosGetHoz,{||STR0015+Transform(nDescImp,"@E 999,999.99")},oPanGetDiscount,,,,.T.,,,) //"Desconto da importação: R$ "
173     oSay6:SetCSS( POSCSS (GetClassName(oSay6), CSS_LABEL_FOCAL ))
174 EndIf
175
176 If oModelMaster:HasField("L1_ARMDESC") .And. oModelMaster:GetValue("L1_ARMDESC") > 0
177     oModelMaster:SetValue("L1_ARMDESC", 0) // Atualizamos o campo sempre que inicio a rotina onde tenha desconto no total.
178 EndIf
179
180 oButton := TButton():New( POSVERT_BTNFOCAL,POSHOR_BTNFOCAL,STR0006,oPanGetDiscount, ; //"Confirmar Desconto"
181     { || STISetReason(oReasons),IIF(STITotDiscVal(nGetValue,nGetPerc,oReasons), STIRegItemInterface(),)},LARGBTN,ALTURABTN,,,,.T.)
182
183 oButton:SetCSS( POSCSS (GetClassName(oButton), CSS_BTN_FOCAL ))
184
185 oBtnCa := TButton():New(POSVERT_BTNFOCAL,POSHOR_1,STR0009,oPanGetDiscount,{||STIRegItemInterface()}, ; //"Cancelar"
186     LARGBTN,ALTURABTN,,,,.T.)
187 oBtnCa:SetCSS( POSCSS (GetClassName(oBtnCa), CSS_BTN_NORMAL ))
188
189 oGetValue:bChange := { || IIf( nGetValue >= 0 , nGetPerc := STBDiscConvert( nGetValue , "V" ) [2] , ) } // Se valor maior que zero seta o foco no percentual e
190     atualiza objeto
191 oGetPerc:bChange := { || IIf( nGetPerc >= 0 , nGetValue := STBDiscConvert( nGetPerc , "P" ) [1] , ) } // Se percentual de desconto for maior que zero seta o foco
192     no valor e atuliza objeto
193
194 If oReasons <> Nil
195     oGetValue:bLostFocus := {|| STIFocusReason(oReasons, oSay5, nGetPerc, nGetValue) }
196     oGetPerc:bLostFocus := {|| STIFocusReason(oReasons, oSay5, nGetPerc, nGetValue) }
197 EndIf
198
199 oGetValue:SetFocus()
200
201 If GetFocus() <> oGetValue:HWND
202     oGetValue:SetFocus()

```

```

201 Endif
202
203 Return oPanGetDiscount
204
205 //-----
206 /*{Protheus.doc} STIFocusReason
207 Confirmação do desconto no total
208
209 @param oReasons      Objeto ListBox de Get de Motivo de Desconto
210 @return
211 @obs
212 @sample
213 */
214 //-----
215 Function STIFocusReason(oReasons, oSay5, nGetPerc, nGetValue)
216
217 Default oReasons      := Nil
218 Default oSay5          := Nil
219
220 If nGetPerc <= 0 .OR. nGetValue <= 0
221     oReasons:lvisiblecontrol:=.F.
222     oSay5:lvisiblecontrol:=.F.
223 else
224     oReasons:lvisiblecontrol:=.T.
225     oSay5:lvisiblecontrol:=.T.
226     oReasons:GoTop()
227     oReasons:SetFocus()
228 EndIf
229
230 Return
231
232 //-----
233 /*{Protheus.doc} STITotDiscVal
234 Confirmação do desconto no total
235
236
237 @param nGetValue, numerico
238 @param nGetPerc , numerico
239 @param oReasons , objeto
240 @return lRet, logico
241 @obs
242 @sample
243 */
244 //-----
245 Function STITotDiscVal( nGetValue , nGetPerc , oReasons)
246 Local lRet          := .T.                // Retorna se efetuou o desconto
247 Local lFinServ       := SuperGetMV("MV_LJCSF",.F.) // Define se habilita o controle de servicos financeiros
248 Local nDiscOrcV      := 0                // Valor de desconto no orçamento (Caso exista)
249 Local nDiscOrcP      := 0                // Percental de desconto no orçamento (Caso exista)
250 Local aDescOrc       := {0,0}           // Array com Informações sobre desconto no orçamento, sendo ele; valor total e percentual total.
251 Local oModelMaster   := STFGetTot()      // Model da SL1
252

```

```

253 Default nGetValue      := 0                // Valor do Desconto
254 Default nGetPerc       := 0                // Percentual do Desconto
255 Default oReasons       := Nil              // Objeto do motivo de desconto
256
257 If oModelMaster:HasField("LI_ARMDESC") .And. nGetValue >= 0
258     oModelMaster:SetValue("LI_ARMDESC", nGetValue) // Atualizamos o campo sempre que confirmado o desconto no total.
259 EndIf
260
261 If FindFunction("STBGetDscImp")
262     //-----
263     //|Valida desconto proveniente da importacao |
264     //|mando atualizar pois se eu tiver um limite |
265     //|no desconto ( permissao ) o desconto    |
266     //|da importacao deve ser considerado      |
267     //-----
268     STIDescImp( NIL , @nDiscOrcV, @nDiscOrcP)
269     aDescOrc    := {nDiscOrcV,nDiscOrcP}
270 EndIf
271
272 If nGetValue <= 0 .AND. nGetPerc <= 0
273
274     If nGetValue < 0 .OR. nGetPerc < 0
275
276         STFMessage(ProcName(),"STOP",STR0010) //"Desconto Invalido."
277         STFShowMessage(ProcName())
278         lRet := .F.
279     Else
280         STIClearDisc()
281     EndIf
282
283 Else
284     //Validacao Servicos Financeiros
285     If lFinServ
286         lRet := STWDescServ()
287     EndIf
288
289     If lRet
290         /* Se Utiliza Motivo de desconto */
291         If STDRsnDesc()
292             If Empty( STIGetReason() )
293                 lRet := .F.
294                 STFMessage("STITotalDiscount","STOP",STR0008) //"obrigatorio selecionar o Motivo de Desconto"
295                 STFShowMessage("STITotalDiscount")
296             EndIf
297         EndIf
298     EndIf
299 EndIf
300
301 If lRet
302
303     Do Case
304

```

```

305         Case nGetPerc > 0 .AND. nGetValue <= 0
306             lRet := STWTotDisc( nGetPerc , "P" , "US",,aDesc0rc )
307         Case nGetValue > 0
308             lRet := STWTotDisc( nGetValue , "V" , "US",,aDesc0rc )
309         Otherwise
310             lRet := STWTotDisc( 0 , "V" , "US" )
311
312     EndCase
313
314     /* Atualiza Interface */
315     If lRet
316
317         STIGridCupRefresh()
318
319     EndIf
320
321 EndIf
322
323 EndIf
324
325 Return lRet
326
327 //-----
328 /*{Protheus.doc} STIClearDisc
329 Zera o desconto aplicado
330
331 @param lLimpaDesc, Lógico, Define se ira limpar os descontos da tela
332 @param nDescParc, Numérico, Valor do desconto que será retirado
333 @param lMantemTFC , Lógico, Define se ira deixar o valor de desconto da fidelização L1_DESCFID
334
335 */
336 //-----
337 Function STIClearDisc(lLimpaDesc,nDescParc,lMantemTFC) // -- nDescParc = Valor de desconto parcial
338 Local lDesTot := STBCDPGDes()
339 Local nDescTot := IIf( lDesTot , STDGPBasket("SL1","L1_DESCTOT") , 0 )//Valida se concedeu desconto no total da venda
340 Local oRaas := Nil
341 Local oFidelityC:= Nil
342 Local nValDFidel:= 0
343
344 Default lLimpaDesc := .F.
345 Default nDescParc := 0
346 Default lMantemTFC := .F.
347
348 LjGrvLog( Nil, " Inicio - Limpeza de desconto - STIClearDisc")
349 LjGrvLog( Nil, " Zerando as referencias de desconto")
350
351 If ExistFunc("LjxRaasInt") .And. LjxRaasInt()
352     oRaas := STBGetRaas()
353     If Valtype(oRaas) == "0" .And. oRaas:ServiceIsActive("TFC")
354         oFidelityC := oRaas:GetFidelityCore()
355         If oFidelityC:ChoseToUse()
356             nValDFidel := oFidelityC:GetBonus()

```

```

357         EndIf
358     EndIf
359 EndIf
360
361 If lDesTot
362     STDSPBasket("SL1","L1_DESCTOT", Iif( nValDFidel > 0, nValDFidel , 0 ) )
363 EndIf
364
365 STBTaxAlt( "NF_DESCTOT" , Iif( nValDFidel > 0, nValDFidel , 0 ) )
366
367 // -- Limpo a Variavel cReasonSelected que esta presente no fonte STIReasonForDiscont
368 If ExistFunc("STICLeReason")
369     STICLeReason()
370 EndIf
371
372 LjGrvLog( Nil, " Limpeza dos Totais ")
373 STFRefTot( 0 )
374
375 If lLimpaDesc //Optei por limpar todos os descontos pela tela de desconto
376
377     If !lMantemTFC
378         STDSPBasket("SL1","L1_DESCFID",0)
379     EndIf
380
381     If nDescParc > 0 .AND. nDescParc < nDescTot
382         nDescTot := nDescTot - nDescParc
383     Else
384         nDescTot := 0
385     EndIf
386 EndIf
387
388 /* So permito recalcular o desconto caso tenha sido
389 dado no total da venda, para que o mesmo seja mantido*/
390 If lDesTot .And. nDescTot > 0
391     LjGrvLog( Nil, " Valor de desconto no total preenchido, portanto o desconto sera somado ", nDescTot)
392     LjGrvLog( Nil, " Para zerar o desconto no total da venda: faca o lancamento de desconto no valor de zero ")
393     STBTaxAlt( "NF_DESCTOT" , nDescTot)
394     LjGrvLog( Nil, " Limpeza dos Totais ")
395     STFRefTot(nDescTot)
396 EndIf
397
398 If ExistFunc("STBSetDiscTotPDV") .AND. nDescTot == 0
399     STBSetDiscTotPDV( .F. )
400 EndIf
401
402 LjGrvLog( Nil, " Fim - Limpeza de desconto - STIClearDisc")
403
404 Return .T.
405
406 //-----
407 /*/{Protheus.doc} STIDescImp
408 Valida valores de descontos provenientes da importacao de orcamento

```

```
409
410 @param nDescImp, numerico
411 @param nGetValue, numerico
412 @param nGetPerc , numerico
413 @return lRet, logico
414 /*/
415 //-----
416 Function STIDescImp( nDescImp, nGetValue, nGetPerc)
417 Local nPerImp := 0
418 Local oTotal := STFGetTot() //Totalizador
419
420 Default nDescImp := 0
421 Default nGetValue:= 0
422 Default nGetPerc := 0
423
424 nDescImp := STBGetDscImp()
425 nPerImp := STBDiscConvert( nDescImp , 'V' , oTotal:GetValue( "L1_VALBRUT" ) )[2]
426 nGetValue+= nDescImp
427 nGetPerc += nPerImp
428
429 Return .T.
```

Componentes/Interface/STIFinService.prw

547 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE 'Protheus.ch'
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "FWEVENTVIEWCONSTS.CH"
4 #INCLUDE "FWADAPTEREAI.CH"
5 #INCLUDE "FWMVCDEF.CH"
6 #INCLUDE "POSCSS.CH"
7 #INCLUDE "STIFINSERVICE.CH"
8 #INCLUDE "STPOS.CH"
9
10 Static oListProduct := Nil
11 Static oListService := Nil
12 Static oListResSF    := Nil
13 Static lListWhen     := .T.
14 Static lDisableBtn   := .T.
15
16 //-----
17 /*{Protheus.doc} STIFinService
18 Apresenta tela de escolha dos serviços financeiros.
19
20 @param1  aFinItens      Array com os serviços financeiros da venda.
21 @param2  aCriticCli     Array que recebera as criticas cadastrais do cliente
22 @return  Nil
23 @obs
24 @sample
25 */
26 //-----
27 Function STIFinService(aFinItens, aCriticCli)
28
29 Local oPanelMVC := STIGetPanel()
30 Local oPanService := TPanel():New(00,00,"",oPanelMVC,,,,,oPanelMVC:nWidth/2,oPanelMVC:nHeight/2) //Painel do MVC //Painel de botoes
31 Local nAltura := (oPanService:nHeight / 2) * 0.20
32 Local nCol := (oPanService:nWidth / 2) * 0.03
33 Local nLargura := (oPanService:nWidth / 2) - (2 * nCol)
34 Local oLblFinServ := Nil
35 Local oLblProduct := Nil
36 Local oLblService := Nil
37 Local oSayRes := Nil
38 Local oLblTotSF := Nil
39 Local oLblTotGer := Nil
40 Local oButton := Nil
41 Local oBtnLimpar := Nil
42
43 Local oTotal := STFGetTot()
44 Local nLblTotSF := 0
45 Local nLblTotGer := oTotal:GetValue("L1_VLRTOT")
46
47 Local nSpace := 40
48 Local cPicDefault := "@E 9,999,999.99"
49 Local aProducts := STIFinProd(aFinItens, nSpace)
50 Local aServices := {{},{}}
51 Local aResSF := {{},{}}

```

//Indica se o listbox de selecao de pagamento é editavel
//Desabilita/Habilita os botoes de Limpar e Finalizar pagamentos

//Objeto de venda > pagamento
//Label de pagamento
//Label do Serviço
//Label Generico do Resumo Financeiro
//Label do resultado total de Serv. Finan.
//Label do resultado total de Serv. Finan com o Total da Venda
//Botão Prosseguir
//Botão Limpar

//Recebe o objeto do model para recuperar os valores
//Total dos Serv. Financeiros selecionados
//Total dos Serviços com o Total da Venda

//Tamanho de espaco entre uma informacao e outra
//Picture padrao para valores
//List de Produtos
//List de Serviços {{Informações Tela},{Informações

//Resumo de Pagamento {{Informações Tela},{Informações

```

        Processamento}}
52
53 /* Variaveis do objeto -> oLblProduct */
54 Local nPosAltTpLbl := oPanelMVC:nHeight * 0.030 //Posicao: Altura do ListBox
55
56 /* Variaveis do objeto -> oListProduct */
57 Local nPosAltListBox := oPanelMVC:nHeight * 0.046 //Posicao: Altura do ListBox
58 Local nTamAltListBox := oPanelMVC:nHeight * 0.08 //Tamanho: Altura do ListBox
59
60 /* Variaveis do objeto -> oGrpPaym */
61 Local nPosAltGroup := oPanelMVC:nHeight/4.807 //Posicao: Altura do GroupBox
62 Local nTamAltGroup := oPanelMVC:nHeight/3.9588 //Tamanho: Altura do GroupBox
63 Local nTamLarGroup := oPanelMVC:nWidth * 0.485 //Tamanho: Lagura do GroupBox
64
65 /* Variaveis do objeto -> oLblGroup1 */
66 Local nPosAltLbl1 := oPanelMVC:nHeight * 0.178 //Posicao: Altura do Label1 GroupBox
67 Local nPosHozLbl1 := oPanelMVC:nWidth * 0.140 //Posicao: Horizontal do Label1 GroupBox
68
69 /* Cria um bloco de codigo para ser utilizado na tela do cheque qdo for chamado pela condicao de pagamento */
70 Local nPosHozPan := oPanelMVC:nWidth/76.2 //Posicao horizontal do painel
71 Local nPosAltPan := oPanService:nHeight/4.807 //Posicao: Altura do Painel
72 Local nTamLagPan := oPanService:nWidth/2.09836 //Tamanho: Largura do Painel
73 Local nTamAltPan := oPanService:nHeight/3.4512 //Tamanho: Altura do Painel
74 Local bCreatePan := {||TPanel():New(nPosAltPan,nPosHozPan,"",oPanService;
75     ,,,,,nTamLagPan,nTamAltPan)} // Bloco de codigo para criacao do Painel adicional
76
77 Default aFinItens := {}
78 Default aCriticCli := {}
79
80 ParamType 0 Var aFinItens As Array Default {}
81 ParamType 1 Var aCriticCli As Array Default {}
82
83 /* Limpa as mensagens das etapas anteriores ao pagamento */
84 STFCleanInterfaceMessage()
85
86 /* Label: Serviços Financeiros */
87 oLblFinServ := TSay():New(POSVERT_CAB, POSHOR_1, {||STR0001}, oPanService,,,,,T,,,nLargura,11.5) //"Serviços Financeiros"
88 oLblFinServ:SetCSS( POSCSS (GetClassName(oLblFinServ), CSS_BREADCUMB ))
89
90 /* Label: Selecione o Produto */
91 oLblProduct := TSay():New(POSVERT_LABEL1*0.8100, POSHOR_1, {||STR0002}, oPanService,,,,,T,,,nLargura,8) //"Selecione o Produto"
92 oLblProduct:SetCSS( POSCSS (GetClassName(oLblProduct), CSS_LABEL_FOCAL ))
93 /* ListBox dos Produtos */
94 oListProduct := TListBox():Create(oPanService, POSVERT_GET1*0.8100, POSHOR_1, Nil, aProducts, nLargura, nTamAltListBox,,,,,T,,, ;
95     {||lListWhen := .T., lDisableBtn := .T., STIAtuServ(aFinItens,nSpace,oListProduct,aProducts,oListService,@aServices)} ;
96     ,,,,{||lListWhen == .T.})
97 oListProduct:SetCSS( POSCSS (GetClassName(oListProduct), CSS_LISTBOX ))
98 oListProduct:Reset()
99 oListProduct:SetArray(aProducts)
100
101 /* Label: Escolha do Serviço */
102 oLblService := TSay():New(POSVERT_LABEL1*2.6000, POSHOR_1, {||STR0003}, oPanService,,,,,T,,,nLargura,8) //"Selecione o Serviço Vinculado"

```

```

103 oLblService:SetCSS( POSCSS (GetClassName(oLblService), CSS_LABEL_FOCAL ))
104 /* ListBox dos Serviços */
105 oListService := TListBox():Create(oPanService, POSVERT_GET1*2.0000, POSHOR_1, Nil, aServices[01], nLargura, nTamAltListBox,,,,.T., ;
106     {||lListWhen := .T., lDisableBtn := .T., STIAtuRes(aFinItens,nSpace,oListService,aServices,oListResSF,@aResSFi,@nLblTotSFi,@nLblTotGer)} ;
107     ,,,{||lListWhen == .T.})
108 oListService:SetCSS( POSCSS (GetClassName(oListService), CSS_LISTBOX ))
109 oListService:Reset()
110 oListService:SetArray(aServices[01])
111
112 /* Label: Resumo do Serviço financeiro */
113 oLblResSF := TSay():New(oPanelMVC:nHeight/3.7000, POSHOR_1, {||STR0004}, oPanService,,,,.T.,,nLargura,8) //"Resumo do Serviço Financeiro"
114 oLblResSF:SetCSS( POSCSS (GetClassName(oLblResSF), CSS_LABEL_FOCAL ))
115
116 /* Label: Serviço */
117 oSayRes := TSay():New(oPanelMVC:nHeight/3.4000, POSHOR_1 * 01.0, {||STR0005}, oPanService,,,,.T.,,nLargura,8) //"Produto"
118 oSayRes:SetCSS( POSCSS (GetClassName(oSayRes), CSS_LABEL_FOCAL ))
119 /* Label: Serviço */
120 oSayRes := TSay():New(oPanelMVC:nHeight/3.4000, POSHOR_1 * 09.0, {||STR0006}, oPanService,,,,.T.,,nLargura,8) //"Serviço"
121 oSayRes:SetCSS( POSCSS (GetClassName(oSayRes), CSS_LABEL_FOCAL ))
122 /* Label: Valor */
123 oSayRes := TSay():New(oPanelMVC:nHeight/3.4000, POSHOR_1 * 18.0, {||STR0007}, oPanService,,,,.T.,,nLargura,8) //"Valor"
124 oSayRes:SetCSS( POSCSS (GetClassName(oSayRes), CSS_LABEL_FOCAL ))
125
126 /* ListBox dos resumos dos serviços */
127 oListResSF := TListBox():Create(oPanService, oPanelMVC:nHeight/3.1400, POSHOR_1, Nil, aResSFi[01], nLargura*0.81, nTamAltListBox,,,,.T., ;
128     {||.T.} ;
129     ,,,{||.T.})
130 oListResSF:SetCSS( POSCSS (GetClassName(oListResSF),CSS_LISTBOX ))
131 oListResSF:Reset()
132 oListResSF:SetArray(aResSFi[01])
133
134 /* Label: Rodape dos Resumos */
135 oSayRes := TSay():New(oPanelMVC:nHeight/3.4000, POSHOR_BTNFOCAL*0.630,{||STR0008},oPanService,,,,.T.,,) //"VLR SERVIÇOS:"
136 oSayRes:SetCSS( POSCSS (GetClassName(oSayRes), CSS_LABEL_NORMAL ))
137 oLblTotSFi := TSay():New(oPanelMVC:nHeight/3.1700, POSHOR_BTNFOCAL*0.500,{||nLblTotSFi}, oPanService,cPicDefault,,,,.T.,,)
138 oLblTotSFi:SetCSS( POSCSS (GetClassName(oLblTotSFi), CSS_LABEL_FOCAL))
139 oSayRes := TSay():New(oPanelMVC:nHeight/2.9000, POSHOR_BTNFOCAL*0.630,{||STR0009},oPanService,,,,.T.,,) //"TOTAL GERAL:"
140 oSayRes:SetCSS( POSCSS (GetClassName(oSayRes), CSS_LABEL_NORMAL))
141 oLblTotGer := TSay():New(oPanelMVC:nHeight/2.7000, POSHOR_BTNFOCAL*0.500,{||nLblTotGer}, oPanService,cPicDefault,,,,.T.,,)
142 oLblTotGer:SetCSS( POSCSS (GetClassName(oLblTotGer), CSS_LABEL_FOCAL))
143
144 /* Button para prosseguir */
145 oButton := TButton():New(POSVERT_BTNFOCAL,POSHOR_BTNFOCAL,STR0010+CRLF+"(CTRL+F)",oPanService,{ || STIConfSF(aFinItens, aCriticCli, aResSFi) }, ; //"Finalizar Venda"
146     LARGBTN,ALTURABTN,,,,.T.,,{||lDisableBtn})
147 oButton:SetCSS( POSCSS (GetClassName(oButton), CSS_BTN_FOCAL ))
148
149 /* Limpar Resumo */
150 oBtnLimpar := TButton():New(POSVERT_BTNFOCAL,POSHOR_1,STR0011+CRLF+"(CTRL+L)",oPanService,{ || STIZeraSF(oListResSF,@aResSFi,@nLblTotSFi,@nLblTotGer,oTotal) },
151     LARGBTN,ALTURABTN,,,,.T.,,{||lDisableBtn}) //"Limpar Escolhas"
151 oBtnLimpar:SetCSS( POSCSS (GetClassName(oBtnLimpar), CSS_BTN_ATIVO ))
152
153 SetKey(06, {|| STIConfSF(aFinItens, aCriticCli, aResSFi) })

```

```

154 SetKey(12, {| STIZeraSF(oListResSF,@ResSFi,@nLbTotSFi,@nLbTotGer,oTotal) })
155
156 /* Posiciona no primeiro registro e seta o focus */
157 oListProduct:GoTop()
158 oListProduct:SetFocus()
159
160 Return oPanService
161
162 //-----
163 /*{Protheus.doc} STIZeraSF
164 Zera todos os Serviços Financeiros escolhidos
165
166 @param      oListResSF      - Objeto ListBox do Resumo
167 @param      aResSFi         - Array com conteudo de Resumos
168 @param      nLbTotSFi       - Total dos Serv. Financeiros selecionados
169 @param      nLbTotGer       - Total dos Serviços com o Total da Venda
170 @param      oTotal          - Objeto do total
171 @return
172 @obs
173 @sample
174 */
175 //-----
176 Function STIZeraSF(oListResSF,aResSFi,nLbTotSFi,nLbTotGer,oTotal)
177
178 // Limpas as variáveis do Label
179 nLbTotSFi := 0
180 nLbTotGer := oTotal:GetValue("L1_VLRTOT")
181 aResSFi := {{},{}}
182 oListResSF:Reset()
183 oListResSF:SetArray(aResSFi[01])
184 oListResSF:SetFocus()
185
186 Return .T.
187
188 //-----
189 /*{Protheus.doc} STIConfSF
190 Confirmacao dos pagamentos
191
192 @param1      aFinItens      Array com os serviços financeiros da venda
193 @param2      aCriticCli     Array que armazenara criticas cadastrais do cliente
194 @param3      aResSFi        Array que contem itens financeiros vinculados
195 @return
196 @obs
197 @sample
198 */
199 //-----
200 Function STIConfSF( aFinItens, aCriticCli, aResSFi )
201
202 Local lRet      := .T.           //Variavel de retorno
203 Local cValPre   := STBGetVales()
204 Local lSTWNxtTelaIt := ExistFunc("STWNxtTelaIt")
205

```

```

206 Default aFinItens      := {}
207 Default aCriticCli     := {}
208 Default aResSFi        := {{},{}}
209
210 ParamType 0 Var         aFinItens      As Array      Default      {}
211 ParamType 1 Var         aCriticCli     As Array      Default      {}
212 ParamType 2 Var         aResSFi        As Array      Default      {{},{}}
213
214 /* Efetua validacao do cliente se foi selecionado item vinculado ou se possuir servico avulso */
215 If Len(aResSFi[1]) > 0 .OR. aScan(aFinItens,{|xVar| xVar[06] == 0 .AND. xVar[08] == .F.}) > 0
216     STBVldClient(aFinItens, @aCriticCli)
217 EndIf
218
219 If Len(aCriticCli) > 0
220     /* Servico Financeiro - Mensagens da validacao do cliente */
221     STIExchangePanel({|| STIFinClient(aCriticCli,aResSFi)})
222 Else
223     /* Se nao houverem criticas ao cliente chama tela Forma de Pagamento */
224     //Se existir Itens servicos Vinculados selecionados, registro sem imprimir no cupom.
225     If Len(aResSFi[1]) > 0
226         STIRegServFin(aResSFi)
227     EndIf
228
229     IIf(lSTWNxtTelaIt,STWNxtTelaIt(),STICallPayment())
230 EndIf
231
232 Return lRet
233
234 //-----
235 /*/{Protheus.doc} STIFinProd
236 Retorna os Produto do Array Financeiro
237
238 @param1   aFinItens      Array Financeiro
239 @param2   nSpace         Espaços entre strings
240 @return   aRet           Array dos Produtos
241 @obs
242 @sample
243 */
244 //-----
245
246 Static Function STIFinProd(aFinItens, nSpace)
247
248 Local aRet      := {}                // Array de retorno
249 Local nI        := 0                // Variavel do Laço
250 Local nItemAnt  := 0                // Item Anterior
251 Local aItensAux := aFinItens        // Array Auxiliar
252
253 Default aFinItens := {}
254 Default nSpace    := 1
255
256 ParamType 0 Var   aFinItens      As Array      Default      {}
257 ParamType 1 Var   nSpace         As Numeric    Default      1

```

```

258
259 //Ordernar do menor para o maior Item
260 ASORT(aItensAux,,{ | x,y | x[06] < y[06] } )
261
262 For nI := 1 To Len(aItensAux)
263
264   If (aItensAux[nI][06] <> 0) .AND. !(aItensAux[nI][08]) //Se item não for avulso e não for deletado
265
266     If nItemAnt <> aItensAux[nI][06]
267       aAdd( aRet, STRZERO(aItensAux[nI][06],2) +SPACE(nSpace-LEN(STRZERO(aItensAux[nI][06],2))) + ;
268           AllTrim(aItensAux[nI][04]) +SPACE(nSpace-LEN(AllTrim(aItensAux[nI][04]))) + ;
269           Upper(AllTrim(aItensAux[nI][05])) )
270       nItemAnt := aItensAux[nI][06]
271     EndIf
272
273   EndIf
274 Next nI
275
276 Return aRet
277
278 //-----
279 /*{Protheus.doc} STIAtuServ
280 Retorna e atualiza os serviços do Array Financeiro
281
282 @param aFinItens - Array com conteudo de Produtos e Serviços da Venda
283 @param nSpace - Espaços entre strings
284 @param oListProduct - Objeto ListBox de Produtos
285 @param aProducts - Array com conteudo de Produtos
286 @param oListService - Objeto ListBox de Serviços
287 @param aServices - Array com conteudo de Serviços
288 @return Nil
289 @obs
290 @sample
291 */
292 //-----
293 Static Function STIAtuServ(aFinItens,nSpace,oListProduct,aProducts,oListService,aServices)
294
295 Local cItemProd := AllTrim( SubStr(oListProduct:aItems[oListProduct:nAt],1,(1*nSpace)) ) // Item Produto atual
296 Local cCodProd := AllTrim( SubStr(oListProduct:aItems[oListProduct:nAt],(1*nSpace)+1,(1*nSpace)) ) // Codigo Produto atual
297 Local cDescProd := AllTrim( SubStr(oListProduct:aItems[oListProduct:nAt],(2*nSpace)+1,(1*nSpace)) ) // Descrição Produto atual
298 Local nItemProd := VAL(cItemProd) // Item Produto atual
299 Local nFor := 0 // Variavel de Laço
300 Local aItensAux := aFinItens // Array Auxiliar
301 Local oTotal := STFGetTot() // Totalizador
302 Local lDesconto := oTotal:GetValue("L1_DESCONT") > 0
303
304 //Valida se existe desconto na venda
305 If lDesconto
306   STFMessage("STIFinService","STOP", "Não é possível inserir Servicos Financeiros pois venda possui desconto")
307   STFShowMessage("STIFinService")
308 EndIf
309

```

```

310 /* Valida quantidade do produto vendido */
311 If STBVldQtdProd(nItemProd) .And. !lDesconto
312     // Limpas as variáveis do Label
313     aServices := {{},{}}
314
315     //Ordernar do menor para o maior Item
316     ASORT(aItensAux,,,{ | x,y | x[06] < y[06] } )
317
318     For nFor := 1 To Len(aItensAux)
319
320         If aItensAux[nFor][06] == nItemProd //Somente os itens do produto selecionado
321             If (aItensAux[nFor][06] <> 0) .AND. !(aItensAux[nFor][08]) //Se item não for avulso e não for deletado
322
323                 aAdd( aServices[01],AllTrim(SUBSTR(aItensAux[nFor][01] ,1,nSPACE-7)) +SPACE(nSpace-LEN(AllTrim(SUBSTR(aItensAux[nFor][01] ,1,nSPACE-7)))) + ;
324                     AllTrim(SUBSTR(aItensAux[nFor][02] ,1,nSPACE-7)) +SPACE(nSpace-LEN(AllTrim(SUBSTR(aItensAux[nFor][02] ,1,nSPACE-7)))) + ;
325                     AllTrim(Transform(aItensAux[nFor][03], PesqPict("MBL","MBL_VALOR") ) )
326                 aAdd( aServices[02],aItensAux[nFor])
327
328             EndIf
329         EndIf
330
331     Next nFor
332
333     oListService:Reset()
334     oListService:SetArray(aServices[01])
335     oListService:SetFocus()
336 EndIf
337
338 Return Nil
339
340 //-----
341 /*{Protheus.doc} STIAtuRes
342 Retorna e atualiza os Resumos do Array Financeiro
343
344 @param      aFinItens      - Array com conteudo de Produtos e Serviços da Venda
345 @param      nSpace         - Espaços entre strings
346 @param      oListService   - Objeto ListBox de Serviços
347 @param      aServices      - Array com conteudo de Serviços
348 @param      oListResSF     - Objeto ListBox do Resumo
349 @param      aResSF         - Array com conteudo de Resumos
350 @param      nLblTotSF      - Total dos Serv. Financeiros selecionados
351 @param      nLblTotGer     - Total dos Serviços com o Total da Venda
352 @return     Nil
353 @obs
354 @sample
355 */
356 //-----
357 Static Function STIAtuRes(aFinItens,nSpace,oListService,aServices,oListResSF,aResSF,nLblTotSF,nLblTotGer)
358
359 Local nPos      := oListService:nAt                                     // Posição atual
360 Local aItensAux := aServices                                           // Array Auxiliar
361 Local cDescProd := aItensAux[02][nPos][05]                             // Descrição Produto Vendido

```

```

362 Local cCodServ := AllTrim( SubStr(oListService:aItems[oListService:nAt],1 , (1*nSpace)) ) // Codigo Serviço atual
363 Local cDescServ := AllTrim( SubStr(oListService:aItems[oListService:nAt], (1*nSpace)+1 , (1*nSpace)) ) // Descrição Serviço atual
364 Local cValServ := AllTrim( SubStr(oListService:aItems[oListService:nAt], (2*nSpace)+1 , (1*nSpace)) ) // Valor Serviço atual
365
366 aAdd( aResSFi[01], SUBSTR(cDescProd +SPACE(nSpace-LEN(AllTrim(cDescProd))),1,20) + ;
367 SPACE(3) + ;
368 SUBSTR(cDescServ +SPACE(nSpace-LEN(AllTrim(cDescServ))),1,20) + ;
369 SPACE(10) + ;
370 cValServ )
371 aAdd( aResSFi[02],aItensAux[02][nPos])
372
373 nLbLTotSFi += Val(StrTran(StrTran(cValServ,"",""),".",""))/100
374 nLbLTotGer += Val(StrTran(StrTran(cValServ,"",""),".",""))/100
375
376 oListResSF:Reset()
377 oListResSF:SetArray(aResSFi[01])
378 oListResSF:SetFocus()
379
380 Return Nil
381
382 //-----
383 /*/{Protheus.doc} STIFinClient
384 Apresenta tela das validacoes do cliente quando Servico Financeiro.
385
386 @param aFinItens Array com os serviços financeiros da venda
387 @param aResSFi Array com conteudo de Resumos
388 @return Nil
389 @obs
390 @sample
391 */
392 //-----
393 Function STIFinClient(aCriticCli, aResSFi )
394
395 Local oPanelMVC := STIGetPanel() //Painel do MVC //Painel de botoes
396 Local oPanClient := TPanel():New(00,00,"",oPanelMVC,,,,,oPanelMVC:nWidth/2,oPanelMVC:nHeight/2) //Painel do pagamento
397 Local nAltura := (oPanClient:nHeight / 2) * 0.20 //Altura
398 Local nCol := (oPanClient:nWidth / 2) * 0.03 //Coordenada horizontal
399 Local nLargura := (oPanClient:nWidth / 2) - (2 * nCol) //Largura
400 Local oLbLTotServ := Nil //Objeto de venda > pagamento
401 Local oLbLTotClient := Nil //Label de pagamento
402 Local nSpace := 25 //Tamanho de espaco entre uma informacao e outra
403
404 /* Variaveis do objeto -> oListClient */
405 Local nTamAltListBox := oPanelMVC:nHeight * 0.30 //Tamanho: Altura do ListBox
406
407 Default aCriticCli := {}
408 Default aResSFi := {{},{}}
409
410 ParamType 0 Var aCriticCli As Array Default {}
411 ParamType 1 Var aResSFi As Array Default {{},{}}
412
413 /* Limpa as mensagens das etapas anteriores ao pagamento */

```

```

414 STFCleanInterfaceMessage()
415
416 nAjustCols := nLargura
417 oPanBkpSF := oPanClient
418
419 /* Label: Validacao Cliente */
420 oLbCliServ := TSay():New(POSVERT_CAB, POSHOR_1, {||STR0012}, oPanClient,,,,,T,,,nLargura,11.5) //"Validação Cliente"
421 oLbCliServ:SetCSS( POSCSS (GetClassName(oLbCliServ), CSS_BREADCUMB ))
422
423 /* ListBox de Inconsistencias do Cliente */
424 oListClient := TListBox():Create(oPanClient, POSVERT_GET1*0.8100, POSHOR_1, Nil, aCriticCli, nLargura, nTamAltListBox,,,,,T,,,,,{|| .T.})
425 oListClient:SetCSS( POSCSS (GetClassName(oListClient), CSS_LISTBOX ))
426 oListClient:Reset()
427 oListClient:SetArray(aCriticCli)
428
429 /* Button para prosseguir */
430 oButton := TButton():New(POSVERT_BTNFOCAL,POSHOR_BTNFOCAL,STR0010+CRLF+"(CTRL+F)",oPanClient,{ || STIConfClient(aResSFi) }, ; //"Finalizar Venda"
431 LARGBTN,ALTURABTN,,,,T,,,,{||!DisableBtn})
432 oButton:SetCSS( POSCSS (GetClassName(oButton), CSS_BTN_FOCAL ))
433
434 SetKey(06, {|| STIConfClient(aResSFi) })
435 SetKey(12, Nil)
436
437 Return oPanClient
438
439 //-----
440 /*{Protheus.doc} STIConfClient
441 Confirmacao dos pagamentos
442
443 @param1      aResSFi      Array com conteudo de Resumos
444 @return
445 @obs
446 @sample
447 */
448 //-----
449 Function STIConfClient( aResSFi )
450 Local lSTWNxtTelaIt := ExistFunc("STWNxtTelaIt")
451
452 Default aResSFi := {{},{}}
453
454 ParamType 0 Var aResSFi          As Array          Default {{},{}}
455
456 If STBPermUser()
457     /* Se existir Itens serviços Vinculados selecionados, registro sem imprimir no cupom */
458     If Len(aResSFi[1]) > 0
459         STIRegServFin(aResSFi)
460     EndIf
461
462     IIf(lSTWNxtTelaIt,STWNxtTelaIt(),STICallPayment())
463 EndIf
464
465 Return Nil

```

```

466
467 //-----
468 /*{Protheus.doc} STIRegServFin
469 Registra itens de serviços financeiros
470
471 @param1      aResSFi      Array com conteudo de Resumos
472 @return
473 @obs
474 @sample
475 */
476 //-----
477 Function STIRegServFin( aResSFi )
478
479 //Local lContinue := STBPermUser()           //Flag continua venda
480 Local nFor := 0                               //Controle de Laço
481 Local nItemLine := 0                          //Item de Venda
482 Local cCliCode := STDGPBasket("SL1","L1_CLIENTE")
483 Local cCliLoja := STDGPBasket("SL1","L1_LOJA")
484 Local nMoeda := 1
485 Local cCliType := STDGPBasket("SL1","L1_TIPOCLI")
486 Local cCodItem := ""
487 Local lRegistred := .F.
488 Local cTesPad := ""
489
490 Default aResSFi := {{},{}}
491
492 ParamType 0 Var aResSFi      As Array      Default {{},{}}
493
494 /* Se Usuario tem permissao chama Condicao de Pagamento */
495 //If lContinue
496 //Se existir Itens serviços Vinculados selecionados, registro sem imprimir no cupom.
497 For nFor := 1 To Len(aResSFi[2])
498     nItemLine := STDPBLength("SL2") + 1
499     cCodItem := aResSFi[02][nFor][01]
500     /*
501         Registra Item
502     */
503
504     //Busca TES do Servico Financeiro
505     MG8->(dbSetOrder(2))
506
507     If MG8->(dbSeek(xFilial("MG8") + cCodItem))
508         cTesPad := MG8->MG8_TESSB1
509     EndIf
510
511     lRegistred := STWItemReg( ;
512         nItemLine      , ;      // Item
513         cCodItem        , ;      //Codigo Prod
514         cCliCode         , ;      //Codigo Cli
515         cCliLoja         , ;      //Loja Cli
516         nMoeda           , ;      //Moeda
517         Nil              , ;      //Valor desconto

```

```

518      Nil          , ;      // Tipo desconto ( Percentual ou Valor )
519      Nil          , ;      // Item adicional?
520      cTesPad      , ;      // TES
521      cCliType     , ;      // Tipo do cliente (A1_TIPO)
522      .F.          , ;      // Registra item no cupom fiscal?
523      aResSFi[02][nFor][03], ;// Preço
524      Nil          , ;      // Tipo do Item
525      Nil          , ;      // Imprime CNPJ no cupom Fiscal
526      Nil          , ;      // Tela do POS está sendo apresentada
527      Nil          , ;      // Total dos segundos entre itens
528      .T.          )        //Indica finalização dos serviços na Venda
529
530      If lRegistered
531          /* Atualiza campos controle Servicos Financeiros vinculados*/
532          STDUpdServFin(StrZero(aResSFi[02][nFor][06], TamSx3("L2_ITEM")[1]), aResSFi[02][nFor][04])
533
534          /* Seta objeto Servico Financeiro como selecionado */
535          STWItemFin( 5          , ; //Tipo do Processo (1=Set - 2=Get - 3=Clear - 4=Del - 5=Sel)
536                  cCodItem      , ; //Codigo Servico Financeiro
537                  aResSFi[02][nFor][03] , ; //Valor do Servico Financeiro
538                  aResSFi[02][nFor][04] , ; //Codigo Produto Vendido
539                  aResSFi[02][nFor][06] , ; //Item Produto Vendido
540                  ""             ) //Tipo Item - Usado para importacao de Orcamento
541      EndIf
542      Next nFor
543
544      STIGridCupRefresh() // Sincroniza a Cesta com a interface
545 //EndIf
546
547 Return Nil

```

Componentes/Interface/STIGiftList.prw

1408 linhas após remoção de metadados de autoria/data.

[illegible]

[illegible]

[illegible]

```

157 Local nLargura      := (oMainPanel:nWidth / 4) - (2 * nCol) //Largura
158 Local nPosAltGroup := oPanelMVC:nHeight /4                //Posicao: Altura do GroupBox
159 Local nTamAltGroup := oPanelMVC:nHeight/4                 //Tamanho: Altura do GroupBox
160 Local nTamLarGroup := (oPanelMVC:nWidth) * 0.485          //Tamanho: Lagura do GroupBox
161 Local nTop          := POSVERT_GET1+65                    //Valor do Top
162 Local nLeft         := 000                                //Valor do Left
163 Local PosVerPay      := POSHOR_1+10                        //Posicao Vertical
164 Local nPosVerTit     := nPosAltGroup/20.500               //Posicao inicial do titulo do painel da Esquerda para Direita
165 Local nPosVerGrp     := nPosAltGroup/12.500               //Posicao inicial do Grupo do painel da Esquerda para Direita
166
167
168 /* Variaveis do objeto -> Grupo Filtro*/
169 Local oGrpFilt       := Nil                                //Objeto groupbox
170 /* Variaveis do objeto -> Filtro*/
171 Local oLblFilt       := Nil                                //Objeto de Label Regra
172 Local oGetFilt      := Nil                                //Objeto de Get Regra
173 Local nNumCols      := 200
174 Local cGetCrit      := SPACE( nNumCols)                  //Correspondente a regra
175 Local aFilters      := {}                                  //Correspondente a lista de filtros
176 Local aLstOpc       := {STR0001,STR0002,STR0003,STR0004,STR0005} //Lista de opcoes de pesquisa"1=Número da lista"#2=Atores"#3=Data do evento"#4=Local do
evento"#5=Nome do organizador"}
177                                     // numero de
178 /* Variaveis do objeto -> Criterio*/
179 Local oGrpCrit      := Nil                                //Objeto groupbox
180 Local oLblCrit      := Nil                                //Objeto de Label Regra
181 Local oGetCrit      := Nil                                //Objeto de Get Regra
182 Local cGetFilt      := ""                                  //Correspondente a regra
183 Local aFiltCmp      := {"ME1_CODIGO","MEE_NOME","ME1_DTEVEN","ME1_LOCAL","A1_NOME"}//Campos da lista de opcao de pesquisa
184
185 Local aLstFilC      := {} //lista dos filtros
186
187 /*label e Combo: Criterios Utilizados */
188 Local LblCrtUt      := NIL
189 Local oLstCrtUt     := NIL
190 Local aCrtUt        := {}
191
192 /* botões */
193 Local oBtnAdic       := NIL //Botão Adicionar
194 Local oBtnRemov      := NIL //Botão Remover
195 Local oBtnSearch     := NIL //Botão Pesquisar
196 /* Variaveis do objeto -> Grupo Filtro - Final */
197
198
199 /* Variaveis do objeto -> Grupo de Listas - inicio */
200 /* Objeto TGroup do Resultado da Lista */
201 Local oGrpList := NIL
202 //Objetos direita
203 Local nAltura2       := (oPanelMVC2:nHeight / 4) * 0.20    //Altura
204 Local nCol2          := (oPanelMVC2:nWidth / 4) * 0.03      //Coordenada horizontal
205 Local nLargura2      := (oPanelMVC2:nWidth / 4) - (2 * nCol) //Largura
206 Local nPosAltGr2     := oPanelMVC2:nHeight /4              //Posicao: Altura do GroupBox
207 Local nTamAltGr2     := oPanelMVC2:nHeight/4               //Tamanho: Altura do GroupBox

```

```

208 Local nTamLarGr2      := nTamLarGroup + 5           //Tamanho: Lagura do GroupBox
209 Local nTop2           := POSVERT_GET1+65           //Valor do Top
210 Local nLeft2          := 000                      //Valor do Left
211 Local nPosVerTit2     := nPosAltGr2/20.500          //Posicao inicial do titulo do painel da Esquerda para Direita
212 Local nPosVerGrp2     := nPosAltGr2/12.500//Posicao inicial do Grupo do painel da Esquerda para Direita
213
214 /* Label e Grid: Lista de Eventos */
215 Local oLblList        := NIL //Label
216 Local aListas         := {} //lista de Eventos
217 Local aDados01        := {} //Dados de Retorno da Lista
218 Local aHeader01       := {} //Header da Lista
219 Local aAuxCombo       := RetSx3Box( Posicione("SX3", 2, PadR("ME1_STATUS",10), "X3CBox()" ),,, 1 ) //Array contendo os status da consulta
220 Local aCoorGrdL       := {} //Coordenadas do grid da lista
221 Local aLstCmp01       := {} //Lista de campos do ME1
222 Local aLstCmpNL       := {"MEH_CODLIS","MEE_CODLIS"} //Campos nao Listados
223 // Local oObj         := NIL //
224 Local oSecPanel        := NIL //Segundo Panel
225 Local oGrdList        := NIL // Grid da Lista de Eventos
226 Local oBtnSel         := NIL //Botão Selecionar
227 Local oBtnAtor        := NIL //Botão Atores
228 Local oBtnEntr        := NIL //Botão Entregas
229 Local PosVerPay2      := (PosVerPay + nTamLarGroup) //Posicao Veritical
230 Local lPosicionado   := .T. //Posicionado no Grid de Lista
231
232 /* Variaveis do objeto -> Grupo de Listas - Final */
233 /* Variaveis do objeto -> Itens de Listas - Inicial */
234 Local oPnlPrd         := NIL //Panel de Itens
235 Local oPnlGrdP        := NIL //Panel do Grid de Itens
236 /* Objeto TGroup da dos Produtos */
237 Local oGrpPrd         := NIL //Grupo de Produtos
238 Local oGrdPrd         := NIL //Grid de Produtos
239 /* Label e Combo: Produtos */
240 Local oLblPrd         := NIL //Label de Produtos
241 Local oBtnInc         := NIL //Botão Incluir
242 Local oBtnConf        := NIL //Botão confirmar
243 Local oGetProd        := NIL //Get de Produto
244 Local cGetProd        := ME2->(space(TamSX3("ME2_PRODUT")[1])) //Variavel do Get de Produto
245 Local oLblProd        := NIL //Label do Produto
246 Local oLblQtde        := NIL //Label da Quantidade
247 Local oLblMet         := NIL //Label Metodo de Entrega
248 Local oGetQtde        := NIL //Get Quantidade
249 Local oCbxCmd         := nil //Combo Metodo de Entrega
250 Local oLblCodBar      := Nil //Label Codigo de Barras
251 Local oGetCodBar      := Nil //Get Codigo de Barras
252 Local aCbxCmd         := {} //Array do Combo dos Métodos de Entrega
253 Local nGetQtde        := 0 //Variavel Quantidade
254 Local cGetCodBar      := Space(TamSX3("B1_CODBAR")[1]) //Variavel Código de Barras
255 Local cGetMet         := ME1->(TamSX3("ME1_TIPO")[1]) //Variavel Metodo de Entrega
256 Local cPictQtde       := ME2->(GetSX3Cache("ME2_QTDSOL","X3_PICTURE")) //Picture Quantidade
257 Local aHeader02       := {} //Header do Grid de Produtos
258 Local aDados02        := {} //Dados do Grid de Produtos
259 Local aLstC02         := {} //Campos de Consulta do Grid

```

```

260 Local aItRet      := {} //Itens de Lista Retornados
261 Local aMsgRet     := {} //Mensagens Retornadas
262 Local cTexto      := "" //Texto 1 do Label de Entrada:
263 Local cTexto2     := "" //Texto 2 do Label Entrada
264 Local cTexto3     := "" //Texto 2 do Label Entrada
265
266 Local bListSel := {} //bloco de Selecao da Lista de Eventos
267
268 /* Label e Grid: Lista de Eventos - final*/
269
270
271 /* Label e Grid: Mensagens/Atores/Entregas */
272 Local oGrpGen      := NIL //Grupo de Atores/Entregas
273 Local oLblGen      := NIL //Label de Atores/Entregas
274 Local oPnlGen      := NIL //Panel de Atores/Entregas
275 Local oGrdGen      := NIL //Grid de Atores/Entregas
276 Local aHeader03    := {} //Objeto Lista de Mensagens/header
277 Local aLstC03      := {} //Lista de Campos de Mensagens
278 Local aDados03     := {} //Dados de Mensagens
279 Local aDados04     := {} //Dados de Atores/Entregas
280 Local cTextoGen    := "" //Texto Genérico
281 /* Button: Selecionar*/
282 Local oBtnCnc      := NIL //Botão Cancelar
283 Local cTextoBtn    := "" //Texto do Botão Cancelar
284 Local oBtnCnf      := NIL //Botão confirmar
285 Local lGrdMsgAtv := .T. //Grid de Mensagens Ativas
286 Local aAreaSX3     := SX3->(GetArea())
287 /* Label e Grid: Mensagens/Atores/Entregas - Final */
288
289 //Busca as estruturas do Grid
290
291 STIGLEstr(aLstCmpNL, "ME1", @aHeader01, @aLstCmp01,;
292 "ME1_STATUS")
293
294
295 STIGLEstr(aLstCmpNL, "ME2", @aHeader02, @aLstC02,;
296 "")
297
298
299 STIGLEstr(aLstCmpNL, "MED", @aHeader03, @aLstC03,;
300 "")
301
302 RestArea(aAreaSX3)
303
304 /* Label: Lista de Eventos */
305 oLblLista := TSay():New(POSVERT_CAB, POSHOR_1, {||STR0010}, oMainPanel,,,,,T,,,nLargura,13.5) //"Lista de Presentes"
306 oLblLista:SetCSS( POSCSS (GetClassName(oLblLista), CSS_BREADCUMB ))
307
308 /* Objeto TGroup da Pesquisa da Lista */
309 oGrpFilt := TGroup():New(nPosAltGroup/13+5,POSHOR_1,nTamAltGroup, nTamLarGroup,',',oMainPanel,,5,.T.)
310 /* Label e Combo: Regra */
311 oLblFilt:= TSay():New( POSVERT_GET1*0.50, PosVerPay, {|| STR0011 }, oMainPanel,,,,,T,,,nLargura,8) //"Pesquisa"

```

```

312 oLblFilt:SetCSS( POSCSS (GetClassName(oLblFilt), CSS_BREADCUMB ))
313 oGetFilt := TComboBox():New(POSVERT_GET1*0.70, PosVerPay, {|u| If(PCount()>0,cGetFilt:=u,cGetFilt)}, aLstOpc, (LARG_LIST_CONSULT*0.6) , ALTURAGET, ;
314 oGrpFilt, Nil , {|cGetCrit := Space(nNumCols),oGetCrit:Picture := STIGPesPic(1,cGetFilt,aFiltCmp),oGetCrit:Refresh(),oGetFilt:SetFocus() }/*Change*/,;
315 /*Valid*/,...T...,/*When*/,...,"cGetFilt")
316 oGetFilt:SetCSS( POSCSS (GetClassName(oGetFilt), CSS_GET_FOCAL ))
317
318
319 /*label e Combo: Criterio*/
320 oLblCrit:= TSay():New( POSVERT_GET1*0.90, PosVerPay, {| STR0012 }, oMainPanel,...,T...,nLargura,8) //"Critério"
321 oLblCrit:SetCSS( POSCSS (GetClassName(oLblCrit), CSS_BREADCUMB ))
322 oGetCrit := TGet():New(POSVERT_GET1*1.1,PosVerPay,{|u| If(PCount()>0,cGetCrit:=u,cGetCrit)},oMainPanel,;
323 LARG_LIST_CONSULT*0.6,ALTURAGET,STIGPesPic(1,cGetFilt,aFiltCmp), ;
324 /*valid*/,.../*font*/,...T...,/*when*/,.../*change*/,...F...,F..., "cGetCrit")
325 oGetCrit:SetCSS( POSCSS (GetClassName(oGetCrit), CSS_GET_FOCAL ))
326
327
328 /*label e lista: Criterios Utilizados */
329 oLblCrtUt:= TSay():New( POSVERT_GET1*1.3+1, PosVerPay, {| STR0013 }, oMainPanel,, ...,T...,8) //"Critérios Utilizados"
330 /* ListBox dos criterios */
331 oLblCrtUt:SetCSS( POSCSS (GetClassName(oLblCrtUt), CSS_BREADCUMB ))
332 oLstCrtUt := TListBox():Create(oMainPanel, POSVERT_GET1*1.5, PosVerPay, Nil, ;
333 aCrtUt,LARG_LIST_CONSULT*0.6, ALT_LIST_CONSULT/2,...,T..., {| } ,...,{| })
334
335 oLstCrtUt:SetCSS( POSCSS (GetClassName(oLstCrtUt), CSS_LISTBOX ))
336 oLstCrtUt:Reset()
337 oLstCrtUt:SetArray(aCrtUt)
338
339
340 /* Button: Adicionar*/
341 oBtnAdic := TButton():New(POSVERT_LABEL1*0.75,nTamLarGroup - ( LARGBTN + 15 ),STR0014,oMainPanel, ; //"Adicionar"
342 {| IIf(Val(cGetFilt) = 0 .OR. !STIGLBAAdd( aLstOpc[Val(cGetFilt)],aFiltCmp[Val(cGetFilt)],cGetCrit,@aLstFilC,;
343 @aLstCrtUt, nNumCols, @aCrtUt),.T.,cGetCrit := Space(nNumCols)),;
344 oGetCrit:SetFocus(),oLstCrtUt:Select(Len(oLstCrtUt:aItems))}, LARGBTN,ALTURABTN,,;
345 ...,T...,;
346 ,,{| Val(cGetFilt) > 0 }) )
347 oBtnAdic:SetCSS( POSCSS (GetClassName(oBtnAdic), CSS_BTN_ATIVO))
348
349 /* Button: Remover*/
350 oBtnRemov := TButton():New(POSVERT_LABEL1 * 1.50 , nTamLarGroup - ( LARGBTN + 15 ),STR0015 ,oMainPanel, ; //"Remover"
351 {| STIGLBDel(@oLstCrtUt,oLstCrtUt:nAt,@aLstFilC, @aCrtUt) }, LARGBTN,ALTURABTN,,;
352 ...,T...,;
353 ,,{| Len(aCrtUt) > 0 })
354 oBtnRemov:SetCSS( POSCSS (GetClassName(oBtnRemov), CSS_BTN_NORMAL))
355
356 /* Button: Pesquisar*/
357 oBtnSearch := TButton():New( POSVERT_LABEL1 * 2.25 ,nTamLarGroup - ( LARGBTN + 15 ),STR0016,oMainPanel, ; //"Pesquisar"
358 {| STWGLSearch( xFilial("ME1_FILIAL") ,aHeader01,aLstFilC,@Dados01,;
359 oSecPanel , @oGrdList , aMainCfg), oGetCodBar:SetFocus() }, LARGBTN,ALTURABTN,,;
360 ...,T...,;
361 ,,{| Len(aLstFilC) > 0 })
362 oBtnSearch:SetCSS( POSCSS (GetClassName(oBtnSearch), CSS_BTN_NORMAL))
363

```

```

364
365 /* Objeto TGroup do Resultado da Lista */
366 oGrpList := TGroup():New(POSVERT_GET1*1.55 + (ALT_LIST_CONSULT/2)+5,POSHOR_1 /*POSHOR_1*/,oPanelMVC:nHeight*0.5, nTamLarGroup ,'',oMainPanel,,5,.T.)
367
368 /* Label e Combo: "Lista de Eventos Localizadas */
369 oLblList:= TSay():New( POSVERT_GET1*1.55 + (ALT_LIST_CONSULT/2)+15, PosVerPay, {|| STR0017 }, oMainPanel,,,,,T,,,nLargura,8) //"Lista de Eventos Localizadas
370 oLblList:SetCSS( POSCSS (GetClassName(oLblList), CSS_BREADCUMB ))
371
372 aCoorGrdL := {POSVERT_GET1*1.55 + (ALT_LIST_CONSULT/2)+30,; //Linha
373               PosVerPay,; //coluna
374               LARG_LIST_CONSULT*0.7,; //tamanho
375               ALT_LIST_CONSULT*0.90} //altura
376
377
378 oSecPanel := TPanel():New(aCoorGrdL[01],aCoorGrdL[02], "",oMainPanel,,,,,aCoorGrdL[03],aCoorGrdL[04]) //Painel do Grid de Listas
379
380
381 STIGLGrid( oSecPanel, @aDados01, aHeader01, aAuxCombo, ;
382           @oGrdList, "aDados01[oBrwConten:nAt]", @aLstCmp01, @aItRet, ;
383           @aMsgRet, aMainCfg, aLstC02, aLstC03, ;
384           aHeader02, @aDados02, @oPnlGrdP, @oGrpPrd, ;
385           @oCbxMet, @aCbxMet , @oGrpGen, @aDados03, ;
386           @lGrdMsgAtv,@cTextoGen, @oLblGen,aHeader03, ;
387           @oPnlGen, @cTextoBtn, @oBtnCnc, @oBtnCnf, ;
388           @lPosicionado, @oBtnConf)
389
390
391 /* Button: Selcionar*/
392 bListSel := {|| STWGLItSearch( STDGLRtCol(2,"ME1_CODIGO",1,oGrdList:nAt,aHeader01,aDados01, .f., .f.), aDados01, aLstCmp01, @aItRet,;
393             @aMsgRet, aMainCfg, aLstC02, aLstC03, ;
394             aHeader02, @aDados02, @oPnlGrdP, @oGrpPrd,;
395             @oCbxMet, @aCbxMet , @oGrpGen, @aDados03,;
396             oGrdList:nAt, @lGrdMsgAtv, @cTextoGen, @oLblGen,;
397             aHeader03, aAuxCombo, @oPnlGen, @cTextoBtn, ;
398             @oBtnCnc, @oBtnCnf, @lPosicionado, @oBtnConf) }
399 oBtnSel := TButton():New(aCoorGrdL[01] ,nTamLarGroup - ( LARGBTN + 15) ,STR0018 ,oMainPanel, ; //"Selecionar"
400             bListSel, LARGBTN,ALTURABTN,,;
401             ,,.T.,,;
402             ,, { || CONOUT(CVALTOCHAR(LEN(ADADOS01))), Len(aDados01) > 0 } ) //"Adicionar"
403 oBtnSel:SetCSS( POSCSS (GetClassName(oBtnSel), CSS_BTN_ATIVO))
404
405 /* Button: Atores*/
406 oBtnAtor := TButton():New(aCoorGrdL[01] + ALTURABTN + 5, nTamLarGroup - ( LARGBTN + 15) ,STR0019,oMainPanel, ; //"Atores"
407             {|| STWGLAtEnt( @lGrdMsgAtv, 2, STDGLRtCol(2,"ME1_CODIGO",1,oGrdList:nAt,aHeader01,aDados01, .f.,
408             .f.),STDGLRtCol(2,"ME1_TIPO",1,oGrdList:nAt,aHeader01,aDados01, .f., .f.), ;
409             oPnlGen, @aDados04, @oGrpGen, @cTextoGen,;
410             @oLblGen, @cTextoBtn, @oBtnCnc, @oBtnCnf) }, LARGBTN,ALTURABTN,,;
411             ,,.T.,,;
412             ,, {|| Len(aDados01) > 0 } )
413 oBtnAtor:SetCSS( POSCSS (GetClassName(oBtnAtor), CSS_BTN_NORMAL))
414
415 /* Button: Entrega*/

```

```

415 oBtnEntr := TButton():New(aCoorGrdL[01] + (ALTURABTN + 5)*2 ,nTamLarGroup - ( LARGBTN + 15) ,STR0020,oMainPanel, ; //"Entrega"
416 {|| STWGLAtEnt( @lGrdMsgAtv, 1, STDGLRtCol(2,"ME1_CODIGO",1,oGrdList:nAt,aHeader01,aDados01, .f., .f.),STDGLRtCol(2,"ME1_TIPO",1,oGrdList:nAt,aHeader01,aDados01, .f.,
.t.), ;
417 oPnlGen, @aDados04, @oGrpGen, @cTextoGen,;
418 @oLblGen, @cTextoBtn, @oBtnCnc, @oBtnCnf }}, LARGBTN,ALTURABTN,,;
419 ...T.,,;
420 ,, {|| Len(aDados01) > 0 } )
421 oBtnEntr:SetCSS( POSCSS (GetClassName(oBtnEntr), CSS_BTN_NORMAL))
422
423
424 //Lado Direito
425 /* Panel de Group - Produtos */
426 oPnlRight = TPanel():New(00, 00,"",oPanelMVC2,,,,,oPanelMVC2:nWidth/2,(oPanelMVC2:nHeight)/2)
427
428 /* Objeto TGroup da dos Produtos */
429 oGrpProd := TGroup():New(nPosAltGroup/13+5,POSHOR_1,(oPnlRight:nHeight/2)*0.55 + 13, (oPnlRight:nWidth/2)*0.98,',oPnlRight,,5,.T.)
430
431
432 /* Label e Combo: Produtos */
433
434 oLblPrd:= TSay():New( POSVERT_GET1*0.40, PosVerPay, {|| STR0021 }, oPnlRight,,,,,.T.,,nLargura,8) //"Produtos"
435 oLblPrd:SetCSS( POSCSS (GetClassName(oLblPrd), CSS_BREADCUMB ))
436
437
438 /*Panel do Grid de Produtos*/
439 oPnlGrdP := TPanel():New(POSVERT_GET1*0.55,PosVerPay-10,"",oPnlRight,,,,,(oPnlRight:nWidth/2)*0.9,((oPnlRight:nHeight/2)*0.55)*0.6 ) //Painel de Produtos
440
441
442 /*Controles Produtos/Quantidade/Método de Entrega
443 //Produtos*/
444 /*Código de Barras*/
445 oLblCodBar:= TSay():New( POSVERT_GET1*2.5 - 13, PosVerPay, {||STR0030}, oPnlRight,,,,.T.,,nLargura,8) //"Código de Barras"
446 oLblCodBar:SetCSS( POSCSS (GetClassName(oLblCodBar), CSS_BREADCUMB ))
447 oGetCodBar := TGet():New(POSVERT_GET1*2.5,(PosVerPay)-1,{||u| If(PCount())>0,cGetCodBar:=u,cGetCodBar}},oPnlRight,;
448 LARG_GET_VALOR * 2,ALTURAGET,"@!",{ || .T. } ,;
449 ,,,,,.T.,,{|| },,,,.f.,,"cGetCodBar")
450 oGetCodBar:SetCSS( POSCSS (GetClassName(oGetCodBar), CSS_GET_FOCAL ))
451 oGetCodBar:lVisible := aMainCfg[POS_MAME]
452 oGetCodBar:Refresh()
453
454 /*Perda de foco do campo Codigo de Barras*/
455 oGetCodBar:bLostFocus := {|| STDSearCodBar(@cGetCodBar, @cGetProd, oGrpPrd, oGetCodBar)}
456
457 If !aMainCfg[POS_MLAQTD]
458 cTexto := STR0022
459 Else
460 cTexto := ""
461 EndIf
462 oLblProd:= TSay():New( POSVERT_GET1*2.5 + 18, PosVerPay, {||cTexto }, oPnlRight,,,,.T.,,nLargura,8)
463 oLblProd:SetCSS( POSCSS (GetClassName(oLblProd), CSS_BREADCUMB ))
464
465 @ POSVERT_GET1*2.75 + 15,PosVerPay MSGET oGetProd VAR cGetProd F3 "SB1" SIZE LARG_GET_VALOR, ALTURAGET OF oPnlRight PIXEL HASBUTTON

```

```

466 oGetProd:SetCSS( POSCSS (GetClassName(oGetProd), CSS_GET_FOCAL ))
467 oGetProd:lVisible := .F.
468 oGetProd:Refresh()
469
470 /*Quantidade*/
471 If aMainCfg[POS_MLAQTD]
472     cTexto3 := STR0023 //"Quantidade"
473 EndIf
474 oLblProd:Refresh()
475 oLblQtde := TSay():New( POSVERT_GET1*2.5 + 18, PosVerPay + LARG_GET_VALOR + 7, {||cTexto3 }, oPnlRight,,,,,T,,,nLargura,8)
476 oLblQtde:SetCSS( POSCSS (GetClassName(oLblQtde), CSS_BREADCUMB ))
477
478 oGetQtde := TGet():New((POSVERT_GET1*2.75) + 15,(PosVerPay)-1 + LARG_GET_VALOR + 7,{|u| If(PCount()>0,nGetQtde:=u,nGetQtde)},oPnlRight,;
479     LARG_GET_VALOR,ALTURAGET,cPictQtde,{ || .t. } ,;
480     ,,,,,T,,,{|| } ,,,,,f,,, "nGetQtde")
481 oGetQtde:SetCSS( POSCSS (GetClassName(oGetQtde), CSS_GET_FOCAL ))
482 oGetQtde:lVisible := .T.
483 oGetQtde:Refresh()
484
485 /*Método de Entrega*/
486 If aMainCfg[POS_MAME]
487     cTexto2 := STR0020 // "Entrega"
488 EndIf
489 oLblMet:= TSay():New( (POSVERT_GET1*2.6) + 17, (PosVerPay) + LARG_GET_VALOR*2 + 17, {||cTexto2 }, oPnlRight,,,,,T,,,nLargura,8)
490 oLblMet:SetCSS( POSCSS (GetClassName(oLblMet), CSS_BREADCUMB ))
491
492
493 oCbxMet := TComboBox():New((POSVERT_GET1*2.8) + 17, (PosVerPay-1) + LARG_GET_VALOR*2 + 17 , {|u| If(PCount()>0,cGetMet:=u,cGetMet)}, aCbxMet,;
494     LARG_GET_VALOR , ALTURAGET+4, oPnlRight, Nil , ;
495     {|| }/*Change*/,/*Valid*/,,,,T,,,,/*When*/,,,,, "cGetMet")
496 oCbxMet:SetCSS( POSCSS (GetClassName(oCbxMet), CSS_GET_FOCAL ))
497 oCbxMet:lVisible := aMainCfg[POS_MAME]
498 oCbxMet:Refresh()
499
500 /*Ativa os controles Quantidade e Método de Entrega*/
501 If (!aMainCfg[POS_MLAQTD] .AND. !aMainCfg[POS_MAME])
502     STIGLATvCtrl(1, @cTexto,@oLblProd, @cTexto2, @cTexto3,;
503         @oLblMet,@oGetProd, @oGetQtde,@oCbxMet )
504 EndIf
505
506
507 /*Monta o Grid de Produtos*/
508 STIGLGdIt( oPnlGrdP, @aDados02, aHeader02,@oGrpPrd, ;
509     "aDados02[oBrwConten2:nAt]" , @cTexto,@oLblProd, @cTexto2, @cTexto3,;
510     @oLblMet,@oGetProd, @oGetQtde, @oCbxMet, ;
511     @nGetQtde, @cGetMet, @lGrdMsgAtv, @aCbxMet )
512
513
514 /* Button: Confirmar Quantidade/Inclusao de Produto */
515 oBtnConf := TButton():New(POSVERT_GET1*2.1 + 15 , (PosVerPay) + LARG_GET_VALOR*2 + 10 , STR0024,oPnlRight, ; //"Confirmar"
516     {|| STBGLConf(oGetQtde,nGetQtde,oCbxMet, cGetMet,;
517         oGetProd, cGetProd , @aItRet,@aDados02, ;

```

[illegible]

```

569 Criação do grid das Listas de Presentes
570 @param o - Panel Principal
571 @param aDados01 - Dados da Lista
572 @param aHeader01 - Header da Lista
573 @param aAuxCombo - Combo de Status da Lista
574 @param oBrwConten - Browse
575 @param cCampo - Campo do Grid
576 @param aLstCmp01 - Lista de Campos da Tabela
577 @param aItRet - Item de Retorno a Lista de Presentes
578 @param aMsgRet - Mensagens da Lista
579 @param aMainCfg - Configurações da Lista
580 @param aLstC02 - Lista de Campos - Itens da Lista
581 @param aLstC03 - Lista de Campos - Mensagens
582 @param aHeader02 - Header dos Itens da Lista
583 @param aDados02 - Dados da Lista
584 @param oPnlGrdP - Panel do Grid de itens da Lista
585 @param oGrpPrd - Panel do Grid de Produtos
586 @param oCbxCbxMet - Combo de Métodos de Entrega
587 @param aCbxCbxMet - Array de Métodos de Entrega
588 @param oGrpGen - Grupo de Atores/Entregas/mensagens
589 @param aDados03 - Dados das Mensagens
590 @param lGrdMsgAtv - Grid de Mensagem Ativa
591 @param cTextoGen - Texto Genérico
592 @param oLblGen - Label Genérico
593 @param aHeader03 - Header de Mesagens
594 @param oPnlGen - Panel Genérico
595 @param cTextoBtn - Texto do Botão cancelart
596 @param oBtnCnc - Botão cancelar
597 @param oBtnCnf - Botão Finalizar
598 @param lPosicionado - Posicionado no Grid de Lista
599 @param oBtnConf - Botão Confirmar
600 @return oBrwConten - Browse
601 @obs
602 @sample
603 /**
604 //%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
605 Static Function STIGLGrid( o,aDados01,aHeader01, aAuxCombo, ;
606                          oBrwConten, cCampo, aLstCmp01,aItRet, ;
607                          aMsgRet, aMainCfg, aLstC02, aLstC03, ;
608                          aHeader02, aDados02, oPnlGrdP, oGrpPrd, ;
609                          oCbxCbxMet, aCbxCbxMet , oGrpGen, aDados03, ;
610                          lGrdMsgAtv,cTextoGen, oLblGen,aHeader03, ;
611                          oPnlGen, cTextoBtn, oBtnCnc, oBtnCnf;;
612                          lPosicionado, oBtnConf)
613
614
615 Local oGetCrtUt := NIL //Conteudo do css
616 Local oColumn := Nil //Objeto do Grid
617 Local cPicTotal := PesqPict("SL1","L1_VLRTOT") //Picture do L1_VLRTOT
618 Local nX := 0 //Variavel de Laco
619 Local aLegenda := {} //Legenda do Grid
620 Local aCores := {} //Cores do Grid

```

```

621 Local nI                := 0                                //Contador 2
622 Local nPosStatus        := aScan(aHeader01, { |h| h[1] == "ME1_STATUS"}) //Campo de Status
623 Local cTmp              := ""                                //Temporario - Campo
624 Local cTmp2             := ""                                //Temporario - Título
625 Local bListSel := {|| STWGLItSearch( STDGLRtCol(2,"ME1_CODIGO",1,oBrwConten:nAt,aHeader01,aDados01, .f., .f.), aDados01, aLstCmp01, @aItRet,;
626 @aMsgRet, aMainCfg, aLstC02, aLstC03, ;
627 aHeader02, @aDados02, @oPnlGrdP, @oGrpPrd,;
628 @oCbxCbxMet, @aCbxCbxMet , @oGrpGen, @aDados03,;
629 oBrwConten:nAt, @lGrdMsgAtv, @cTextoGen, @oLblGen,;
630 aHeader03, aAuxCombo, @oPnlGen, @cTextoBtn, ;
631 @oBtnCnc, @oBtnCnf, @lPosicionado) } //Botão de Seleção
632
633 Default cCampo := "aDados01[oBrwConten:nAt]"
634
635
636 If Len(aAuxCombo) > 0
637   aAdd(aLegenda,{"ENABLE" ,aAuxCombo[1][3]}) //Credito Liberado
638   aAdd(aLegenda,{"DISABLE" ,aAuxCombo[2][3]}) //Debito
639
640   //Adiciono as regras de cores no array a Cores
641   aAdd(aCores,{"ME1_STATUS", 'ME1_STATUS == "' + Alltrim(aAuxCombo[1,3]) + '"',aLegenda[1][1]})
642   aAdd(aCores,{"ME1_STATUS", 'ME1_STATUS == "' + Alltrim(aAuxCombo[2,3]) + '"',aLegenda[2][1]})
643 EndIf
644
645 For nX := 1 To Len(aDados01)
646
647   For nI := 1 to Len(aHeader01)
648
649     cTmp := aHeader01[nI][2]
650     If !Empty(cTmp)
651       Do Case
652         Case aHeader01[nI][5] == "N"
653           aDados01[nX, nI] := AllTrim(Transform(aDados01[nX, nI], cTmp))
654         Otherwise
655           aDados01[nX, nI] := Transform(aDados01[nX, nI], cTmp)
656       EndCase
657     Endif
658   Endif
659
660   Next nI
661
662 Next nX
663
664 oGetCrtUt := POSBrwContainer(o)
665
666 cTmp := ""
667
668
669
670 DEFINE FWBROWSE oBrwConten DATA ARRAY ARRAY aDados01 CHANGE { || lPosicionado := .F., IIf(ValType(oBtnConf) == "0", oBtnConf:Refresh(), ) } NO LOCATE NO CONFIG NO
671 REPORT OF oGetCrtUt

```

[illegible]

```

724             nGetQtde, cGetMetMet, lGrdMsgAtv, aCbxMet)
725
726 Local oGetCrtUt := NIL                //Conteudo
727 Local oColumn   := Nil                //Objeto do Grid
728 Local nX        := 0                  //Variavel de Laco
729 Local nI         := 0                  //Variaval do Laco
730 Local cTmp       := ""                 //Temporário Coluna
731 Local cTmp2      := ""                 //Temporário Título
732 Local bDbClick   := { || STIGLAtvCtrl(2, @cTexto,@oLblProd, @cTexto2, @cTexto3, ;
733                                     @oLblMet,@oGetProd, @oGetQtde, @oCbxMet ), ;
734                                     STIGPrdCg( @nGetQtde, oBrwConten2, aHeader02,aDados02, ;
735                                     @cGetMetMet, @oGetQtde, @oCbxMet ),;
736                                     lGrdMsgAtv := .T. } //Duplo Click do Browse
737 Local bChange    := { || STIGPrdCg( @nGetQtde, oBrwConten2, aHeader02,aDados02, ;
738                                     @cGetMetMet, @oGetQtde, @oCbxMet ) } //Change do Browse
739
740
741 Default cCampo := "aDados02[oBrwConten2:nAt]"
742
743 For nX := 1 To Len(aDados02)
744
745     For nI := 1 to Len(aHeader02)
746
747         cTmp := aHeader02[nI][2]
748         If !Empty(cTmp)
749             Do Case
750                 Case aHeader02[nI][5] == "N"
751                     aDados02[nX, nI] := AllTrim(Transform(aDados02[nX, nI], cTmp))
752                 Otherwise
753                     aDados02[nX, nI] := Transform(aDados02[nX, nI], cTmp)
754             EndCase
755         Endif
756     Endif
757
758 Next nI
759
760 Next nX
761
762 oGetCrtUt := POSBrwContainer(o)
763
764 cTmp := ""
765
766
767 DEFINE FWBROWSE oBrwConten2 DATA ARRAY ARRAY aDados02 CHANGE bChange DOUBLECLICK bDbClick NO LOCATE NO CONFIG NO REPORT OF oGetCrtUt
768
769 oBrwConten2:nRowHeight := 25
770 oBrwConten2:SetVScroll(.T.)
771 For nI := 1 to Len(aHeader02)
772     cTmp := &("{|| " + cCampo + "[" + AllTrim(Str(nI)) + " ]}")
773     cTmp2 := &("'" + AllTrim(aHeader02[nI, 03]) + "'")
774     ADD COLUMN oColumn DATA cTmp TITLE cTmp2 SIZE 008 OF oBrwConten2 //"Tipo"
775

```

[illegible]

```

828      aAdd(aLegenda,{"DISABLE"      ,aAuxCombo[2][3]}) //Debito
829
830      //Adiciono as regras de cores no array a Cores
831      aAdd(aCores,{"SEL", 'SEL' + Alltrim(aAuxCombo[1,3]) + ''},aLegenda[1][1])
832      aAdd(aCores,{"SEL", 'SEL' + Alltrim(aAuxCombo[2,3]) + ''},aLegenda[2][1])
833  EndIf
834
835  For nX := 1 To Len(aDados03)
836
837      For nI := 1 to Len(aHeader03)
838
839
840          cTmp := aHeader03[nI][2]
841          If !Empty(cTmp)
842              Do Case
843                  Case aHeader03[nI][5] == "N"
844                      aDados03[nX, nI] := AllTrim(Transform(aDados03[nX, nI], cTmp))
845                  Otherwise
846                      aDados03[nX, nI] := Transform(aDados03[nX, nI], cTmp)
847              EndCase
848          Endif
849
850      Next nI
851
852  Next nX
853
854  If lCriaCont
855      oGetContGen := POSBrwContainer(o)
856  Else
857      If ValType(oBrwConten3) == "0"
858          oBrwConten3:DeActivate(.t.)
859      EndIf
860  EndIf
861
862  cTmp := ""
863
864  DEFINE FWBROWSE oBrwConten3 DATA ARRAY ARRAY aDados03 DOUBLECLICK bDbClick NO LOCATE NO CONFIG NO REPORT OF oGetContGen
865
866  oBrwConten3:nRowHeight := 25
867  oBrwConten3:SetVScroll(.T.)
868  For nI := 1 to Len(aHeader03)
869      If nI <> nPosStatus
870          cTmp := &("{ | " + cCampo + " | " + AllTrim(Str(nI)) + " }")
871          cTmp2 := &("'+AllTrim(aHeader03[nI, 03]) +'")
872          ADD COLUMN oColumn DATA cTmp TITLE cTmp2 SIZE 008 OF oBrwConten3 //"Tipo"
873      Else
874          cTmp := &("{ | IIf( " + cCampo + " | " + AllTrim(Str(nI)) + " == '1','" + aLegenda[1][1]+'','" + aLegenda[2][1] + "' ) }")
875
876          ADD STATUSCOLUMN oColumn DATA cTmp DOUBLECLICK { | .T. } OF oBrwConten3
877      EndIf
878  EndIf
879

```

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

```

1192 EndIf
1193
1194 If aMainCfg[POS_MAME]
1195     cTexto2 := STR0020 //"Entrega"
1196     oCbxCbxMet:lVisible := .T.
1197 Else
1198     cTexto2 := ""
1199     oCbxCbxMet:lVisible := .F.
1200 EndIf
1201 oGetProd:lVisible := .F.
1202
1203 EndIf
1204
1205 oLblProd:Refresh()
1206 oLblMet:Refresh()
1207 oGetProd:Refresh()
1208 oCbxCbxMet:Refresh()
1209 oGetQtde:Refresh()
1210 EndIf
1211
1212 // Grava a opção selecionado para validar inclusão de produto não cadastrado na lista
1213 If nOpc == 1
1214     lIncluirPrd := .T.
1215     STIGlIncProd(lIncluirPrd)
1216 ElseIf nOpc == 2
1217     lIncluirPrd := .F.
1218     STIGlIncProd(lIncluirPrd)
1219 EndIf
1220
1221 Return
1222
1223
1224 //////////////////////////////////////
1225 /*{Protheus.doc} STIGlAltSt
1226 Funcao para alterar o Status do Grid de Mensagens
1227 @param      aDados03 - Dados das Mensagens
1228 @param      nPos - Posição do Grid de Mensagens
1229 @param      nPosStatus - Posicao do Campo Status
1230 @param      oBrwConten3 - Grid de Mensagens
1231 @param      aHeader03 - Header de Mensagens
1232 @return NIL
1233 @obs
1234 @sample
1235 */
1236 //////////////////////////////////////
1237 Static Function STIGlAltSt( aDados03, nPos, nPosStatus, oBrwConten3, ;
1238                             aHeader03)
1239     Local cValor := ""
1240     Local lHabilita := .F.
1241     Local nX := 0
1242
1243     If AllTrim(STDGLRtCol(2,"SEL",1,nPos,aHeader03,aDados03,.F.,.F.)) = "1"

```

```

1244 cRet := ""
1245 Else
1246     cValor := "1"
1247 EndIf
1248
1249 lHabilita := cValor == "1"
1250
1251 If lHabilita
1252     //Desabilita os demais
1253     For nX := 1 to Len(aDados03)
1254         aDados03[nX, nPosStatus] := ""
1255     Next
1256 EndIf
1257
1258 aDados03[nPos, nPosStatus] := cValor
1259 oBrwConten3.Refresh()
1260
1261 Return
1262
1263 /*{Protheus.doc} STIGLBTCan
1264 */
1265 Funcao do botão de cancelamento do Grid Mensagens/Atores Entregas
1266 @param      lGrdMsgAtv - Panel de Mensagens ativo
1267 @param      oPnlGen - Panel do Grid
1268 @param      aDados03 - Dados de Mensagens
1269 @param      aHeader03 - Header de Mensagens
1270 @param      aAuxCombo - Array de Status
1271 @param      oGrpGen - Grupo Genérico
1272 @param      cTextoGen - Texto Genérico
1273 @param      oLblGen - Label Genérico
1274 @param      cTextoBtn - Texto do botão
1275 @param      oBtnCnc - Botão Cancelar
1276 @param      oBtnCnf - Botão Confirmar
1277 @return NIL
1278 @obs
1279 @sample
1280 /*
1281 //XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
1282 Static Function STIGLBTCan(lGrdMsgAtv, oPnlGen, aDados03, aHeader03, ;
1283                             aAuxCombo, oGrpGen, cTextoGen, oLblGen, ;
1284                             cTextoBtn, oBtnCnc, oBtnCnf)
1285
1286 If lGrdMsgAtv
1287     //Grid de Mensagens Ativo - Encerra o Panel
1288     aMainCFG := nil
1289     STIExchangePanel( { || STIPanItemRegister() } )
1290 Else
1291     //Grid de Atores/Entregas - Chama o Grid de Mensagens
1292     STWGLGdGe( @oPnlGen, @aDados03, @aHeader03, aAuxCombo, ;
1293                @oGrpGen, @lGrdMsgAtv, @cTextoGen, @oLblGen, ;
1294                @cTextoBtn, @oBtnCnc, @oBtnCnf)
1295 EndIf

```

[illegible]

[illegible]

[illegible]